The CAEM Book

How Confidence-Aware Episodic Memory with Self-Improvement Actually Works

A from-the-ground-up teaching companion
built from the live code and the thesis, component by component

Md. Aksan Gony Alif
Department of Computer Science & Engineering
BRAC University

This book is a learning notebook, not the thesis. It is allowed to show code, draw intuition pictures, and walk through concrete examples, everything the formal report cannot. Read it to *understand* the system; read the thesis to *cite* it.

Each chapter is grounded in the live `caem/` source and cross-checked against the main report.

Contents

I Foundations 1

1 The Hallucination Problem, and Why CAEM Exists 2
- 1.1 What a hallucination actually is . . . 2
- 1.2 Scale does not fix it . . . 2
- 1.3 The three structural flaws . . . 3
- 1.4 Why existing fixes do not close the gap . . . 5
- 1.5 The architectural opportunity . . . 6
- 1.6 The scope of the claim . . . 7
- 1.7 What the rest of the book does . . . 7

2 The Big Picture: Eight Stages and a Loop 9
- 2.1 Two clocks, not one . . . 9
- 2.2 The per-query pipeline at a glance . . . 10
- 2.3 The tier asymmetry, the load-bearing branch . . . 12
- 2.4 A query, end to end . . . 12
- 2.5 What the pipeline hands back . . . 14
- 2.6 The cycle loop that wraps it all . . . 15
- 2.7 The map of the book . . . 16

3 Prerequisites: The Concepts We Need First 19
- 3.1 The generator: what a decoder language model does . . . 19
- 3.2 Embeddings: turning meaning into geometry . . . 20
- 3.3 Similarity search at scale: FAISS . . . 21
- 3.4 Calibration: making a score mean something . . . 22
- 3.5 Natural-language inference: entailment as a grounding test . . . 24
- 3.6 Low-rank adaptation: learning without forgetting . . . 25
- 3.7 How the toolbox maps onto the architecture . . . 26

II The Per-Query Pipeline 28

4 Stage 1: Pre-Routing Confidence ($u_{\mathbf{pre}}$) 29
- 4.1 What Stage 1 is for . . . 29
- 4.2 Signal one: token confidence (u_{token}) . . . 30
- 4.3 Signal two: internal convergence (c_{conv}) . . . 31
- 4.4 Fusing and calibrating the two signals . . . 32

4.5 The safety override . . . 33
4.6 Why Stage 1 is cheap by design . . . 34
4.7 The two readings, at the u_{pre} level . . . 34
4.8 An honest limitation . . . 35

5 Stage 2: The Episodic Memory Store 37
5.1 What a single memory holds . . . 37
5.2 The per-query operation: encode and probe . . . 38
5.3 Admission: the novelty gate . . . 39
5.4 Capacity and pruning . . . 39
5.5 Living metadata: trust that adapts to service . . . 40
5.6 What the store does at a cycle boundary . . . 41
5.7 The store survives a rollback . . . 42

6 Stage 3: The Three-Tier Router 44
6.1 What the router sees and what it returns . . . 44
6.2 Three mechanisms, in a fixed order . . . 45
6.3 Why the two tests are asymmetric . . . 46
6.4 Why three mechanisms instead of one threshold . . . 47
6.5 The router at work . . . 48

7 Stage 3b: The Retrieval-Utility Classifier 50
7.1 The wrong-default failure mode . . . 50
7.2 Where the classifier fires, and where it does not . . . 52
7.3 What the classifier is . . . 52
7.4 The features, by family . . . 53
7.5 The star feature: the self-knowledge probe . . . 54
7.6 How the classifier was trained . . . 55
7.7 Honest limitations . . . 56
7.8 The classifier in the lineage of selective retrieval . . . 57
7.9 The two readings, refined . . . 57

8 Stage 4: Tier Execution 58
8.1 The shared contract: scaffolded chain of thought . . . 58
8.2 Tier 1: serve from memory . . . 59
8.3 Tier 2: zero-shot generation . . . 60
8.4 Tier 3: retrieval-augmented generation . . . 61
8.5 The escalation edge . . . 62
8.6 What leaves Stage 4, and where it goes . . . 62

9 Stage 5: The Nine-Signal Verifier 64
9.1 Four families, nine headline signals . . . 64
9.2 How the signals are produced, cheaply . . . 65
9.3 Family one: internal calibration . . . 66

9.4 Family two: self-consistency . . . 66
9.5 Family three: external grounding . . . 67
9.6 Family four: relevance, and two entity guards . . . 68
9.7 The confabulation early-exit . . . 69
9.8 What the verifier emits . . . 70
9.9 Honest limitations . . . 71

10 Stage 6: The Calibrated Composite and Four-Outcome Gate 73
10.1 Why fusion is hard: scales and sign-flips . . . 73
10.2 Step one: turning one raw signal into a calibrated probability . . . 74
10.3 Step two: per-benchmark curves, shrunk toward a shared one . . . 76
10.4 Step three: combining nine probabilities into one . . . 78
10.5 How the composite is fitted: the training phase . . . 79
10.6 The whole computation, end to end . . . 82
10.7 The four-outcome gate . . . 83
10.8 Why one fixed threshold governs five benchmarks . . . 84

11 Stage 7: The Output Contract 86
11.1 Two answers, kept separate . . . 86
11.2 Mapping the decision to a display string . . . 86
11.3 The answer never travels alone: the confidence envelope . . . 87
11.4 Conflicting is not the same as insufficient . . . 88
11.5 Deferral speaks in the system's own clock . . . 89
11.6 Where the calibration mismatch is repaired . . . 89

III The Self-Improvement Loop 91

12 Stage 8: The Cycle-Boundary Self-Improvement Loop 92
12.1 Two clocks, in full . . . 92
12.2 What one cycle does, in order . . . 93
12.3 What it trains on: the verified episodes . . . 93
12.4 The fine-tune: a small adapter, a frozen model . . . 95
12.5 The retention guard: commit or abort . . . 96
12.6 The rest of the boundary, and what is gated on committing . . . 97
12.7 When the loop stops . . . 98
12.8 Why this closes the static flaw . . . 98

IV Theory and Evaluation 100

13 The Theory: Purity, Monotonicity, Convergence 101
13.1 What the theorems are about, and what they are not . . . 101
13.2 The data-purity result: why an imperfect verifier is enough . . . 102
13.3 Monotonicity: a better generator stores more . . . 104

13.4 Convergence: the gap contracts to a stable band . . . 105
13.5 The Tier-1 floor: memory hallucination is bounded by pool impurity . . . 105
13.6 Self-correction: bad episodes are pruned in finite time . . . 106
13.7 Stochastic equilibrium: learning under a guard that sometimes says no . . . 107
13.8 The results, assembled . . . 108

14 Benchmarks, Baselines, and Metrics 110
14.1 The benchmark panel . . . 110
14.2 The baselines: what we compare against . . . 113
14.3 The metrics . . . 115
14.4 Reading the panel: what good looks like . . . 121

15 The Realised Trajectory: Cycles 0 to 5 125
15.1 The shape of the run . . . 125
15.2 Accuracy, benchmark by benchmark . . . 126
15.3 How the gate shifts . . . 127
15.4 The three tiers in motion . . . 128
15.5 Retention, and the cycle that was thrown away . . . 129
15.6 The honesty lens on the misconception benchmark . . . 129
15.7 Inside the loop, and when it stopped . . . 130
15.8 What the theory predicted, and what the run did . . . 131

16 Ablations: What Each Piece Is Worth 132
16.1 The ablation registry, and the claim it defends . . . 132
16.2 Ablating the whole architecture, and the self-improvement slice . . . 133
16.3 The retrieval-utility classifier, switched off . . . 134
16.4 The forgetting guard, ablated by the run itself . . . 135
16.5 The lesion still owed, and the honest ledger . . . 136

V Reference 138

A The Registered-Constant Registry 139
A.1 Backbone and generation . . . 139
A.2 Episodic memory . . . 140
A.3 Retrieval and routing . . . 140
A.4 Retrieval-utility classifier . . . 141
A.5 The nine-signal verifier . . . 141
A.6 Composite and four-outcome gate . . . 141
A.7 Self-improvement and retention . . . 142
A.8 Benchmarks, evaluation, and metrics . . . 142

B Glossary of Symbols and Terms **144**
B.1 Symbols 144
B.2 Terms 146

References **149**

PART I

Foundations

CHAPTER 1

The Hallucination Problem, and Why CAEM Exists

Before we touch a single component, we need to feel the problem clearly. Every design choice in CAEM, namely the memory, the nine signals, the calibrated gate, the self-improvement loop, and the rollback, is a direct response to a specific failure of ordinary language models. Once we understand that failure, the architecture stops looking like a pile of clever tricks and starts looking like the *only* sensible response. That is the goal of this chapter.

Intuition

A language model is a fluent guesser. It is extraordinarily good at producing text that *sounds* right. The trouble is that "sounds right" and "is right" are different properties, the model has no built-in way to tell them apart, and, worse, no way to signal which is which to a reader. A confident lie and a confident truth come out of the model wearing the same clothes. CAEM exists to put a disciplined system around the guesser, so that the answers it commits to are trustworthy, it knows how much to trust each one, and it gets better over time without forgetting what it already knew.

1.1 What a hallucination actually is

A *hallucination* is a model output that is fluent and confident but factually wrong. The word covers a range of causes, from a retrieval system that missed, to a genuine gap in the model's knowledge, to outright confabulation under uncertainty. The symptom is always the same: the model states something untrue as though it were true.

The danger is not the wrongness by itself. We tolerate wrong answers from search engines, from textbooks, and from each other. The danger is the **calibration mismatch**: the model expresses the same high confidence whether it is right or wrong, so a reader with no independent way to check cannot tell which answers to trust. A false answer phrased like a true one is harder to catch than a false answer that sounds unsure. In a casual chat that is an annoyance. In a clinical, legal, or financial setting it is a hazard.

1.2 Scale does not fix it

The first instinct of the field was that hallucination is a small-model problem that scale will erase. The evidence says otherwise.

- On **TruthfulQA**, a benchmark deliberately built around questions where humans hold

confident misconceptions, the largest frontier model evaluated reached only **58%** on the truthfulness axis and **25%** on the combined truthful-and-informative axis, against a human baseline of **94%** on both [1].

- The same benchmark documented **inverse scaling**: larger models were *more*, not less, prone to echo human falsehoods. Bigger made the result worse on exactly the questions that matter [1].
- On **long-form generation**, the FactScore atomic-fact decomposition protocol found that only **58%** of decomposed atomic claims in chat-style biographical generation were supported by evidence; the rest ranged from unsupported speculation to outright fabrication [2].
- In **clinical question answering**, frontier models hallucinated in **69–77%** of responses, an order of magnitude above any threshold a responsible deployment would tolerate [3].

A number that appeared on the poster

An earlier draft of the poster quoted a "3 to 91 percent" deployment-domain range. That specific range is *not* in the thesis, and we do not use it here. The grounded figures are the four above. If the 3–91% number comes up, it should be treated as a loose summary rather than a thesis claim. This book uses only what the report actually establishes.

The survey literature draws the conclusion bluntly: hallucination is "a structural property of stateless generation under uncertainty rather than a defect to be trained away" [4, 5]. That sentence rewards a second reading. It says the problem is not in the *weights*; it is in the *way the model is used*. A bigger guesser is still a guesser. **Any system that aims to reduce hallucination must change the structure, not just train harder.** That single sentence is the seed of the entire thesis.

1.3 The three structural flaws

The thesis pins the structural problem to three specific shortcomings of the standard inference paradigm. These three words, **stateless**, **overconfident**, and **static**, are worth committing to memory, because every part of CAEM maps back to one of them.

Flaw 1: Stateless

Standard inference processes every query independently. No representation of past successful answers is kept across sessions, and the same question asked twice can produce two different answers. A reasoning chain that produced a correct answer this morning is discarded the moment the answer is delivered. This wastes computation on questions the system has effectively already solved, and it prevents any accumulation of domain knowledge through use. The model never gets to say "I have seen this before, and last time I got it right."

Flaw 2: Overconfident (miscalibrated)

Modern neural networks are systematically overconfident. Calibration studies report that both classifiers and language models assign high probability to incorrect outputs at rates that exceed their actual accuracy by **ten percentage points or more** [6, 7]. The obvious fix, reading off the model's own confidence, does not work either: single-signal correctness predictors achieve area-under-the-ROC scores between only **0.50 and 0.60**, barely above random chance [8]. The consequence is the one we opened with: a user cannot reliably tell which answers are likely wrong from confidence alone.

Flaw 3: Static

Once trained, a deployed model neither learns from its mistakes nor adapts to the institution using it, because naive online fine-tuning incurs **catastrophic forgetting**: it improves on new material by erasing the broad capabilities the base training instilled [9]. The implication is stark. Verification effort spent today produces no compounding benefit tomorrow. A thousand answers verified by humans this morning will not change a single answer the model gives this afternoon.

Why the three are a knot, not a list

The crucial observation is that these flaws **reinforce one another**. They are not three independent bugs that could be fixed one at a time; they form a cycle.

- Without **memory**, the model cannot reuse verified knowledge.
- Without **calibrated confidence**, the system cannot recognise when memory is missing and worth consulting, so even a perfect memory would be under-used.
- Without **self-improvement**, the verifications that humans or external systems do produce decay into one-shot interventions.

This is why fixing one flaw alone fails: each unsolved flaw pulls the solved one back toward the broken state. Figure 1.1 draws the cycle. The whole architecture of CAEM is the claim that the cycle can be broken along all three axes at once: memory for statelessness, a calibrated verifier for overconfidence, and a guarded self-improvement loop for staticness.

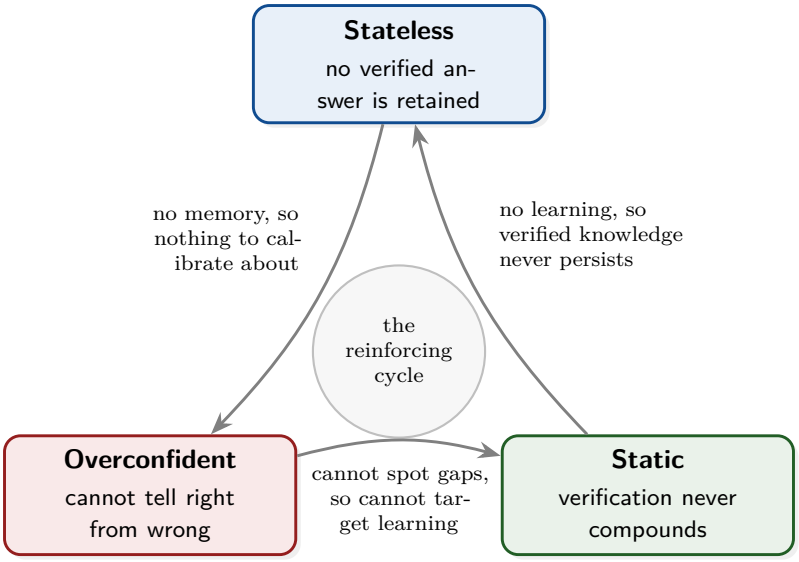


Figure 1.1: The three structural flaws form a reinforcing cycle, not an independent list. Fixing any one in isolation leaves the other two to drag it back. The thesis of CAEM is that they must be attacked *simultaneously*. (Schematic.)

1.4 Why existing fixes do not close the gap

Three families of prior systems each attack hallucination, and each has a structural limitation that prevents it from closing the gap alone.

Retrieval-augmented generation (RAG).
RAG grounds the model in retrieved evidence [10], which raises factual recall *when retrieval succeeds*. It is silent, however, when retrieval misses or returns evidence the model summarises incorrectly. The output is still a single-pass generation whose quality is capped by retrieval quality, and the system keeps *no record of which past answers were correct*. RAG does not touch statelessness.

Prompting interventions.
Chain-of-thought reasoning and self-consistency over sampled chains [11, 12] expose internal reasoning and exploit agreement across resampled answers. They provide *no persistence*, however: each session begins with no memory of prior verified answers, and the same question tomorrow triggers the same reasoning effort as today. They make a single answer better; they do not make the *system* better.

Post-generation verification.
Methods such as SelfCheckGPT and semantic entropy [13, 14] detect hallucinations after they occur and support refusal or revision. The verdict *does not feed back*, however, into the model's weights or any memory, so the identical hallucination recurs in the next session. Verification without a feedback loop is a one-shot intervention.

The pattern across all three

RAG fixes grounding but not statelessness. Prompting fixes single-answer quality but not persistence. Verification detects errors but never feeds the fix back. Each family solves a different *slice* of the problem and leaves the rest untouched, which is exactly why stacking "a stronger retriever, longer chains, a sharper verifier" yields diminishing returns. None of them closes the loop.

The thesis states the implication directly: hallucination reduction at deployment-relevant rates "is unlikely to come from incremental improvement of any one family." What is missing is **a closed loop**, a system in which verification feeds memory, memory feeds routing, routing feeds generation, and improvement compounds across cycles instead of evaporating at session boundaries. Arguing for that system, and building it, is the work of the thesis.

1.5 The architectural opportunity

If the three flaws are a knot, the opportunity is that three capabilities, taken *together*, untie it. CAEM commits to exactly three, each aimed at one flaw.

1. **A verified episodic memory with a four-outcome admission rule**, finer than a single binary threshold. Outputs are sorted into store, defer, abstain, or discard, so borderline answers wait in a deferred buffer *without contaminating the high-precision pool*. This attacks statelessness.

2. **Confidence-aware routing with a retrieval-utility decision**. A calibrated pre-generation confidence and a learned classifier decide *how hard to work* and *whether retrieval will even help* on each query. This attacks overconfidence.

3. **A constrained self-improvement loop with retention rollback**. The model re-trains on its own verified answers each cycle, but a guard reverts any update that erodes prior capability. This attacks staticness.

Taken together, these convert the two statelessness problems into two *stateful* mechanisms: the verified store absorbs recurring queries at lookup cost, and the self-improvement loop folds verified retrieval-augmented reasoning into the model's own parametric capacity. The three serving tiers, memory, zero-shot, and retrieval, stop being fixed cost categories and become a **self-organising compute ladder** under the cycle loop.

The data-purity precondition, or why an imperfect verifier is enough

The natural objection is this: if the verifier is imperfect, will it not admit wrong answers and poison the memory? The thesis answers with a base-rate argument. Under reasonable conditions on the verifier's balanced accuracy and the base generation accuracy, **the purity of the stored pool can exceed the verifier's own accuracy**, because base-rate dynamics favour the gate whenever the verifier is more likely to admit a correct answer than an incorrect one. The precondition is mild: the verifier's balanced accuracy need only exceed one half on the distribution of claims it actually

scores. We prove and test this in Chapter 13. For now, the intuition to hold is that a perfect verifier is not required, only one that is better than a coin flip on the claims it sees. On the realised system this precondition is met with room to spare: the deployed verifier's balanced accuracy on the calibration-matched distribution is about 0.67, and the stored pool's realised purity sits near 0.74, above the verifier's own accuracy exactly as the identity predicts (the proof is in Chapter 13 and the empirical receipt in the evaluation chapter).

1.6 The scope of the claim

So the thesis does not get to claim that CAEM solves hallucination. It claims something sharper and falsifiable. The intervention is bounded so that it can be argued empirically within an undergraduate research budget.

- **One small frozen generator**: Qwen-2.5-3B-Instruct, a three-billion-parameter open-weight instruction-tuned decoder. The backbone is *frozen*; only a low-rank adapter on the attention and feed-forward projections trains, at roughly **2%** of the parameter count.
- **One consumer GPU**: the whole system fits on a single thirty-two-gigabyte graphics processor under sixteen-bit precision, so the result is reproducible without specialised infrastructure.
- **Five benchmarks**: a three-benchmark *training panel* (FEVER, TriviaQA, CommonsenseQA) and a two-benchmark *transfer panel* (TruthfulQA, StrategyQA), with multitask language understanding (MMLU) held out as an out-of-distribution retention control rather than an optimisation target.
- **A fixed open generator**, with open verification machinery and no external commercial calls, so the architectural claim is separable from any pre-training advantage and reproducible end-to-end.

The one-sentence framing of the whole thesis

"Hallucination is a structural property of stateless, overconfident, static generation, not a defect that scale removes. CAEM attacks all three at once on a fixed small model: a verified episodic memory for statelessness, a calibrated nine-signal verifier for overconfidence, and a guarded self-improvement loop for staticness; and we show the gains compound across cycles while a retention guard prevents regression."

1.7 What the rest of the book does

We now have the *why*. The rest of the book is the *how*, built bottom-up.

- Chapter 2 gives the whole machine in one picture, the eight-stage per-query pipeline and the cycle loop that wraps it, so we have a map before zooming in.

- Chapter 3 fills in the concepts the later chapters lean on (embeddings, calibration, natural-language inference, low-rank adaptation, and similarity search), so the book is self-contained.
- Each stage then gets its own chapter, grounded in the live code: what it does, the exact mechanism, a worked example, the traps, and how to defend it.

It helps to keep Figure 1.1 in mind throughout. Whenever a component seems elaborate, we can ask which of the three flaws it is breaking the cycle on. The answer is always one of the three.

CHAPTER 2

The Big Picture: Eight Stages and a Loop

Chapter 1 argued the *why*: hallucination is a structural property of stateless, overconfident, static generation, and the only honest response is a system that attacks all three at once. This chapter gives the *shape* of that system in a single sitting. We will not prove anything here and we will not open any one component. The goal is a reliable mental map, so that when a later chapter spends thirty pages on the verifier or the router, we already know where that piece sits, what feeds it, and what it feeds.

The whole machine in three sentences

CAEM runs every question through a short assembly line that decides how hard to work, produces an answer, and then judges that answer before deciding whether to trust it, refuse, or file it away for later. That assembly line is the **per-query pipeline**, and it runs thousands of times without ever changing the model. Wrapped around it is a slower **cycle loop** that periodically retrains the model on the answers the pipeline judged trustworthy, with a guard that undoes any retraining that makes the model worse. One fast clock answers questions; one slow clock makes the answerer better. Everything in the book is a detail of one of those two clocks.

2.1 Two clocks, not one

The single most important idea in this chapter is that CAEM has *two independent clocks*, and almost every later confusion dissolves once we keep them apart.

The per-query clock (Stages 1 to 7).
One tick is one question. The system encodes the query, decides which of three tiers should handle it, generates or retrieves an answer, scores that answer with a verifier, and applies a four-way decision. This clock *reads* the current memory, the current model weights, and the current calibration, and at most it *writes one episode* into memory. It never trains anything. It ticks thousands of times per cycle.

The per-cycle clock (Stage 8).
One tick is one *cycle boundary*, reached after a batch of queries has been processed. At a boundary the system fine-tunes the model on its own verified answers, re-checks that it has not forgotten anything, recalibrates the verifier, and re-scores the stored memory. This clock *mutates* the weights, the calibration, and the consolidated memory. It ticks a handful of times across the whole run, six times in the realised trajectory of Chapter 15.

The two clocks are nested: the per-cycle clock is the outer loop, and each of its ticks drives the per-query clock thousands of times in batch. Stages 1 through 7 are where the model, the calibration, and the memory are merely *consumed*; Stage 8 is the only place they are *changed*. Figure 2.1 draws the nesting; the rest of the chapter walks the inner clock first, then the outer.

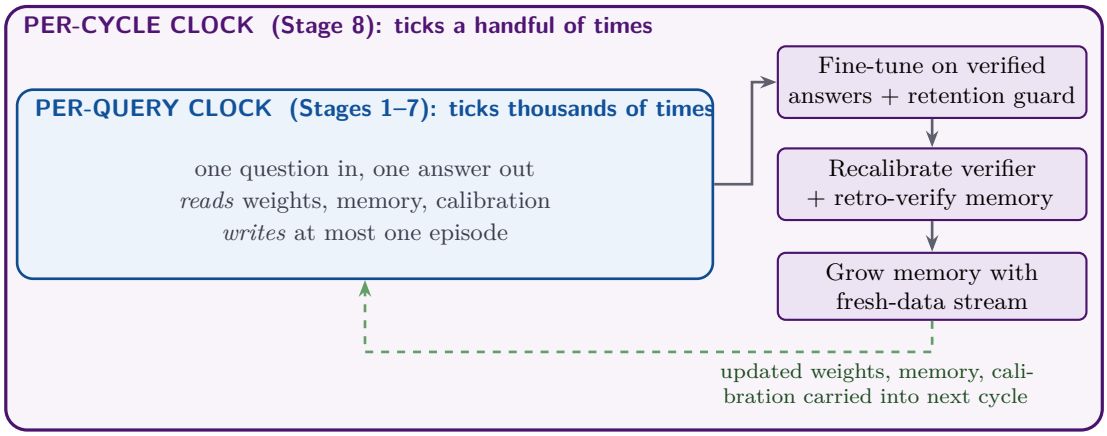


Figure 2.1: The two clocks of CAEM. The inner per-query pipeline (Stages 1 to 7) answers questions without ever training. The outer per-cycle loop (Stage 8) is the only place the weights, calibration, and consolidated memory change. Each cycle boundary fine-tunes the model on the answers the pipeline judged trustworthy, then hands the updated state back to the inner clock. (Schematic.)

2.2 The per-query pipeline at a glance

A single question enters through one public entry point and leaves as one structured record. Everything between is a fixed sequence of stages. Figure 2.2 is the master diagram of the book; it is worth a long look, because every chapter in Parts II and III is a zoom into one of its boxes.

The seven inner stages in one line each

Reading Figure 2.2 left to right, with the chapter that opens each box:

1. **Pre-routing confidence** (u_{pre}, Chapter 4). A single short forward pass, with no full generation, produces a calibrated guess at how likely the model is to answer this query correctly on its own. It fuses a token-probability signal with an encoder-convergence signal.

2. **Encode and probe** (Chapter 5). The query is turned into a 768-dimensional embedding, which is used both to search the episodic memory for the single nearest stored episode and, later, to ground the verifier.

3. **Route** (Chapter 6). The retrieved similarity and the stored answer's quality are fused into one dispatch score that selects a tier, with a safety override that escalates whenever u_{pre} falls below a fixed floor.

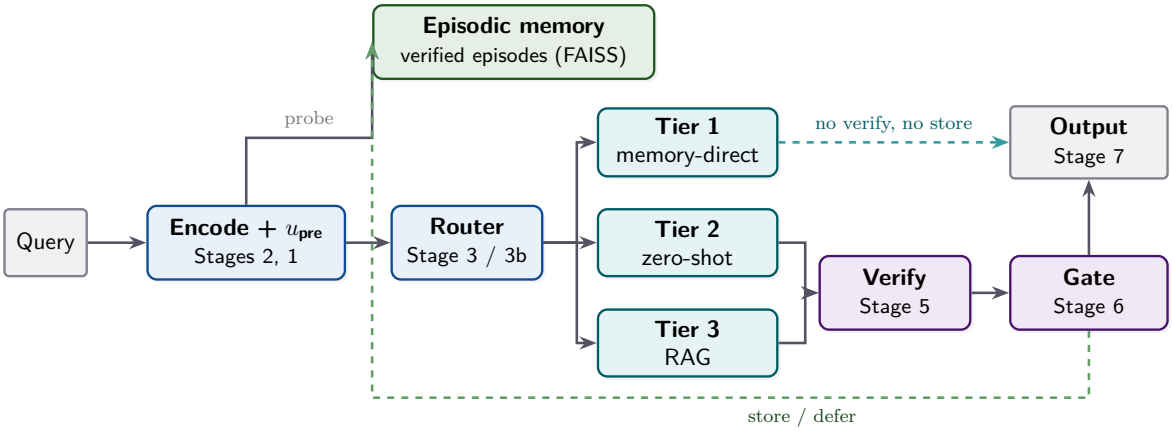


Figure 2.2: The per-query pipeline. A query is encoded and scored for confidence, the router picks one of three tiers, and the chosen tier produces an answer. Tier 1 (a memory hit) returns its stored answer directly, skipping verification and storage entirely. Tiers 2 and 3 send their generated answer through the nine-signal verifier and the four-outcome gate, which may return it, refuse it, or write it back into memory. The dashed green path is the only way an episode enters memory; the dashed teal path is the only tier that bypasses the judge. (Schematic.)

4. **Refine with the retrieval-utility classifier** (Stage 3b, Chapter 7). When the router would default to retrieval, a small learned classifier decides whether retrieval will actually help this query, or whether the model is better off answering zero-shot. This is the centrepiece of the second version of the system.

5. **Execute the tier** (Chapter 8). Tier 1 serves the stored answer; Tier 2 generates zero-shot on the fine-tuned model; Tier 3 retrieves passages from a large public corpus and generates a grounded answer.

6. **Verify** (Chapter 9). Every generated answer, but never a Tier 1 hit, is scored on nine signals across four families: token and dropout uncertainty, internal-state probes, semantic consistency across resampled chains, and entailment against retrieved evidence.

7. **Decide and output** (Chapters 10 and 11). The nine signals are folded into one calibrated probability, the four-outcome gate maps that probability to *store*, *defer*, *abstain*, or *discard*, and the output stage returns the answer or a calibrated refusal.

Where this lives, and one honest subtlety

The entire inner pipeline is one method, `CAEMPipeline.answer(query, store_to_memory, source_benchmark)` in `caem/pipeline.py`. It is the single public per-query entry point and it returns one `PipelineResult` object. Two details are worth flagging now so they do not surprise us later. First, the stage *numbers* are a conceptual taxonomy, not the execution order: in the actual code the query is encoded, then scored for u_{pre}, then the memory is searched, then the route is chosen. We draw the diagram in code order. Second, an older design carried a separate post-generation confidence gate; it was retired, and the verifier of Chapter 9 is now the single source of truth after generation. Where the code comments still mention that retired gate, they are stale.

2.3 The tier asymmetry, the load-bearing branch

If only one fact from this chapter survives, it should be this one. The three tiers are not symmetric, and the asymmetry is the whole point of the design.

- **Tier 1** is a memory hit. The router found a stored episode close enough to the query that its verified answer can be served directly. There is *no generation* and *no verification*: the answer and its already-known trust score are read straight off the stored entry, and the pipeline returns in well under half a second. A Tier 1 answer was already judged trustworthy on the cycle it was stored, so re-judging it would be wasted work.
- **Tier 2 and Tier 3** are fresh generations, zero-shot and retrieval-augmented respectively. A fresh generation has never been judged, so it *must* pass through the verifier and the gate before the system will trust it or store it.

This is the branch that turns the three tiers into a self-organising compute ladder. Cheap, repeated questions collapse onto Tier 1 over time as their answers get memorised; novel or hard questions climb to Tiers 2 and 3 and pay the full verification cost. The cost of trust is paid once per fact, not once per ask.

Tier 1 is not "the model is sure"; it is "we have seen this answered well before"

It is tempting to read the tiers as a confidence ladder, with Tier 1 meaning "the model is most confident." That is wrong. Tier 1 means a *stored episode* matches the query closely enough, and that episode carries the trust score it earned when it was first verified. The confidence that the *current* query is well served comes from the match quality and the stored score together, not from a fresh judgement of the answer. This is exactly why Tier 1 can skip the verifier: the judging already happened, on a past cycle, and its verdict was written into the episode.

One escalation edge is worth noting on the diagram. If a Tier 2 generation comes back empty or fails, the pipeline quietly escalates that single query to Tier 3 and marks the result as escalated. The reported tier still reads two, but the answer came from retrieval. It is a small robustness valve, not a separate stage.

2.4 A query, end to end

Abstractions settle once we watch a real query move through the machine. Two scenarios follow, the first a claim that ends up stored, the second a question that ends up refused. The refusal scenario is an actual record from a cold-start pipeline pass, reproduced with its logged signals. The stored-claim scenario is an illustrative walkthrough of the defer-then-promote path, hand-authored to make that two-cycle behaviour concrete rather than transcribed verbatim from a single record. Together they exercise the two behaviours that define the architecture: promoting a tentatively answered claim to memory after later review, and turning a low-confidence question into an honest refusal rather than a fabricated answer.

A claim that gets stored: "The French Revolution began before 1792" (FEVER)

The task is to label the claim as *supports*, *refutes*, or *not enough information*.

1. **Pre-routing.** The fused, temperature-scaled confidence comfortably clears the safety floor, so no override fires.
2. **Encode and probe.** On this cycle-zero pass the memory is empty, so there is no neighbour to retrieve.
3. **Route.** With nothing in memory, the router defaults the query toward the retrieval tier.
4. **Refine (RUC).** The retrieval-utility classifier overrides that default to the *zero-shot* tier, judging the claim short, self-contained, and unlikely to benefit from passage retrieval.
5. **Execute.** Tier 2 generates the label *supports*.
6. **Verify.** The nine signals read as follows: top-passage entailment is high (above 0.94), the pooled-mean and atomic entailment are lower because only part of the supporting evidence sits in the context window, semantic consistency across resampled chains is high (above 0.85), and question-answer relevance is moderate (around 0.69). Folded together, the calibrated composite is about 0.51.
7. **Decide.** A composite of 0.51 clears the deferral threshold ($\tau_{\text{defer}} = 0.45$) but falls just below the storage threshold ($\tau_{\text{store}} = 0.60$), so the entry is parked in the *deferred buffer* for reconsideration at the next cycle boundary, rather than stored or discarded.
8. **Cycle boundary (Stage 8).** After the model is fine-tuned, the retroactive re-verification pass re-scores the same entry under the improved model. This time it crosses τ_{store} and is promoted into the cold-start memory as entry identifier zero.
9. **Output.** The user receives the label *supports*, which is correct.

The lesson: a borderline-but-promising answer is neither trusted blindly nor thrown away. It waits, and a later, better model decides its fate.

A question that gets refused: "'If I Ruled The World' comes from which musical?" (TriviaQA)

1. **Pre-routing.** The pre-routing confidence lands around 0.66, comfortably above the safety floor, so no override fires. On this cold-start pass the episodic memory is still empty, so no cheap memory path can be earned and the query is scored normally.
2. **Refine (RUC).** Consulted on the same query, the classifier confirms the retrieval tier: the base model's self-knowledge probe reads low, and the dense passage corpus does carry supporting evidence.
3. **Execute.** Tier 3 retrieves passages and runs a grounded generation, which returns

an *empty* answer, an explicit refusal signal.

4. **Verify.** The signals on the empty answer are damning: token-level uncertainty is very low (around 0.02), top-passage entailment is low (around 0.11), atomic entailment is low (around 0.09), and question-answer relevance sits at chance (around 0.50). The calibrated composite is about 0.13, well below both thresholds.
5. **Decide.** The composite fails both the storage and deferral cut-points, and the top-passage entailment falls below the registered abstention floor, so the gate returns *abstain* rather than a silent discard.
6. **Output.** The user sees the registered refusal string, and the raw model prediction is suppressed from the user surface.

The lesson: when the evidence does not support an answer, the system says so, instead of emitting a confident guess. This is the calibration mismatch of Chapter 1 being repaired in practice.

2.5 What the pipeline hands back

Each tick of the per-query clock returns one structured record, and it is worth knowing its shape, because the rest of the system, the evaluation harness, and the cycle loop all read from it. The record carries the raw answer string kept verbatim for scoring, the tier that handled the query, a flag for whether the answer was stored, the full routing decision, the pre-routing confidence, the complete verifier record with all nine signals and the gate decision, the single trust scalar, the memory identifier if one was written, an escalation flag, and a display answer that maps an abstention to a refusal string and a discard to an empty surface.

The four gate outcomes and their thresholds

The gate of Chapter 10 maps the calibrated composite to one of four outcomes. Holding the locked operating point in mind makes the scenarios above exact rather than approximate.

- **Store** when the composite is at least $\tau_{\text{store}} = 0.60$: write the episode into high-precision memory.
- **Defer** when it lands in $[\tau_{\text{defer}}, \tau_{\text{store}}) = [0.45, 0.60)$: park it in the deferred buffer for reconsideration next cycle.
- **Abstain** when it is below τ_{defer} *and* an abstention condition holds (for instance grounding below its floor): return a calibrated refusal.
- **Discard** otherwise: drop the answer silently without storing.

A single fixed pair of thresholds works across five very different benchmarks because the signals are *per-benchmark calibrated* before they are compared, a point we develop in Chapter 10.

2.6 The cycle loop that wraps it all

Now the outer clock. A cycle boundary is not a single step but an ordered block, and the order matters because some steps depend on the model having actually improved. Figure 2.3 sketches the block; the precise mechanics are the subject of Chapter 12.

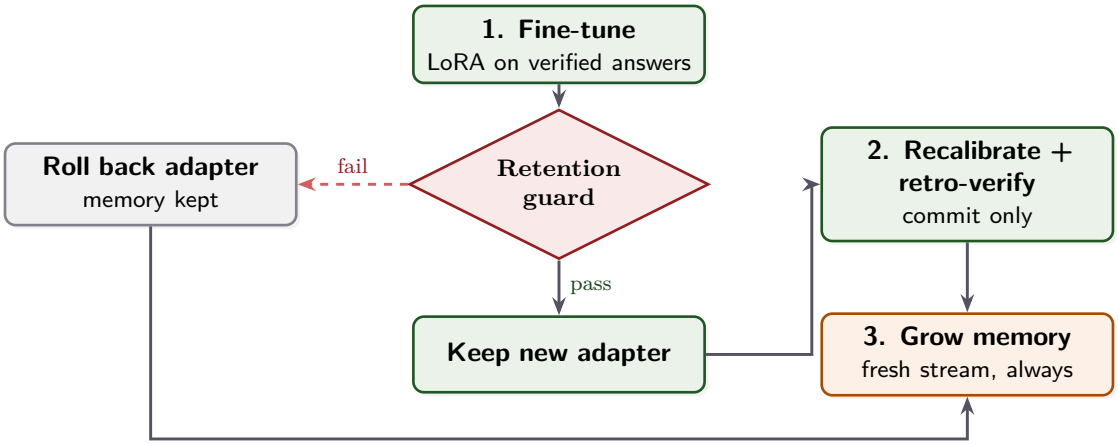


Figure 2.3: The cycle boundary. The model is fine-tuned on its own verified answers, then a retention guard checks it has not regressed on held-out probes. On a pass (green) the new adapter is kept, the verifier is recalibrated, and the stored memory is re-scored. On a fail (red) the adapter is rolled back to the previous cycle, and recalibration and re-verification are skipped. Either way (orange), the memory grows with the cycle's fresh data. This asymmetry, weights revert but memory persists, is the core safety property. (Schematic.)

The commit-versus-abort fork

The retraining step is the only place the model can get worse, so it is the only place the system is willing to undo its own work. After fine-tuning, the loop runs a set of retention probes on held-out material and compares the worst probe against a pristine baseline anchored once at the very start. If any probe ratio drops below the retention floor ($\rho_{\min} = 0.93$), the cycle is declared an *abort*: the freshly fine-tuned adapter is thrown away and the previous cycle's adapter is restored.

Asymmetric rollback, the single most important safety property

On an abort, the model weights revert, but the episodic memory does *not*. Everything the pipeline stored this cycle survives; only the parametric update is undone. This asymmetry is deliberate. Memory growth is monotone and high-precision by construction (each episode passed the gate), so it is safe to keep. A weight update, by contrast, is a global change that can quietly erode unrelated capabilities, which is exactly the catastrophic forgetting of Chapter 1. So the cheap, reversible, local thing (the adapter) is the thing we are willing to throw away, and the expensive, accumulated, verified thing (the memory) is the thing we protect. The realised trajectory hit this fork for real: at cycle four a retention probe fell to 0.886, below the 0.93 floor, so the adapter rolled back to its cycle-three state while the memory carried forward untouched.

What carries across a boundary, and what resets

The asymmetry generalises into a clean rule about state. Four things persist across cycle boundaries, and several things are rebuilt fresh each cycle.

Carries forward.
The **episodic memory** (always, even on abort); the **adapter weights** (on commit they advance, on abort they revert to the last good state); the **calibration**, namely the temperature and the per-signal curves (refreshed on commit, retained on abort); and the **deferred buffer** of borderline answers awaiting reconsideration.

Resets each cycle.
The **training pool** is rebuilt fresh from the current memory; the cycle consumes a **fresh, disjoint slice** of questions it has never seen; and the optimiser and dataloader are torn down and rebuilt. The **pristine retention baseline** is the one deliberate exception: it is anchored once at the start and never re-anchored, because re-anchoring it against an already-trained model would hide the very regression it exists to catch.

An abort is not a no-op cycle

It is easy to assume that an aborted cycle does nothing. It does a great deal. Only the *commit-gated* half is skipped: the weight update, the recalibration, the retroactive re-verification, the deferred-buffer reconsideration, and the memory consolidation. The *unconditional* half runs regardless: the pipeline still processes the cycle's fresh data and grows the memory, still runs the held-out evaluation, still checkpoints memory and the deferred buffer, and still consults the early-stop gate. An abort costs the system its weight update for that cycle, not its progress.

2.7 The map of the book

With the shape in hand, the rest of the book is a guided descent into each box. Table 2.1 is the index to keep a thumb on: every stage and its chapter.

Table 2.1: Where each stage of Figure 2.2 and Figure 2.3 is opened in full.

Stage	What it decides	Chapter
1	Pre-routing confidence u_{pre}: how likely is a correct self-answer?	Chapter 4
2	Episodic memory: encode the query and find the nearest stored episode	Chapter 5
3	Router: fuse similarity and stored quality into a tier choice	Chapter 6
3b	Retrieval-utility classifier: will retrieval actually help here?	Chapter 7
4	Tier execution: serve from memory, generate, or retrieve and generate	Chapter 8
5	Nine-signal verifier: score the generated answer's trustworthiness	Chapter 9
6	Calibrated composite and four-outcome gate: store, defer, abstain, discard	Chapter 10
7	Output contract: return the answer or a calibrated refusal	Chapter 11
8	Cycle boundary: fine-tune, guard, recalibrate, re-verify, grow	Chapter 12

Part IV then steps back from the mechanism. Chapter 13 proves the data-purity property that Chapter 1 promised, namely that a verifier only modestly better than chance still yields a memory pool cleaner than the verifier itself. Chapter 14 lays out the five benchmarks, the baselines, and the hallucination metrics. Chapter 15 walks the realised run from cycle zero to cycle five, including the cycle-four abort we have already met. Chapter 16 measures what each piece is actually worth by removing it. The appendices collect every registered constant (Chapter A) and a glossary of every symbol (Chapter B).

If an examiner asks: "what is the one-paragraph summary of how it works?"

CAEM answers each question through a seven-stage pipeline that first estimates how confidently it can answer, then routes the query to one of three tiers, memory lookup, zero-shot generation, or retrieval-augmented generation, picking the cheapest tier that will do. Generated answers, but not memory hits, pass through a nine-signal verifier whose calibrated composite gates a four-outcome decision: store the answer, defer it, refuse, or discard it. That pipeline never trains. Around it, a cycle loop periodically fine-tunes the model on the answers it judged trustworthy, under a retention guard that rolls back any update which degrades held-out performance, while the verified memory it accumulated persists regardless. Two clocks: one answers, one improves, and a guard keeps improvement from becoming regression.

For a single reference that holds the whole machine at once, Figure 2.4 reproduces the complete eight-stage pipeline on one page exactly as the main report draws it, every stage, formula, and threshold in one view. It is dense by design: the rest of the book is a guided tour of its boxes, and it rewards returning to as the trajectory and ablation chapters refer back to specific stages.

We now have the map. The next chapter fills in the handful of background concepts, namely embeddings, calibration, natural-language inference, low-rank adaptation, and similarity search, that the per-stage chapters lean on, so that the descent into each box is self-contained.

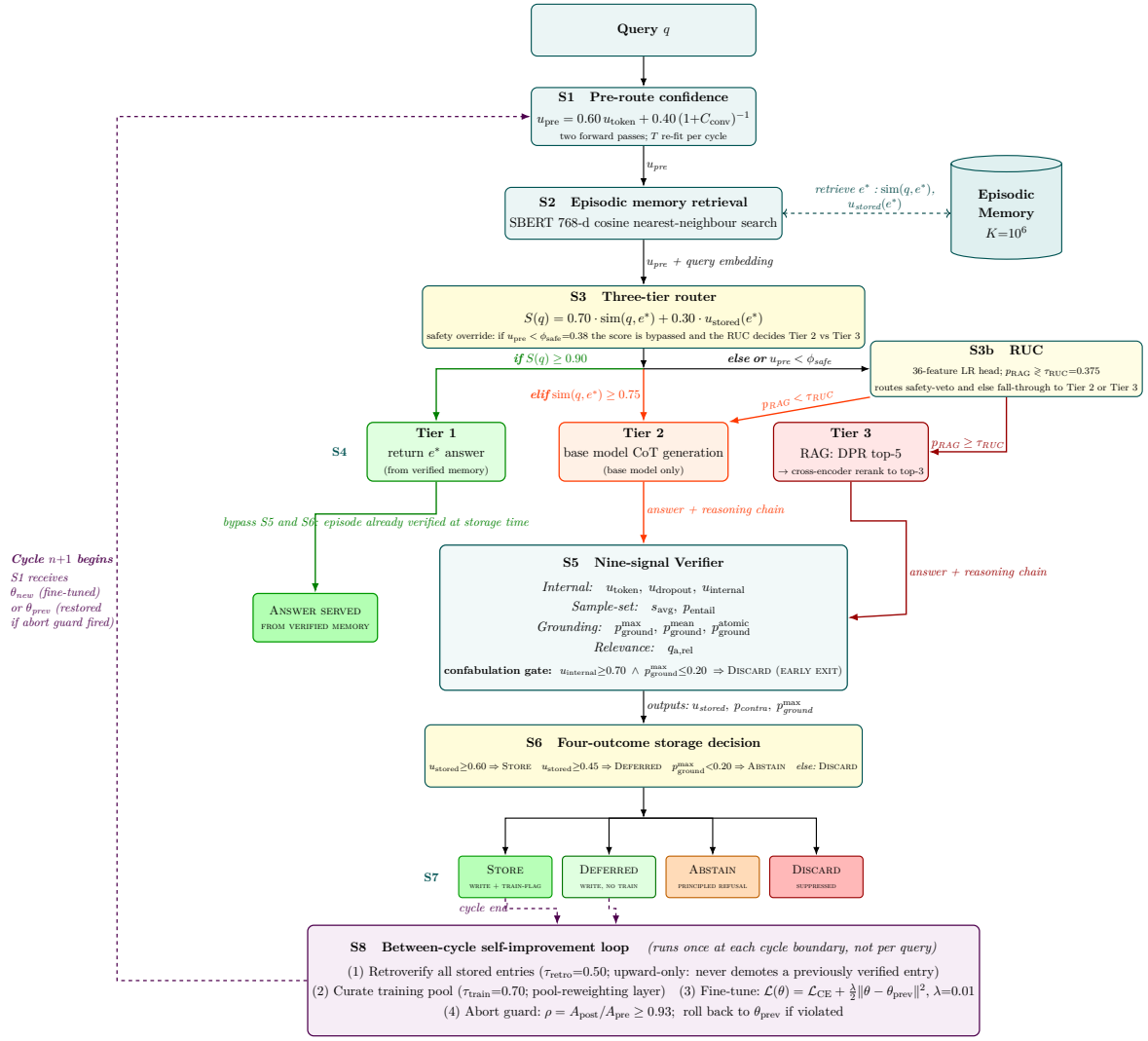


Figure 2.4: The complete eight-stage CAEM pipeline on a single page, reproduced from the main report. Stages S1 to S8 are each opened in their own chapter across Parts II and III; the retrieval-utility classifier (Stage S3b, Chapter 7) sits between the three-tier router and the retrieval-augmented tier, intercepting the safety-veto and score-formula fall-through dispatches and redirecting retrieval-hurt queries to the zero-shot tier. The nine deployed verifier signals at S5 are the registered set minus the internal-state norm (registered then retired at the cold-start audit) and the contradiction probability (structurally zero under the deployed natural-language inference judge). The outer dashed arrow is the cycle-boundary handoff from the close of one cycle to the start of the next.

CHAPTER 3

Prerequisites: The Concepts We Need First

Chapter 2 gave the map. Before we descend into each box, this chapter assembles the toolbox. Five ideas recur in every later chapter, namely the language model that generates, the embeddings that measure meaning, the similarity search that scales them, the calibration that makes a score trustworthy, the entailment test that grounds an answer, and the low-rank adaptation that lets the model learn without forgetting. None of these is unique to CAEM, but each is used in a specific way, with specific numbers, that the rest of the book assumes. We treat each as a self-contained briefing: the general idea first, then exactly how CAEM uses it, taken from the live code.

Why a whole chapter on background

Every component of CAEM is a familiar tool put to a precise purpose. The verifier is mostly entailment and calibration; the router is mostly embeddings and similarity search; the self-improvement loop is mostly low-rank adaptation under a guard. If the tools are clear, the architecture reads as a series of natural choices rather than a wall of jargon. So this chapter front-loads the toolbox once, and the later chapters simply reach for it.

3.1 The generator: what a decoder language model does

At the centre of CAEM sits one frozen language model that produces text. It is worth being precise about what "produces text" means, because two of the verifier's nine signals are read directly off the mechanics below.

A modern generator is an *autoregressive decoder*. It reads a sequence of tokens, which are sub-word units rather than whole words, and predicts the next token, one at a time. For each position it produces a vector of *logits*, one real number per token in its vocabulary. A softmax turns those logits into a probability distribution over the vocabulary, and the model either takes the most probable token (*greedy decoding*) or samples from the distribution. The chosen token is appended to the sequence and the process repeats until a stop token appears.

From logits to a per-token probability

For vocabulary position v at step t, the model emits a logit $z_{t,v}$. The probability of token v is the softmax

$$p_t(v) \;=\; \frac{\exp(z_{t,v})}{\sum_w \exp(z_{t,w})}.$$

The probability the model assigned to the token it *actually* produced, $p_t(\hat{v}_t)$, is the raw material of the token-level confidence signal. A sequence the model found easy has high per-token probabilities; a sequence it was unsure about has low ones. Aggregating these across the generated answer, by a geometric mean of the per-token probabilities, gives the u_{token} signal that appears in both the pre-routing confidence of Chapter 4 and the verifier of Chapter 9.

Two further ideas matter. **Temperature** rescales the logits before the softmax: dividing by a temperature above one flattens the distribution and makes sampling more random, while dividing by a value below one sharpens it. We will meet temperature again, in a completely different role, as a *calibration* knob in Section 3.4; the two uses share a formula but not a purpose. **Chain-of-thought** is the practice of having the model generate intermediate reasoning before its final answer. CAEM uses a scaffolded form, prompting the model to emit a *Reasoning* field and then an *Answer* field, so that the reasoning is available for inspection and the short answer is cleanly extractable.

The specific generator

The backbone is `Qwen-2.5-3B-Instruct`, a three-billion-parameter instruction-tuned decoder, used through the chat-markup prompt format. Generation is *greedy* (`do_sample=False`) for reproducibility, capped at 512 new tokens for both the zero-shot and retrieval paths. Crucially, the backbone is **frozen**: across the entire run its original weights never change. The only thing that trains is a small adapter bolted onto it, which we cover in Section 3.6. Freezing the backbone is not an incidental choice; it is the mechanism that bounds the catastrophic forgetting of Chapter 1.

3.2 Embeddings: turning meaning into geometry

The memory in CAEM has to answer one question quickly: have we seen a query like this one before? Comparing raw strings cannot do this, because "Who wrote *Hamlet*?" and "Name the author of *Hamlet*" share almost no characters yet mean the same thing. Embeddings solve this by mapping text to points in a high-dimensional space, arranged so that similar meanings sit close together.

An embedding is a meaning-coordinate

Imagine every possible sentence placed somewhere in a space with hundreds of axes, positioned so that sentences with similar meaning land near one another and unrelated sentences land far apart. An embedding model is the function that computes those coordinates. Once two sentences are points, "are they similar?" becomes "are these two points close?", which is a geometry question a computer answers in microseconds.

The standard closeness measure for embeddings is *cosine similarity*, the cosine of the angle between two vectors. It is 1 when the vectors point the same way, 0 when they are

orthogonal, and -1 when they are opposed. For two vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\cos(\mathbf{a}, \mathbf{b}) \;=\; \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}.$$

There is a useful trick here. If every embedding is *normalised* to unit length in advance, the denominator becomes one, and cosine similarity collapses to a plain dot product. That single optimisation is why CAEM normalises every embedding the moment it is produced: it lets the similarity search use the fastest possible inner-product machinery while still computing true cosine similarity.

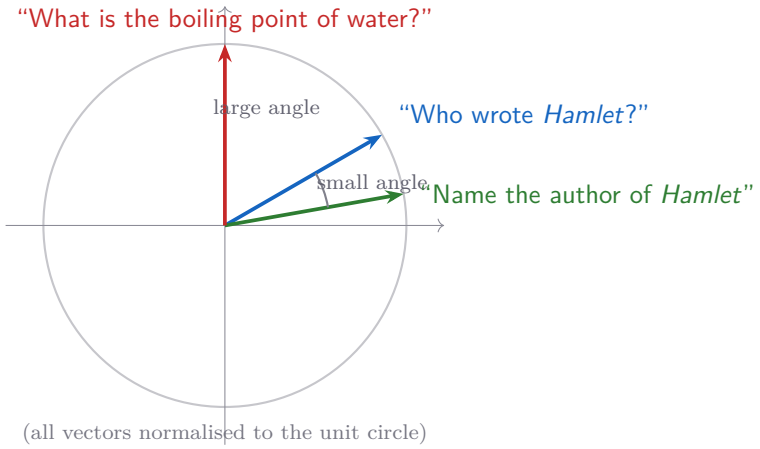


Figure 3.1: Cosine similarity is the cosine of the angle between two normalised embeddings. The two paraphrases of the same question (blue and green) point in nearly the same direction, so their cosine similarity is high. The unrelated question (red) points elsewhere, so its similarity is low. Once vectors are normalised to unit length, this cosine equals their dot product. (Schematic.)

The specific embedding model

The encoder is `sentence-transformers/all-mpnet-base-v2`, a sentence-level embedding model that maps any text to a 768-dimensional vector. CAEM stores the output as 32-bit floats and L2-normalises every vector to unit length, so that an inner product equals cosine similarity exactly. The code guards this invariant aggressively: it validates the dimension on load (refusing a model that does not produce 768 dimensions) and re-normalises defensively if any vector's length drifts from one. This same embedding is reused three times per query, for the memory search, for grounding in the verifier, and for retrieval in the retrieval-augmented tier.

3.3 Similarity search at scale: FAISS

Cosine similarity tells us how to compare *two* vectors. But the memory may hold up to a million episodes, and the retrieval corpus holds tens of millions of passages. Comparing a query against every stored vector one by one would be far too slow. This is the problem that a vector index solves, and CAEM uses the FAISS library for it.

There are two regimes, and CAEM uses both. An **exact** index compares the query against every vector but does so with heavily optimised inner-product arithmetic; it returns the true nearest neighbour every time, and it is fast enough while the collection is small. An **approximate** index trades a sliver of accuracy for a large speed gain, and becomes necessary at scale. The approximate scheme CAEM uses combines two techniques:

Inverted file (IVF).
The vectors are clustered into a fixed number of cells, each with a representative centroid. At search time the query is compared only against the vectors in the few cells whose centroids are nearest, rather than the whole collection. The number of cells is the *nlist* parameter and the number probed per query is *nprobe*; together they trade recall against speed.

Product quantisation (PQ).
Each vector is split into sub-vectors, and each sub-vector is replaced by the index of the nearest entry in a small learned codebook. This compresses each vector to a handful of bytes, so that millions of vectors fit in memory and the distance arithmetic runs on the compact codes.

The two indexes inside CAEM

The episodic memory of Chapter 5 bootstraps as an *exact* inner-product index wrapped so that entries can be deleted by identifier, and then **auto-promotes** to an approximate inverted-file product-quantised index once it holds enough vectors to train the clustering (a threshold of twenty thousand points; cells = 4096, probes = 32, sixty-four sub-quantisers at eight bits each). Early cycles therefore enjoy exact search, and only large stores pay the approximate-search tax. The retrieval corpus of Chapter 8 is far larger and is approximate from the start, with a finer clustering (cells = 32768, probes = 64). The memory's novelty rule also rides on this search: a candidate episode whose nearest stored neighbour exceeds a cosine similarity of 0.95 is judged a near-duplicate and is not stored again.

The vector machinery at a glance

- Embedding model: `all-mpnet-base-v2`, 768 dimensions, unit-normalised.
- Episodic memory: exact index up to 20,000 vectors, then inverted-file product-quantised; capacity 1,000,000; novelty threshold 0.95.
- Retrieval corpus: tens of millions of passages, approximate from the start.
- Similarity: cosine, computed as an inner product on normalised vectors.

3.4 Calibration: making a score mean something

This is the most important idea in the chapter, because it is the idea Chapter 1 identified as the missing piece. A model can be accurate and yet *miscalibrated*: its confidence numbers do not match its actual hit rate. We say a score is *calibrated* when it can be read as a probability. If we collect every answer a calibrated system rated 0.7, then close to seventy percent of

them should be correct. Calibration is what lets a single fixed threshold mean the same thing across very different questions.

Calibrated versus merely high

A confidence score is useful only if it is honest. A weather forecaster who says "seventy percent chance of rain" is well calibrated if it rains on about seventy percent of such days, regardless of whether the forecaster is usually optimistic or pessimistic. A language model, left alone, is a badly calibrated forecaster: it says ninety percent and is right sixty percent of the time. Calibration is the post-processing step that bends those raw scores back onto the line where the number equals the frequency.

The standard way to measure the gap is the *reliability diagram*: bucket predictions by their stated confidence, and plot the average confidence in each bucket against the actual accuracy in that bucket. A perfectly calibrated system lies on the diagonal. The average vertical gap, weighted by bucket size, is the *expected calibration error*, and a curve sitting below the diagonal is the signature of overconfidence.

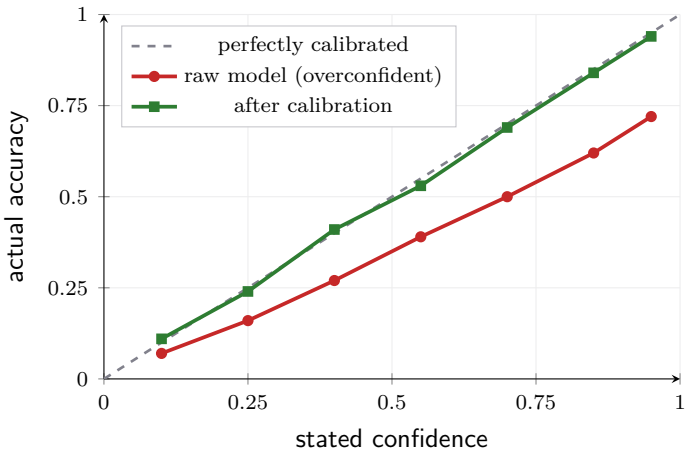


Figure 3.2: A reliability diagram (schematic, drawn to illustrate the concept). The raw model (red) sits well below the diagonal: when it says 0.85 it is right only about 0.62 of the time, the overconfidence of Chapter 1. Calibration (green) bends the scores back toward the diagonal, so a stated confidence can be read as an actual probability. The vertical gap to the diagonal, averaged over buckets, is the expected calibration error.

CAEM uses four calibration tools, each for a different signal, and it is worth naming them once here so the later chapters can refer back.

Temperature scaling.
The simplest fix: divide the score's logit by a single learned temperature before the sigmoid. One number, fit to minimise the calibration error, slides the whole confidence curve toward the diagonal. CAEM uses this for the pre-routing confidence of Chapter 4, with a *separate temperature per benchmark*.

Isotonic regression.
A more flexible, non-parametric fit that learns any monotone mapping from raw score to

calibrated probability. It can bend the curve into any increasing shape, which matters when a signal is informative but not linearly so. CAEM fits one isotonic curve *per verifier signal* and combines them, as the next paragraph describes.

Platt scaling.
A logistic fit, two parameters, that puts the output of one model onto the probability scale of another. CAEM uses it to align the long-context entailment judge with the short-context one (Section 3.5), so that both judges report comparable numbers.

Shrinkage toward a pool.
When a per-benchmark fit is estimated from only a few hundred labelled examples, it overfits. The fix is to pull each per-benchmark curve partway toward the pooled curve estimated from all benchmarks at once. CAEM sets this shrinkage weight to 0.6, keeping moderate per-benchmark adaptation while regularising the weakest benchmarks.

How the nine signals become one calibrated number

The verifier's composite is built so that the final score *is* a calibrated probability rather than an arbitrary weighted sum. Each of the underlying signals is passed through its own isotonic curve, which converts the raw signal into a calibrated per-signal probability and hence a log-odds contribution. The contributions are summed, and a sigmoid turns the total back into a probability. A signal that carries no information fits a nearly flat curve and contributes close to zero log-odds automatically, with no manual zeroing. A signal whose useful direction flips between benchmarks, for instance a relevance score that means opposite things on a fact-checking task and an open-ended question, is handled because its curve is fit per benchmark and can decrease on one and increase on another. This is why a single pair of storage thresholds can govern five different benchmarks: the comparison happens after every signal has been mapped onto the same probability scale.

3.5 Natural-language inference: entailment as a grounding test

The verifier's strongest evidence that an answer is grounded is not whether the model felt confident, but whether the retrieved evidence actually *supports* the answer. The tool for that is natural-language inference.

Natural-language inference asks, given a *premise* and a *hypothesis*, whether the premise entails the hypothesis, contradicts it, or is neutral. CAEM puts the retrieved passage in the premise slot and the model's answer in the hypothesis slot, and reads the entailment probability as a grounding score: a high value means the evidence backs the answer, a low value means the answer floats free of its evidence. This is the heart of the difference between a grounded answer and a confabulation that merely sounds right.

Entailment is directional, and absence of support is not contradiction

Two traps lurk here. First, entailment is *directional*: "the passage entails the answer" is the claim we want, not the reverse. Second, the two judges CAEM uses report only an entailment probability and treat *not supported* the same as *neutral*; they do not raise a separate contradiction alarm. CAEM therefore relies on *low grounding* as its low-support signal rather than a hard contradiction veto. A reader who expects a three-way support, refute, neutral verdict from these judges will misread the verifier; the operative signal is the continuous grounding probability, not a discrete label.

Two judges, dispatched by length

CAEM runs entailment through an adaptive dispatcher with two judges behind it. A specialised, fast checker handles short hypotheses, and a long-context model handles long ones. The split point is 408 tokens, and it is not arbitrary: the fast checker's encoder caps at 512 tokens, of which 4 are reserved for prompt overhead and 100 for a minimum slice of premise, leaving $512 - 4 - 100 = 408$ tokens of hypothesis that fit without truncation. Anything longer routes to the long-context judge, whose output is Platt-scaled (Section 3.4) against the fast checker on a shared five-hundred-sample fold, so the two report numbers on the same scale. The dispatcher is deliberately invisible to the rest of the verifier, which sees one entailment probability and never needs to know which judge produced it. One honest caveat applies to the deployed run: the long-context judge was set aside because its post-hoc probability fit did not clear the required correlation floor, so the shipped verifier ran the fast checker for every prediction, with longer hypotheses truncated at the checker's window (a limitation recorded among the threats).

3.6 Low-rank adaptation: learning without forgetting

The last tool is the one that powers the self-improvement loop. The challenge from Chapter 1 was that retraining a model on new material tends to erase its old capabilities. Low-rank adaptation, or LoRA, is the technique that lets CAEM retrain on its own verified answers each cycle while bounding that risk.

Freeze the library, add a thin notebook

Picture the model's knowledge as a vast reference library that took enormous effort to assemble. Naive fine-tuning rewrites pages of that library to absorb new facts, and in doing so smudges unrelated pages, which is catastrophic forgetting. LoRA instead leaves the library completely untouched and attaches a thin notebook beside it. The notebook is small, it is the only thing that gets written, and at any time it can be torn out and the library is exactly as it was. CAEM writes a fresh page into that notebook each cycle, and the retention guard of Chapter 12 is the rule that tears the page out if it made things worse.

The mechanism is a low-rank matrix. A weight matrix W inside the frozen model maps

an input $\mathbf{x}$ to an output $W\mathbf{x}$. LoRA freezes W and adds a trainable correction built from two small matrices, A and B, whose product has a deliberately low rank r:

$$\mathbf{h} \;=\; W\mathbf{x} \;+\; \frac{\alpha}{r}\, BA\,\mathbf{x}, \qquad A \in \mathbb{R}^{r\times k},\; B \in \mathbb{R}^{d\times r},\; r \ll \min(d,k).$$

Only A and B are learned; W never moves. The scalar α/r controls how strongly the correction is applied. Because r is tiny, the number of trainable parameters is a small fraction of the matrix it corrects.

Why "low rank" is such a large saving

Take a single square projection of size $d \times d$. Training it fully means learning d^2 numbers. LoRA instead learns A and B with $r\,d$ numbers each, for $2rd$ in total. With a rank of $r = 32$ and a typical projection a couple of thousand wide, $2rd$ is on the order of a few percent of d^2, roughly a thirty-fold reduction *on that one matrix*. Aggregated over the handful of projections CAEM adapts, the trainable footprint comes to roughly two percent of the full model. The other ninety-eight percent stays frozen, which is exactly the property that keeps a cycle's update from rewriting unrelated knowledge.

The specific adapter

CAEM adapts with rank $r = 32$ and scaling $\alpha = 64$ (the standard "alpha equals twice the rank" rule, giving a correction scale of two), a dropout of 0.05, and a learning rate of 2×10^{-4}. The adapter is attached to all seven linear projections of each transformer block: the four attention projections (query, key, value, output) and the three feed-forward projections (gate, up, down). This is the "adapt every linear layer" recommendation from the LoRA-scaling literature. Because the base is frozen, the explicit anchor term that a full fine-tune would need to stay near its starting point is dropped on the LoRA path; the freezing does that job for free. This adapter is the only thing the self-improvement loop of Chapter 12 ever changes, and rolling a cycle back means nothing more than discarding the latest version of these two small matrices per projection.

3.7 How the toolbox maps onto the architecture

Each tool reappears at a definite place, and it helps to fix the correspondence before moving on. Table 3.1 is that correspondence.

If an examiner asks: "which of these tools are novel?"

None of the six tools is novel, and the thesis does not claim them. Embeddings, similarity search, calibration, entailment, and low-rank adaptation are all standard machinery. The contribution of CAEM is not a new tool but the *architecture* that composes them: a calibrated confidence deciding how hard to work, an entailment-grounded composite gating a four-outcome admission rule, and a low-rank update under a retention guard closing the loop. The novelty is in the wiring, and the wiring

Table 3.1: Where each prerequisite concept does its work in the chapters ahead.

Concept	**Where it does its work in CAEM**	**Chapter**
Per-token probability	The u_{token} signal in pre-routing and verification	Chapters 4 and 9
Embeddings + cosine	Encoding queries and matching them to stored episodes	Chapters 5 and 6
FAISS search	Memory lookup and retrieval-corpus search at scale	Chapters 5 and 8
Calibration	Honest pre-routing confidence and the calibrated composite	Chapters 4 and 10
Entailment (NLI)	Grounding the answer against retrieved evidence	Chapter 9
Low-rank adaptation	Learning from verified answers without forgetting	Chapter 12

is what the rest of the book is about.

With the toolbox assembled, we are ready to open the first box. Chapter 4 begins the per-query pipeline with the pre-routing confidence, the single number that decides, before any answer is generated, how much work a query deserves.

PART II

The Per-Query Pipeline

CHAPTER 4

Stage 1: Pre-Routing Confidence (u_{pre})

We now open the first box of the per-query pipeline. Before CAEM retrieves anything, generates anything, or verifies anything, it asks a single cheap question about the query in front of it: how ready is the model to answer this on its own? The answer is one number, u_{pre}, the pre-routing confidence. It is the earliest signal in the pipeline and the only one computed before the system commits to a tier, which is exactly why it carries a special responsibility, namely the safety override that refuses to let an unready model answer without grounded retrieval.

A glance before the work

Imagine a triage nurse who, in a few seconds and without running any tests, has to decide whether a patient can wait or needs immediate care. The nurse is not diagnosing; the nurse is deciding *how much machinery to spend*. Stage 1 is that glance. From one short look at the query, before any real answer exists, it produces a confidence that decides whether the model has earned the right to take a cheap path or must be sent down the expensive, grounded one. It is deliberately fast and deliberately humble: when in any doubt, it escalates.

4.1 What Stage 1 is for

It helps to be exact about the one job u_{pre} does, because it is narrower than its prominence suggests. The pre-routing confidence has a single decisive consumer in the router of Chapter 6: the **safety override**. The router checks, before anything else, whether u_{pre} has cleared a fixed floor. If it has not, the query is forced off the cheap paths and onto the retrieval-augmented tier, no matter how good a memory match might look.

u_{pre} does not enter the dispatch score

It is natural to assume that a confidence computed this early feeds into the tier-selection score. In the live code it does not. The router's combined score is built only from the memory similarity and the neighbour's stored quality, the subject of Chapter 6. The pre-routing confidence enters the router at exactly one point, the safety override that is evaluated *before* the score, and nowhere else. Keeping these two mechanisms separate is deliberate: the score answers "does memory cover this query?", while u_{pre} answers a different question, "is the model ready to generate at all?" Conflating them would

over-retrieve when memory is strong but the model is shaky, and under-retrieve in the opposite case.

So u_{pre} is the model's pre-generation readiness, distilled into one calibrated scalar, and its power is the veto. The rest of this chapter builds that scalar from two cheap signals, fuses them, calibrates the result, and then shows the veto in action.

4.2 Signal one: token confidence (u_{token})

The first ingredient is the model's own commitment to its words. We met the raw material in Section 3.1: at each step the model assigns a probability to the token it actually emits. If those probabilities are high, the model is moving through the answer decisively; if they are low, it is hedging across many alternatives at every step, which is the textual signature of guessing.

To turn this into a signal, Stage 1 runs a *short* greedy decode, just thirty-two tokens, under the chat template, and records the probability the model assigned to each chosen token. It then takes the geometric mean of those probabilities, which is the same as exponentiating the average log-probability. The geometric mean is the right average here because probabilities multiply along a sequence; averaging in log-space keeps one very low token from being masked by many confident ones.

The u_{token} definition

Let the short decode produce tokens $\hat{v}_1, \ldots, \hat{v}_n$ (stopping at the end-of-sequence token), each with model probability $p_t(\hat{v}_t)$. Then

$$u_{\text{token}} \;=\; \exp\!\left(\frac{1}{n}\sum_{t=1}^{n}\log p_t(\hat{v}_t)\right) \;=\; \left(\prod_{t=1}^{n} p_t(\hat{v}_t)\right)^{1/n}.$$

This is the per-token average probability under the model: a value near one means the model was confident at every step, a value near zero means it was unsure somewhere along the way. The decode is capped at thirty-two tokens because Stage 1 needs a confidence proxy, not a full answer, and the short cap is what keeps the whole stage inside its latency budget.

The actual computation is short enough to read in full, and it is worth doing so, because two details in it matter.

Listing 4.1: The u_{token} computation, condensed from `caem/confidence/pre_routing.py`.

```
out = model.generate(**inputs, max_new_tokens=32, do_sample=False,
                     output_scores=True, return_dict_in_generate=True)

log_probs = []
for t, step_logits in enumerate(out.scores):          # one entry per
    generated token
    log_soft  = log_softmax(step_logits[0], dim=-1)
```

```
    token_id  = out.sequences[0, gen_start + t].item()
    if token_id == tokenizer.eos_token_id:
        break                                          # stop at
            end-of-sequence
    log_probs.append(log_soft[token_id].item())        # log P(chosen
        token)

u_token = exp(sum(log_probs) / len(log_probs))         # geometric mean
```

The first detail is the greedy, deterministic decode (`do_sample=False`): the signal must be reproducible, so the same query always yields the same u_{token}. The second is the fail-safe. If generation throws, or returns no scorable tokens, the function returns 0.0 rather than raising. A zero pre-routing confidence is the most cautious value possible, and as we will see it forces the query onto the grounded tier. Failure in Stage 1 therefore degrades toward safety, never toward a confident wrong answer.

Raw text collapses the signal; the chat template rescues it

One non-obvious requirement: the query must be wrapped in the chat-markup envelope before the decode. An instruction-tuned model fed a bare question scatters its probability mass across the vocabulary, and u_{token} collapses to roughly 10^{-9} for every query, informative about nothing. Wrapping the query as a user turn with a generation prompt puts the model into "assistant answering a user" mode, where its per-token probabilities become meaningful. The envelope adds about twenty fixed tokens, identical for every query, which is why it does not distort the second signal we turn to next.

4.3 Signal two: internal convergence (c_{conv})

Token confidence reads the model's output. The second signal reads its *internals*, and asks a different question: has the model settled on a single, stable interpretation of the query, or is it still of several minds about what is being asked? This is the internal-convergence ratio, adapted from [15].

The idea rests on how a decoder processes a query. The same query produces a stack of hidden-state representations, one per layer. When the model has a clear, unambiguous reading of the question, the representation stabilises: the later layers vary little, having converged on an interpretation. When the question is ambiguous or unfamiliar, the representation keeps shifting across layers, and the early-layer variance stays high relative to the late layers. The ratio of those two variances is a cheap proxy for "did the model converge?"

The c_{conv} definition

A single forward pass over the query, with no generation, exposes the hidden-state stack $\mathbf{H}^{(1)}, \ldots, \mathbf{H}^{(L)}$ for the L decoder layers (the backbone has $L = 36$). Splitting the stack into an early half and a late half, the convergence ratio is the variance of the

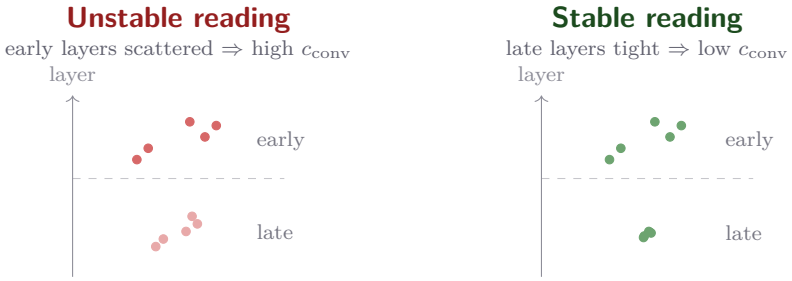


Figure 4.1: The convergence ratio compares the spread of the early-layer representations with that of the late layers. On the left the late layers are still scattered: the model has not converged on one reading, the ratio c_{conv} is high, and the confidence $1/(1 + c_{\text{conv}})$ is low. On the right the late layers are tight: the model has settled, c_{conv} is low, and the confidence is high. (Schematic.)

early half divided by that of the late half,

$$c_{\text{conv}} \;=\; \frac{\operatorname{Var}\big(\mathbf{H}^{(1:L/2)}\big)}{\operatorname{Var}\big(\mathbf{H}^{(L/2+1:L)}\big) + \varepsilon},$$

and it is turned into a confidence by the decreasing map

$$\text{confidence from } c_{\text{conv}} \;=\; \frac{1}{1 + c_{\text{conv}}}.$$

A high ratio (early layers far more variable than late) reads as instability and maps toward zero; a low ratio reads as a settled representation and maps toward one. As with u_{token}, any failure in this forward pass returns a ratio of zero, the most confident value, but here the fail-safe is overridden in fusion because the token signal will usually have failed too, and the combined floor still escalates. The embedding layer is excluded from both halves so that only the transformer computation, not the raw lookup, is measured.

4.4 Fusing and calibrating the two signals

Two signals become one through a fixed weighted blend, followed by a calibration step. The blend places more weight on the token signal, which is the more direct reliability cue, and uses the convergence signal as a secondary prior on query stability.

From two signals to a calibrated u_{pre}

The raw pre-routing confidence is the registered blend

$$u_{\text{pre}}^{\text{raw}} \;=\; 0.60 \cdot u_{\text{token}} \;+\; 0.40 \cdot \frac{1}{1 + c_{\text{conv}}},$$

clipped to $[0, 1]$. The mixing weights 0.60 and 0.40 are *registered before* the calibration phase rather than learned, so the meaning of the fused scalar does not drift as the generator is fine-tuned across cycles. The raw value is then passed through temperature

scaling (Section 3.4) to make it a calibrated probability,

$$u_{\text{pre}} = \sigma\left(\frac{\text{logit}\,(u_{\text{pre}}^{\text{raw}})}{T_b}\right),$$

where the temperature T_b is fit *per benchmark*. A temperature of one leaves the value untouched; a larger temperature softens an overconfident raw distribution toward the diagonal of the reliability diagram.

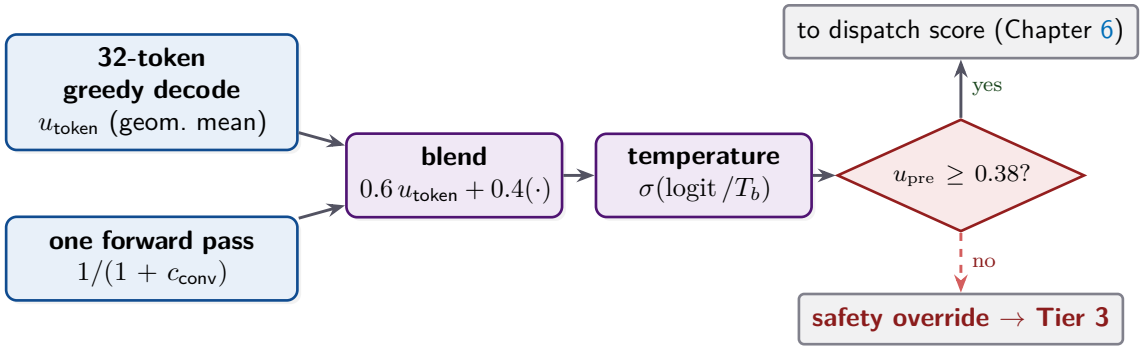


Figure 4.2: The pre-routing confidence is built from two cheap signals, a thirty-two-token greedy decode for u_{token} and a single forward pass for c_{conv}, blended with fixed weights, calibrated by a per-benchmark temperature, and then tested against the safety floor. Clearing the floor hands control to the dispatch score; failing it triggers the safety override straight to the grounded tier. (Schematic.)

4.5 The safety override

Now the payoff. The router evaluates one rule before any other: if u_{pre} is below the registered floor $\phi_{\text{safe}} = 0.38$, the query is dispatched to the retrieval-augmented tier and the override flag is recorded, regardless of how strong the memory match is. This is the architectural expression of a single principle: **the system must not generate without grounded retrieval whenever its pre-generation readiness is low**. The cost of grounded retrieval is explicitly preferred over the risk of a confident wrong answer from memory or zero-shot generation.

The floor, and why it is 0.38 rather than 0.60

The floor began life at 0.60 and was lowered to 0.38 during the cycle-one routing audit. The audit found that the higher floor was forcing well-supported queries onto the grounded tier under the post-calibration pre-routing distribution, which depressed the architecture's effective use of the cheaper tiers without a matching gain in safety. Lowering the floor preserved the override's intent, refuse to generate when readiness is genuinely low, while admitting a wider band of capable queries to the cheap paths. The floor is read per benchmark through a configuration helper, with the pooled $\phi_{\text{safe}} = 0.38$ used as the registered default. The per-benchmark floors are re-fit across cycles, and

they are not all equal: the fact-checking and trivia probes hold at the pooled 0.38, while the commonsense probe drifts upward, to about 0.49 by the third cycle and about 0.57 by the fifth, as its calibrated readiness distribution shifts. The pooled 0.38 remains the fallback wherever a benchmark-specific value is unavailable.

The ordering is the subtle part. Because the override is checked first, a query with a near-perfect memory match but a low readiness signal is still sent to retrieval. The override is a veto, not a vote: it cannot be outweighed by a good score, only avoided by clearing the floor. When the floor is cleared, u_{pre} has done its job and steps aside; the tier is then chosen by the similarity-and-quality score of Chapter 6.

On the realised trajectory the override is a worst-case safety contract that, in fact, never activates: the calibrated pre-routing confidence stays above the active floor on every one of the roughly nine thousand evaluation queries, so the override never fires. This is best read not as evidence that the mechanism is unneeded, but as evidence that the panel distribution simply does not exercise it. The contract is cheap to carry and would fire the moment a genuinely unready query appeared; the deployed query mix never produced one that fell through the floor.

4.6 Why Stage 1 is cheap by design

A theme of the whole architecture is paying for confidence only where it is needed, and Stage 1 is the first instance. The pre-routing confidence pointedly does *not* use the expensive multi-chain resampling that the post-generation verifier of Chapter 9 relies on. It costs one forward pass over the query plus a thirty-two-token greedy decode, and that is all.

The right signal at the right price

There are two confidence questions in the pipeline, asked at different prices. Stage 1 asks the cheap one, "should the model even try?", and pays a single short decode for it, because every query must be triaged. The expensive question, "is this generated answer trustworthy enough to keep?", is asked only of queries that actually reach the generating tiers, and is paid for with the resampling and entailment machinery of Chapter 9. A memory-direct query never pays the expensive cost at all. Spending a cheap signal on everyone and an expensive signal on the few who need it is the economic logic of the entire compute ladder.

4.7 The two readings, at the u_{pre} level

The two queries we traced in Chapter 2 diverge for the first time exactly here, at Stage 1, and it is instructive to watch only this stage.

The claim that clears the floor (FEVER)

For the claim "The French Revolution began before 1792", the short decode commits to its tokens with reasonable probability and the forward pass shows a settled late-layer representation. The blended raw confidence, after the per-benchmark temperature, lands comfortably above 0.38. The safety override does *not* fire. Control passes to the dispatch score, and because the cycle-zero memory is empty, the query continues down the pipeline toward generation rather than being vetoed here. Stage 1's verdict: this query is ready enough; let the later stages decide the tier.

The question routed to grounding (trivia)

For " 'If I Ruled The World' comes from which musical?", the short decode commits only weakly to its tokens (the token-probability aggregate sits near 0.02), yet the calibrated pre-routing confidence still lands around $u_{\text{pre}} = 0.66$, comfortably above the 0.38 safety floor. The override therefore does *not* fire here. The query nonetheless goes to the retrieval-augmented tier, because at the cold start the episodic memory is empty and no cheap path can be earned: the dispatch score finds no neighbour to draw on. Grounded generation is what later gives the verifier something real to judge, and ultimately what lets the system refuse rather than fabricate. The honest refusal at the end of the pipeline begins with this early decision to ground.

4.8 An honest limitation

One property of u_{pre} deserves a frank note, because it surfaces in the trajectory of Chapter 15. The single-parameter temperature that calibrates the pre-routing confidence (distinct from the composite's own temperature in Chapter 10) behaves oddly across the run. At the cold start the fit saturates its optimisation bound, returning a temperature near 20 that is best flagged as pathological calibration data; even there the calibration error does fall, from about 0.28 to about 0.07. The per-cycle re-fits that follow then settle at low interior values, roughly 1.09, 0.84, and 0.74 over the next cycles, while the calibration error barely moves, staying around 0.04 to 0.05. The honest reading is therefore not that the temperature is pinned at its ceiling every cycle, but that single-parameter temperature scaling on this particular surface is largely inert across the trajectory: it neither helps nor hurts u_{pre} much once the cold-start boundary is past.

Why the drift is tolerable

A miscalibrated u_{pre} would be a serious problem if it gated what CAEM stores. It does not. The pre-routing confidence governs only *routing*: the safety override and, indirectly, which tier pays the verification cost. The decision of whether an answer is trustworthy enough to keep is made downstream, by the calibrated composite of Chapter 10, on signals that carry their own per-cycle calibration. So a drift in u_{pre} can send a few queries down a more expensive tier than strictly necessary, which costs compute, but it cannot admit a wrong answer into memory. The architecture localises

the consequences of this imperfection to cost, not correctness, which is precisely why the cheap signal is allowed to be imperfect.

Stage 1 constants

- u_{token} decode length: 32 tokens, greedy, deterministic.
- Fusion weights: 0.60 on u_{token}, 0.40 on $1/(1 + c_{\text{conv}})$, registered (not learned).
- Convergence split: early half versus late half of $L = 36$ decoder layers, embedding layer excluded.
- Calibration: per-benchmark temperature T_b via $\sigma(\text{logit}(\cdot)/T_b)$.
- Safety floor: $\phi_{\text{safe}} = 0.38$ (lowered from 0.60 at the cycle-one audit).
- Fail-safe: any error returns 0.0, the most cautious value, forcing the grounded tier.

If an examiner asks: "is two signals not too few for a confidence estimate?"

The pre-routing confidence is deliberately thin because of what it is for. It is a triage signal whose only hard consequence is a conservative veto, backed by a fail-safe that escalates on any doubt, so the cost of an occasional false low-confidence reading is bounded compute rather than a wrong answer. The literature is in any case discouraging about single-signal confidence, which reaches only chance-to-modest discrimination on its own; CAEM answers that not by piling more signals into the cheap pre-generation estimate, but by deferring the heavy, many-signal judgement to the post-generation verifier, where it can be afforded. Two cheap signals to triage, nine calibrated signals to decide: the split is the point, not an oversight.

With the query triaged, the pipeline turns to memory. Chapter 5 takes up Stage 2, the episodic store: how a query is encoded, how the nearest past episode is found, and what an episode actually carries.

CHAPTER 5

Stage 2: The Episodic Memory Store

Stage 1 decided how ready the model is. Stage 2 asks a different question: have we answered something like this well before? The episodic memory store is CAEM's answer to the first structural flaw of Chapter 1, statelessness. It is the component that lets the system say "I have seen this, and last time I got it right," and it is the substrate that every later stage either reads from or writes to. This chapter covers the store as a whole: what a single memory holds, how a query finds its nearest match, how a new memory is admitted, and how the store keeps itself small and trustworthy over time.

A notebook of verified answers

Picture a researcher who keeps a notebook of every question they have settled, with the reasoning that settled it and a note of how sure they were. When a new question arrives, they first flip to the most similar past entry. If it is close enough and was confidently resolved, they reuse it instead of redoing the work. The notebook is curated: near-duplicate entries are merged, shaky entries are re-checked when the researcher learns more, and if it ever overflows, the least useful pages are torn out first. The episodic memory store is that notebook, made precise enough for a machine to keep.

5.1 What a single memory holds

The unit of memory is an *episode*: a verified triple of question, reasoning, and answer, wrapped in the metadata that lets the rest of the system reason about it. The fields fall into three groups, and the grouping is not cosmetic, because the groups have different mutability rules.

Why store the reasoning, not just the answer

The most valuable field is often the one a cache would omit: the reasoning chain. A plain lookup table would store only the question and the answer. CAEM stores the chain of reasoning too, because that is the transferable knowledge the self-improvement loop of Chapter 12 learns from. An answer tells the model *what*; the chain tells it *how*, and the "how" is what generalises to the next, slightly different question. The memory is therefore not just a cache of answers but a corpus of worked solutions.

The trust score u_{stored} deserves a forward pointer rather than a definition here. It is the calibrated composite produced by the verifier and gate of Chapters 9 and 10; for this chapter it is enough to read it as a number in $[0, 1]$ that says how much the system trusts this episode's answer. Stage 2 only *consumes* it, during routing; it is *produced* downstream.

Table 5.1: The fields of a stored episode, by group. Content is frozen at storage time because the fine-tune of Chapter 12 trains directly on it; quality and usage evolve across cycles.

Group	Fields and meaning
Content (immutable)	The *question*, the *reasoning chain* that produced the answer, the short *answer* string, and the 768-dimensional unit-normalised *embedding*. Plus provenance: the *storage cycle*, a timestamp, the *source benchmark*, and any benchmark tags merged in during consolidation. These never change after storage, because the self-improvement loop trains on them and a moving target would corrupt the dataset.
Quality (mutable)	The stored trust score u_{stored}, the verifier signals recorded on the episode (u_{token}, u_{dropout}, u_{internal}, s_{avg}, p_{entail}, $p_{\text{ground,max}}$, $p_{\text{ground,mean}}$, $p_{\text{ground,atomic}}$, $q_{\text{a,rel}}$) together with the retired-but-recorded h_{norm} diagnostic, the gate *decision* (one of store, defer, abstain, discard), and a confabulation flag. These are overwritten when the episode is re-scored at a cycle boundary.
Usage (mutable)	The *retrieval count*, the running *success rate* of times the episode was served and accepted, a *hit counter* since its last re-check, and a *retroverified* flag. These track how the episode performs in service and drive both the trust-feedback update and the pruning score.

5.2 The per-query operation: encode and probe

In the per-query pipeline, Stage 2's active job is small and fast. The query is encoded into its 768-dimensional embedding by the sentence encoder of Section 3.2, and that vector is used to probe the store for its single nearest neighbour. Because every vector is unit-normalised, the nearest-neighbour search by inner product returns exactly the highest cosine similarity, and the search returns both the matched episode and that similarity score.

Listing 5.1: The nearest-neighbour probe, condensed from `caem/memory/store.py`.

```
def search(self, embedding, k=1):
    if self.is_empty:
        return []                                   # cold start: nothing to
            match
    vec = self._to_faiss_matrix(embedding)          # (1, 768) float32,
        unit-norm
    sims, ids = self._index.search(vec, k)          # inner product == cosine
    return [(self._metadata[int(i)], float(s))      # (episode, similarity)
            for s, i in zip(sims[0], ids[0]) if i != -1]
```

Two things are worth noticing. First, an empty store returns nothing, which is the cold-start case: on the very first cycle the memory is empty, so every query falls through to generation, exactly as we saw for the FEVER claim in Chapter 2. Second, the search returns the episode object itself, not just an identifier, so the router can immediately read the neighbour's stored trust score without a second lookup. A companion form of the call also returns the integer identifier, which the pipeline uses when it later needs to update that episode's usage statistics after a Tier 1 hit.

The substrate underneath

The search rides on the FAISS index described in Section 3.3. The store bootstraps as an *exact* inner-product index so that early cycles, when the store is small, get exact nearest neighbours, and it *auto-promotes* to the approximate inverted-file product-quantised index once it holds enough vectors to train the clustering. The index is wrapped so that episodes can be deleted by identifier, which matters for the pruning and consolidation operations later in this chapter. None of this is visible to the caller: the probe returns the nearest episode and a cosine score regardless of which index is active underneath.

5.3 Admission: the novelty gate

When the gate of Chapter 10 decides an answer is worth storing, the store does not admit it blindly. It first checks *novelty*: if the candidate's nearest existing neighbour is too similar, the candidate is judged a near-duplicate and is not stored. "Too similar" is a cosine similarity above 0.95.

Why duplicates are actively harmful

A cache would happily store the same answer twice; it costs only space. For CAEM a duplicate costs more than space, because the memory is also the training set. Storing the same verified answer many times would let a handful of popular questions dominate the fine-tune, teaching the model their phrasing at the expense of breadth. The novelty gate keeps the memory a set of *distinct* solved problems rather than a frequency table of the common ones, which is what keeps the downstream training signal balanced.

The novelty check reuses the very same nearest-neighbour search as the per-query probe: it asks for the top neighbour and rejects the candidate if that neighbour's similarity exceeds the threshold. The threshold sits high, at 0.95, deliberately. Paraphrases that are merely related should still be stored as separate episodes, because they may carry different reasoning; only near-identical restatements are screened out at admission. Genuine paraphrase clusters are handled later and more carefully by consolidation (Section 5.6), which can inspect the answers before merging rather than rejecting at the door.

5.4 Capacity and pruning

The store has a hard capacity of one million episodes. Long before that, when it reaches ninety-five percent full, it triggers a prune that removes the least valuable fifth of its episodes. "Least valuable" is not "oldest" or "least confident" alone; it is a composite *Value* score that balances how good an episode is against how useful it has proven and how recent it is.

The Value score

Each episode's Value combines an importance term and a recency term,

$$\text{Value} \;=\; 0.70 \cdot \text{Importance} \;+\; 0.30 \cdot \text{Recency},$$

where importance is a weighted blend of four cues,

$$\text{Importance} \;=\; 0.40\,\varphi \;+\; 0.30\,\frac{r}{r_{\max}} \;+\; 0.20\,\text{success} \;+\; 0.10\,u_{\text{stored}},$$

and recency decays exponentially with the episode's age in seconds,

$$\text{Recency} \;=\; \exp(-0.01 \cdot \text{age}).$$

Here $\varphi = (\text{storage cycle} + 1)/(\text{number of cycles} + 1)$ is a proxy for how recently the episode was stored, $r/r_{\max}$ is the episode's retrieval count normalised by the busiest episode's, success is its running acceptance rate in service, and u_{stored} is its trust score. When a prune fires, episodes are sorted by Value and the bottom twenty percent are removed.

Which episode survives a prune

As an illustration (the capacity prune was never reached in the realised run, whose store peaked near ten thousand episodes against a one-million capacity), consider two episodes competing to stay. Episode A was stored in an early cycle, has never been retrieved, and carries a middling trust score. Its φ is small, its retrieval and success terms are zero, and its recency has decayed, so its Value is low and it is a prime candidate for removal. Episode B was stored recently, has been retrieved many times and accepted almost every time, and carries a high trust score. Every term in its importance is large and its recency is near one, so its Value is high and it is retained. The lesson is that the store does not just keep its most confident episodes; it keeps the ones that have *earned their place* by being useful in service, which is a stronger signal of value than confidence at storage time alone.

5.5 Living metadata: trust that adapts to service

An episode's trust score is not frozen the moment it is stored. Every time the episode is served from memory and the answer is accepted or overridden, the store updates the episode's usage statistics, and once the episode has been retrieved enough times, it also nudges the trust score itself.

Listing 5.2: Retrieval feedback, condensed from `caem/memory/store.py`.

```
entry.retrieval_count += 1
entry.success_rate = (old_rate * n + float(was_accepted)) / (n + 1)    # running mean

if entry.retrieval_count >= 5:                    #
```

```
    feedback_loop_min_retrievals
    eta = 0.01                                      # feedback_loop_eta
    if was_accepted:
        entry.u_stored += eta * (1.0 - entry.u_stored)    # nudge toward 1
    else:
        entry.u_stored -= eta * entry.u_stored            # nudge toward 0
```

The success rate is a running mean, so it reflects the episode's whole service history, not just its last use. The trust nudge is deliberately small and deliberately gated: it applies only after at least five retrievals, so that a single acceptance or rejection cannot swing an episode's standing, and even then it moves the score by only a small fraction of the remaining distance to one or zero. The effect accumulates: an episode that is repeatedly served and accepted drifts slowly upward in trust, while one that is repeatedly overridden drifts down, eventually toward the pruning threshold. There is also a separate hit counter that, once an episode has been served ten times since its last re-check, flags it for forced re-verification at the next cycle boundary, on the principle that a popular episode that is quietly wrong does the most damage and is worth re-checking early.

5.6 What the store does at a cycle boundary

Two of the store's most important operations are not driven by the per-query pipeline at all. They are invoked by the self-improvement loop of Chapter 12 once per cycle, and we introduce them here only because they are the store's own methods; their full treatment belongs to that chapter.

Retroactive re-verification.
Every stored episode is re-scored through the recalibrated verifier under the freshly fine-tuned model. An episode whose new trust score falls below a floor is pruned; otherwise its quality fields are overwritten with the new reading, which may raise or lower its trust. This is how the memory benefits from the model getting better: yesterday's borderline episode can be re-judged confidently today, and a stale confident episode can be caught and demoted. In the deployed run this pass pruned a declining fraction at each committed cycle, from about thirteen percent at the first cycle to about four percent by the last (about 13, 9, 7, 4 percent), the signature of a store whose composition shifts toward higher-confidence survivors as the model improves; the prune floor of 0.50 sits below the storage cut-point, so re-verification removes only entries that have genuinely lost confidence.

Consolidation.
Near-paraphrase episodes are clustered, and each cluster is collapsed onto a single representative, the one with the highest trust. The merge is guarded: it proceeds only if the cluster's answers agree after normalisation and their trust scores do not spread too widely, and it never merges across the training-versus-transfer boundary, so that consolidation cannot leak held-out content into the training pool. The representative inherits the cluster's combined retrieval history and benchmark tags. In the deployed run the clustering threshold sat at cosine 0.92 over the nearest twenty neighbours; the answer-equality guard and a trust-spread

guard, which rejected any cluster whose trust range exceeded 0.15, screened most candidates, and the cross-pool guard rejected every candidate straddling the training-versus-transfer boundary. Across the trajectory the pass merged 57 clusters and removed 58 episodes. Note that the consolidation threshold of 0.92 stays distinct from the admission novelty gate of 0.95 described in Section 5.3: consolidation inspects answers and trust before merging, whereas the novelty gate rejects only near-identical restatements at the door.

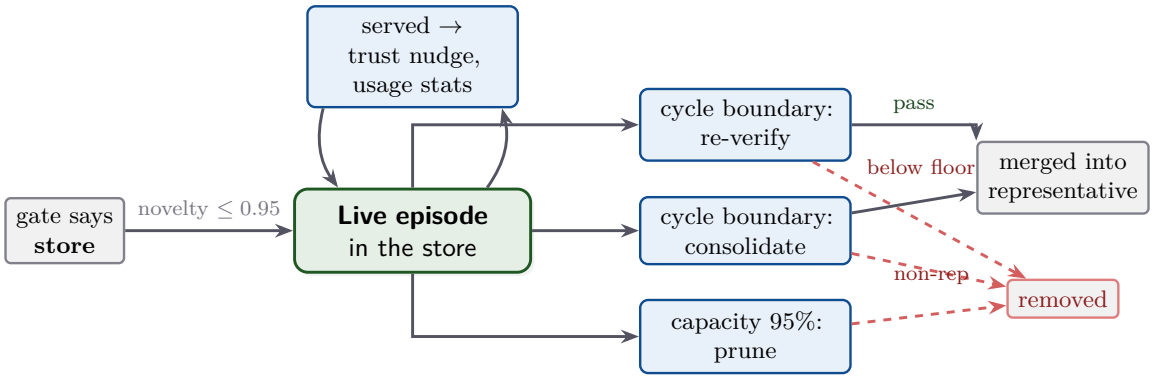


Figure 5.1: The life of an episode. It is admitted only if it clears the novelty gate, then lives in the store being served (each serve nudges its trust and usage stats). At each cycle boundary it is re-verified (kept with an updated trust, or removed if it falls below the floor) and may be consolidated into a representative; under capacity pressure the least valuable episodes are pruned. The dashed paths are the only ways an episode leaves the store. (Schematic.)

The memory is curated, not merely accumulated

It would be a mistake to picture the store as an append-only log that only ever grows. It is continuously curated. New episodes are screened at admission, served episodes have their trust nudged by experience, the whole store is re-judged each cycle as the model improves, paraphrase clusters are merged, and the least valuable episodes are pruned under capacity pressure. The store's quality is maintained by these forces acting together, which is what lets a single fixed trust threshold keep meaning the same thing as the store ages.

5.7 The store survives a rollback

One property from Chapter 2 is worth restating in the store's own terms, because it is the reason the memory is the system's most durable asset. When a cycle's fine-tune fails the retention guard and the model weights are rolled back, the episodic memory is *not* rolled back with them. Every episode admitted during that cycle survives. This fired once in the deployed run; at one cycle the worst of the three retention probes (the held-out open-text factoid probe) fell to a ratio of 0.886 against its pristine baseline, below the fixed floor of 0.93, while the general-knowledge probe held at 1.018 and the commonsense probe at 0.982; the adapter was restored to the previous cycle's weights, yet the store was untouched and kept growing across the boundary, from about five thousand to nearly ten thousand episodes by the final cycle.

Why memory growth is safe to keep even on a failed cycle

The asymmetry is principled. Each episode in the store passed the four-outcome gate at storage time, so memory growth is monotone and high-precision by construction: adding a gated episode can only add a trustworthy fact. A weight update has no such guarantee; it is a global change that can erode unrelated capabilities, which is the catastrophic forgetting the whole loop is built to avoid. So the system protects the verified, accumulated asset (the memory) and is willing to discard the risky, reversible one (the adapter). This is why a rollback costs a cycle's training but never a cycle's remembered knowledge. The store is also checkpointed to disk each cycle, as a paired index and metadata file, so the accumulated memory is recoverable independently of the model.

Episodic memory constants

- Embedding: 768-dimensional, unit-normalised; search by cosine (inner product).
- Capacity: 1,000,000 episodes; prune trigger at 95% full; prune the bottom 20% by Value.
- Novelty: candidates with a nearest neighbour above cosine 0.95 are not stored.
- Value = 0.70·Importance+0.30·Recency; recency decay rate 0.01 per second of age.
- Trust feedback: nudge step 0.01, applied only after ≥ 5 retrievals.
- Forced re-check: flagged after 10 Tier 1 serves since the last re-verification.

If an examiner asks: "how is this different from a retrieval cache?"

A cache stores answers and serves them back; the episodic store does three things a cache does not. It stores the *reasoning*, not just the answer, because the memory doubles as the training corpus for the self-improvement loop. It carries a *calibrated trust score* on every entry, so the router can weigh a match by quality and not only by similarity. And it is *curated against its own model*: each cycle re-judges, merges, and prunes its contents as the model improves, so the store's precision is maintained rather than left to decay. A cache is a passive lookup table; this is an actively maintained, quality-scored knowledge base that happens to also answer lookups.

The store can now tell the router, for any query, both how similar the nearest episode is and how much that episode is trusted. Chapter 6 takes up Stage 3, where those two numbers are fused into a single dispatch score and a tier is finally chosen.

CHAPTER 6

Stage 3: The Three-Tier Router

We now have everything the router needs. Stage 1 produced a readiness signal, u_{pre}; Stage 2 produced, for the nearest stored episode, a similarity score and a stored trust score. The router is the small piece of logic that turns those three numbers into a single decision: which of the three tiers should answer this query. It is the hinge of the whole per-query pipeline, the point where the self-organising compute ladder of Chapter 2 actually does its organising.

The cheapest tier that will do

The router's guiding rule is economy under safety. It wants the *cheapest* tier that can answer well, so it prefers serving a trusted memory directly, then generating zero-shot, and only then paying for retrieval. But it will spend more the instant the cheap options stop being safe. Think of it as a dispatcher with three crews of rising cost: it sends the cheapest crew that is clearly competent for the job, and escalates the moment competence is in doubt. The art is in defining "clearly competent," and that is what the router's thresholds encode.

6.1 What the router sees and what it returns

The router is deliberately starved of raw data. It does not touch the model, the memory index, or the retrieval corpus; it only reads scalars the upstream stages already computed. Its inputs are the pre-routing confidence u_{pre}, the cosine similarity s of the query to its nearest stored episode, and that episode's stored trust score u_{stored}. From these it produces a tier index in $\{1, 2, 3\}$ together with bookkeeping: the routing score it computed, a flag recording whether the safety override fired, the identifier of the retrieved episode, and, when consulted, the output of the retrieval-utility classifier of Chapter 7.

Two numbers about memory, asking different questions

The two memory numbers are not redundant, and the router treats them differently. Similarity asks "how close is the nearest thing we have seen?" Stored trust asks "and was that thing actually any good?" A query can score high on one and low on the other. We have a near-identical match to a past episode whose answer we never trusted; or a loosely related match to an episode we trust completely. Keeping the two questions separate is what lets the router make the right call in each case, as we will see in Section 6.3.

6.2 Three mechanisms, in a fixed order

The router applies three mechanisms, and the order is not negotiable, because each expresses a different constraint that must dominate the ones after it.

Mechanism 1, the safety override (checked first).
If u_{pre} is below the floor $\phi_{\text{safe}} = 0.38$, the query is dispatched off the score path to the retrieval-augmented tier and the override flag is set, regardless of how strong the memory match is. This is the veto from Chapter 4: low pre-generation readiness means the system must not answer without grounded retrieval.

Mechanism 2, the dispatch score.
If the override does not fire, the router fuses the two memory numbers into one score and compares it against a threshold ladder, choosing Tier 1, Tier 2, or a fall-through.

Mechanism 3, the retrieval-utility classifier (Stage 3b).
The two outcomes that would otherwise route unconditionally to retrieval, the safety override and the score fall-through, are intercepted by a learned classifier that may redirect the query to zero-shot generation instead. The classifier is the subject of Chapter 7; here we note only where it plugs in.

The dispatch score itself is a weighted blend of the two memory numbers,

$$\text{routing score} \;=\; \lambda \cdot s \;+\; (1-\lambda)\cdot u_{\text{stored}}, \qquad \lambda = 0.70,$$

which leans on similarity (weight 0.70) more than on stored trust (weight 0.30), because a close match is the stronger evidence that memory covers the query. The score then meets a two-rung ladder: a query whose score reaches the combined threshold $\theta_1 = 0.90$ is served directly from memory (Tier 1); a query that misses that rung but whose *similarity alone* exceeds $\theta_{\text{sim}} = 0.75$ is generated zero-shot (Tier 2); anything else falls through toward retrieval (Tier 3), subject to the classifier interception.

Table 6.1: The router's dispatch rules, evaluated top to bottom; the first matching rule wins. The classifier of Chapter 7 intercepts the two rules that default to retrieval and may redirect them to the zero-shot tier.

Order	Condition	Meaning	Tier
1	$u_{\text{pre}} < 0.38$	safety override: readiness too low to generate ungrounded	3 (or 2 via RUC)
2	$0.70\,s + 0.30\,u_{\text{stored}} \geq 0.90$	strong, trusted memory match: serve it	1
3	$s > 0.75$	similar enough to be worth a fresh zero-shot answer	2
4	otherwise	no cheap path earns the query: retrieve and ground	3 (or 2 via RUC)

Listing 6.1: The dispatch logic, condensed from `caem/routing/router.py`.

```
# Mechanism 1 -- safety override, evaluated FIRST
if u_pre < safety_thr:                                  # safety_thr = 0.38
    return consult_ruc(default_tier=3, entry_point="safety_veto")  # may
        redirect to T2

# Mechanism 2 -- dispatch score
score = 0.70 * similarity + 0.30 * u_stored_retrieved
if score >= 0.90:                                       #
    tier1_combined_threshold
    tier = 1                                            # serve stored answer
        directly
elif similarity > 0.75:                                 #
    tier2_similarity_threshold
    tier = 2                                            # generate zero-shot
else:
    tier = consult_ruc(default_tier=3, entry_point="fall_through")  # may
        redirect to T2
```

6.3 Why the two tests are asymmetric

The single most instructive feature of the ladder is that the two rungs test *different things*. The Tier 1 rung tests the combined score, which needs both similarity and stored trust to be high. The Tier 2 rung tests similarity alone, and ignores stored trust entirely. This asymmetry is not an oversight; it follows directly from what each tier does with the matched episode.

Serve-the-answer versus borrow-the-neighbourhood

Tier 1 *serves the stored answer verbatim*. To do that safely, the match must be both close (so the stored answer is actually relevant) *and* trusted (so the stored answer is actually good). Both conditions are necessary, which is exactly why the rung is a combined score with weight on each. Tier 2 does something different: it *generates a fresh answer*, using the existence of a similar past episode only as evidence that the query lies in a region the model has handled before. The stored answer is never served, so its quality is irrelevant to the Tier 2 decision; all that matters is that a similar question exists, which is why the Tier 2 rung looks at similarity alone. The asymmetry is the difference between reusing an answer and reusing a neighbourhood.

A concrete consequence is the case that a single-threshold router would get wrong: a query with a near-identical match to an episode whose stored answer was never trusted. The combined score stays below 0.90 because the trust term is low, so Tier 1 is correctly refused, the system will not serve the bad stored answer. But similarity is high, comfortably above 0.75, so the query routes to Tier 2 and the model generates a fresh answer for a question it has clearly seen before. The router declines to reuse the *answer* while still exploiting the *familiarity*.

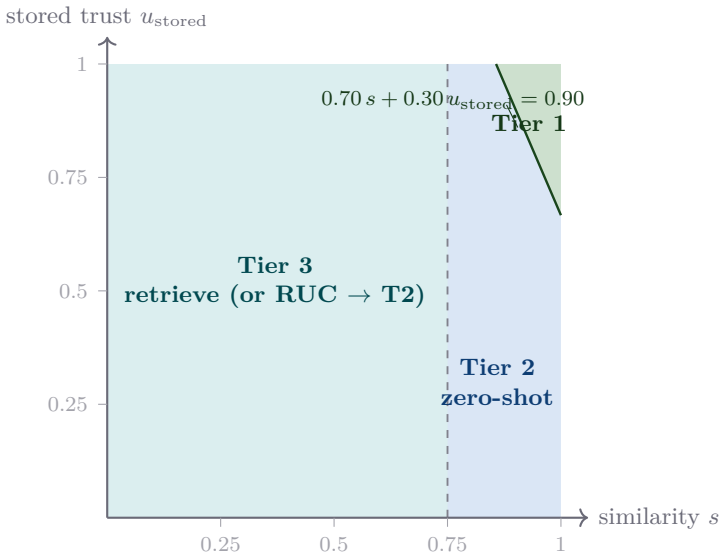


Figure 6.1: The dispatch score and the similarity rung carve the (similarity, stored-trust) plane into three regions. Tier 1 (green) is the top-right corner where the combined score clears 0.90: a close *and* trusted match. Tier 2 (blue) is the rest of the high-similarity strip ($s > 0.75$): close enough to generate fresh, even when the stored answer is not trusted. Tier 3 (teal) is everything to the left: no cheap path is earned, so the query retrieves, subject to the classifier of Chapter 7. The safety override of Mechanism 1 sits outside this plane: when $u_{\text{pre}} < 0.38$ it pre-empts the map entirely and forces the right edge. (Schematic.)

6.4 Why three mechanisms instead of one threshold

The router's design separates three questions that a naive dispatcher would conflate. The dispatch score captures *whether memory covers the query.* The safety floor captures *whether the model is ready to generate at all.* The classifier of Chapter 7 captures *whether retrieval will actually help on this particular query.* Each is a genuinely different question, and answering them with one number would force bad trades.

What a single threshold would get wrong

Suppose we collapsed everything into one score and one cutoff. Then a query with a strong memory match but low readiness would be served from the cheap tier, because the match dominates the number, exactly the confident-but-wrong case the safety floor exists to prevent. Or a query with weak memory coverage would be sent to retrieval even when retrieval is known to hurt it, because nothing in the single number asks whether retrieval helps. By keeping the three questions on three mechanisms, the router serves the cheap tier only when coverage and readiness *both* say it is safe, while the classifier governs the choice between zero-shot and retrieval on the queries that are left.

If an examiner asks: "how is this different from Adaptive-RAG?"

Prior three-tier routers such as Adaptive-RAG [16] gate their tiers with a single query-complexity classifier: the query's predicted difficulty alone decides the tier. CAEM's

router differs in two ways. It dispatches on *per-entry stored quality over a shared, evolving memory*, not on a static property of the query, so the same query routes differently as the memory learns. And it separates the three concerns above into three mechanisms evaluated in order, rather than folding them into one difficulty score. The retrieval-utility classifier of Chapter 7 is closest in spirit to the Adaptive-RAG idea, but it sits *downstream* of the memory-coverage and readiness mechanisms, refining only the queries those mechanisms could not place on a cheap tier.

6.5 The router at work

Five readings cover the router's whole behaviour. The first is the cold start we have already met; the rest are the four corners of Figure 6.1 plus the override.

Five queries, five routes

1. **Cold start (empty memory).** No episode exists, so similarity and stored trust are both zero, the score is zero, and similarity is below 0.75. The query falls through to Tier 3 (or the classifier redirects it to Tier 2). This is the FEVER claim of Chapter 2 on cycle zero.
2. **Close and trusted.** A near-identical match to a high-trust episode pushes the combined score past 0.90. Tier 1: the stored answer is served directly, no generation, no verification.
3. **Close but not trusted.** A near-identical match to a low-trust episode: the score stays under 0.90 because the trust term drags it down, but similarity clears 0.75. Tier 2: generate a fresh answer rather than serve the distrusted one.
4. **Loosely related.** Similarity below 0.75 and the score well under 0.90. The query falls through to Tier 3, where the classifier decides whether retrieval will help or whether zero-shot is the better bet.
5. **Unready, regardless of match.** The pre-routing confidence is below 0.38. The safety override fires first and pre-empts the entire map: the query goes to the grounded tier (or the classifier sends it to zero-shot) no matter how good the memory match looked. This is the TriviaQA question of Chapter 2.

These five readings are a conceptual map of the router's behaviour, but the deployed run shows how unevenly the tiers are actually used. The cheap memory tier (Tier 1) is exercised on only a tiny fraction of queries: none at the cold start, and roughly one in a thousand once memory has grown (two of fifteen hundred at the third cycle, both returning the correct stored answer). Live dispatch is therefore overwhelmingly a choice between zero-shot generation (about forty-five percent of queries) and grounded retrieval (about fifty-five percent), with Tier 1 acting as a precision-first cache for near-verbatim repeats rather than a high-traffic path. The five-scenario walkthrough still names every route the router can take; the deployed shares simply tell us which of those routes carry the traffic.

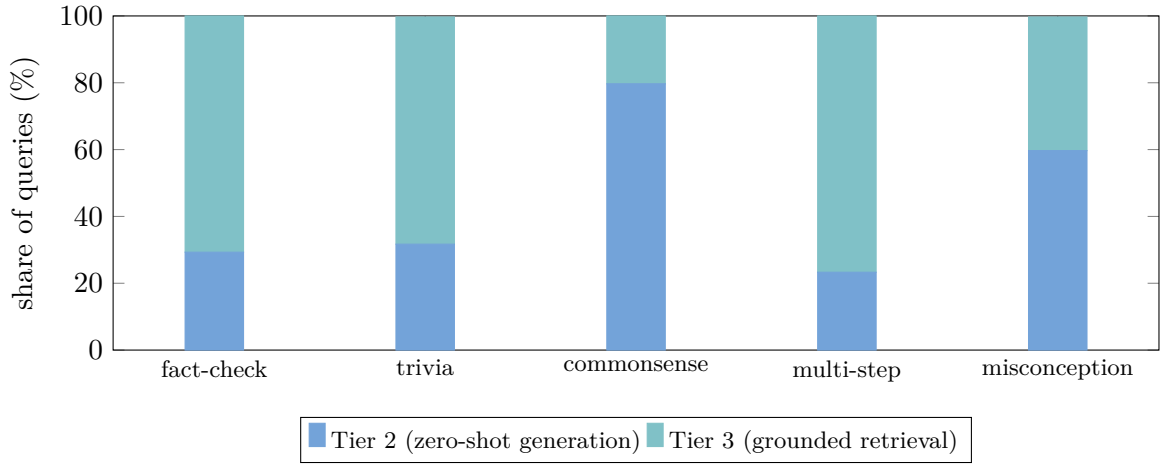


Figure 6.2: Actual deployed tier shares at the third cycle of the full run, per benchmark. The cheap memory tier (Tier 1) is a hairline at this scale and is omitted. The commonsense probe routes mostly to zero-shot generation (Tier 2), while the fact-checking, trivia, multi-step reasoning, and misconception probes lean on grounded retrieval (Tier 3). This is the measured counterpart to the conceptual regions of Figure 6.1.

Router constants

- Score blend: $\lambda = 0.70$ on similarity, 0.30 on stored trust.
- Tier 1 rung: combined score $\geq \theta_1 = 0.90$ (close *and* trusted).
- Tier 2 rung: similarity $> \theta_{\text{sim}} = 0.75$ (close enough to generate).
- Safety override: $u_{\text{pre}} < \phi_{\text{safe}} = 0.38$, checked before the score.
- Evaluation order is fixed: override, then Tier 1 rung, then Tier 2 rung, then fall-through.

The router has now chosen a tier for every query, but for two of its outcomes it deferred to a classifier we have only named. Chapter 7 opens that box, Stage 3b, the retrieval-utility classifier: the learned component that decides, for a fall-through query, whether retrieval will actually help or quietly hurt.

CHAPTER 7

Stage 3b: The Retrieval-Utility Classifier

This is the chapter the architecture builds toward. The router of Chapter 6 sends every query it cannot place on a cheap tier to retrieval, as a fallback. That fallback rests on an assumption that turns out to be false: that retrieval always helps. The retrieval-utility classifier is CAEM's correction. It is a small learned model that sits on the two fall-through paths and decides, per query, whether retrieval will genuinely help the generator or quietly hurt it. It is the newest component of the system and the one that most directly improves the architecture's behaviour on the questions where ordinary retrieval-augmented generation fails.

Retrieval is a tool, not a reflex

Retrieval-augmented generation is usually framed as strictly safer: ground the model in evidence and it hallucinates less. That is true on the questions where the evidence supports the right answer. It is false on a specific and important class of questions, the ones built around a common misconception or the ones the model already knows well, where the retrieved passages either reinforce the trap or distract from a stronger parametric answer. On those, retrieval makes the answer *worse*. The retrieval-utility classifier learns to tell the two cases apart, so that retrieval becomes a tool used when it helps rather than a reflex applied to everything.

7.1 The wrong-default failure mode

The empirical anchor for this chapter is a regression the architecture showed *before* the classifier existed. On misconception-bait benchmarks, exact-match accuracy declined cycle over cycle. The cause was not the verifier and not the memory; it was the unconditional retrieval dispatch feeding the self-improvement loop. When a misconception question was sent to retrieval, the retrieved passages nudged the generator toward the popular-but-wrong answer, the verifier sometimes let that confident answer through, and the self-improvement loop of Chapter 12 then folded the retrieval-amplified error into the model's own weights. Each cycle made the model a little more confidently wrong on exactly the questions designed to catch it.

The tier-conditional self-improvement hypothesis

The classifier's deeper purpose is to make self-improvement *benchmark conditional*. The self-improvement loop trains on the answers the system produced and trusted, so the *tier* that produced an answer determines what the model learns from it. If retrieval-hurt queries are routed to retrieval, the loop ingests retrieval-polluted answers and the model drifts on those benchmarks. If instead those queries are routed to zero-shot generation, the loop ingests clean parametric answers and the model improves. The classifier is therefore not only a per-query cost optimiser; it is the gate that decides *which kind of answer enters the training pool* for each query. Routing retrieval-helpful queries to the grounded tier and retrieval-hurt queries to the zero-shot tier lets the loop learn grounded reasoning where grounding helps and clean parametric reasoning where it does not, which is what allows one self-improvement loop to raise accuracy across a panel of benchmarks that reward opposite things.

The asymmetric-rescue payoff, with one caveat

The deployed on/off ablation puts numbers on the mechanism. Turning the classifier on lifts pooled exact match by about 5.8 percentage points and collapses the retrieval-tier share by about 34 points, and the gains are concentrated exactly where retrieval was hurting: the commonsense probe rises by about 14 points and the misconception probe (exact match judged by the language-model judge) by about 10 points, while the retrieval-helpful fact-checking benchmark stays flat. The pattern is the asymmetric rescue the hypothesis predicts: lift the benchmarks retrieval was hurting without disturbing the one it was helping. One caveat keeps the contrast honest. This is a bundled on/off comparison, with the classifier enabled alongside concurrent pipeline corrections, not a clean classifier-only isolation, so the figures bound the joint effect of the change rather than attribute every point to the classifier alone.

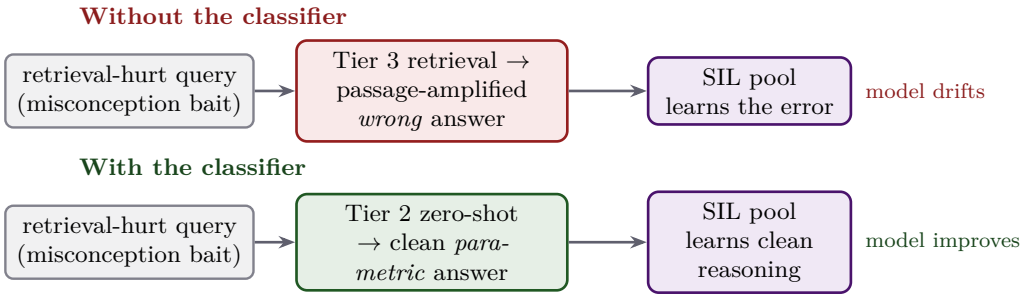


Figure 7.1: Why the classifier matters beyond cost. On a retrieval-hurt query, the unconditional dispatch (top) sends a passage-amplified wrong answer into the self-improvement pool, and the model drifts on that benchmark cycle over cycle. The classifier (bottom) routes the same query to zero-shot generation, so a clean parametric answer enters the pool and the model improves. The classifier decides not just which tier answers, but which kind of answer the loop learns from. (Schematic.)

7.2 Where the classifier fires, and where it does not

The classifier is a refinement, not a replacement, so it is important to be exact about its reach. It is consulted at exactly the two points in Chapter 6 that would otherwise route unconditionally to retrieval: the safety-override path and the score fall-through. At both, the router's default is the grounded tier, and the classifier is given the chance to redirect that default to zero-shot generation instead.

The classifier never overrides a cheap-tier decision

A query that earns Tier 1 on its combined score, or Tier 2 on its similarity, never consults the classifier at all. The memory-direct and similarity-based zero-shot decisions stand on their own. The classifier only ever chooses between the grounded tier and the zero-shot tier *for queries the router was about to send to retrieval.* Its job is to split the retrieval fallback into "really retrieve" and "actually, generate directly," and nothing more. This keeps its influence localised and its failures bounded: a misfire can only move a query between the two most expensive tiers, never onto the memory-direct path.

Concretely, the classifier reads a feature vector for the query, produces a probability p_{RAG} that retrieval is the better path, and the router escalates to the grounded tier if and only if that probability clears a threshold. When no classifier is wired in, or the query text is unavailable, the router falls back to its old behaviour and routes the fall-through to retrieval, so the classifier degrades safely to the pre-classifier system.

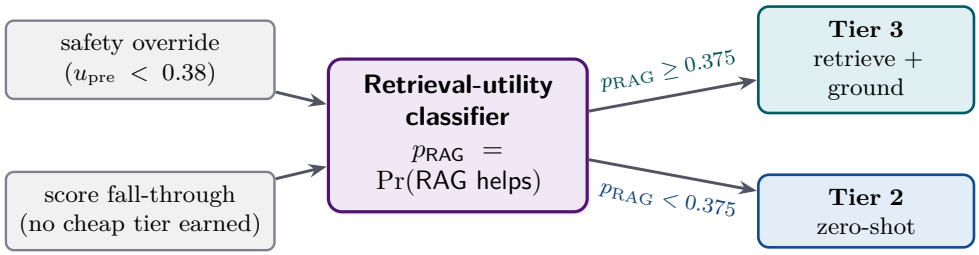


Figure 7.2: The classifier intercepts the two router outcomes that default to retrieval. It emits a probability that retrieval helps; the router commits to the grounded tier above the threshold and to zero-shot generation below it. Tier 1 and similarity-based Tier 2 decisions never reach this box. (Schematic.)

7.3 What the classifier is

For all its importance, the classifier is deliberately simple: a logistic regression on a standardised feature vector. There is no neural network, no fine-tuning, and no dependence on the generator's weights at inference. Thirty-six engineered features are assembled for the query, standardised to zero mean and unit variance, and fed to a logistic regression that outputs p_{RAG}. The decision is a threshold at $\tau_{\mathrm{RUC}} = 0.375$.

Listing 7.1: The classifier's prediction, condensed from `caem/routing/ruc.py`.

```
feats = build_feature_vector(question,                  # 23 surface-form
    features
                             top1_passage_sim,          # +
                                 retrieval-evidence features
                             top1_passage_entity_overlap,
                             p_ik,                       # +
                                 self-knowledge probe
                             **extra_features)           # 36 features in
                                 total
x     = [feats[f] for f in feature_order]
x_s   = scaler.transform([x])                           # standardise
    (zero mean, unit var)
p_rag = lr.predict_proba(x_s)[0, 1]                     # P(retrieval is
    the better path)
decision = "RAG" if p_rag >= 0.375 else "DIRECT"        # tau_RUC = 0.375
```

Why a logistic regression and not a neural network

A more powerful model is tempting, but three reasons make the linear choice the right one here. First, the decision is binary and the signal is concentrated in a handful of features, so a linear boundary already separates the classes well. Second, a logistic regression's coefficients are directly readable, which lets us audit *why* the classifier routes a query the way it does, a property we use in Section 7.5. Third, with only a few thousand labelled queries, a linear model with strong regularisation generalises where a larger model would overfit. The classifier is small on purpose; its leverage comes from the labels it was trained on, not from its capacity.

7.4 The features, by family

The thirty-six features fall into three families, distinguished by what they cost to compute and what they capture. Table 7.1 lays them out.

The three families answer three complementary questions. The surface-form family asks *what kind of question is this*, catching the misconception-bait and opinion-seeking phrasings where retrieval tends to hurt. The retrieval-evidence family asks *does the corpus actually have something useful*, since retrieval cannot help when the passages are irrelevant or disagree with one another. And the self-knowledge probe asks *does the model already know this*, because a question the model can answer from its own parameters gains nothing from retrieval and may be distracted by it.

Table 7.1: The thirty-six features, by family. The surface-form family is computed inline from the question text; the retrieval-evidence family is supplied by the pipeline from a cheap top-passage lookup; the self-knowledge probe is read from the base model.

Family	Count	What it captures
Question surface form	23	Cheap, text-only cues: question length, presence of negation, temporal, numerical, and misconception-bait phrasing; the interrogative type (who, what, when, where, why, how, yes-no); named-entity count and entity popularity; adversarial, opinion-seeking, and open-ended phrasing; and answer-type matches between the question and the top passage (does the question want a year or a number, and does the passage contain one).
Retrieval evidence	12	Whether the corpus actually has good evidence: the top passage's similarity and entity overlap with the question; the top-five similarity spread (max, mean, standard deviation); cross-encoder rerank scores (top one, top-five mean and spread); pairwise entailment among the retrieved passages (mean, minimum, spread), which measures whether the passages agree; and a binary Wikidata entity hit.
Self-knowledge	1	The probe p_{IK}: the base model's own estimate of whether it can answer the question correctly without retrieval. This single feature carries the most weight of any in the model.

7.5 The star feature: the self-knowledge probe

If one feature defines the classifier, it is the self-knowledge probe p_{IK}. It is by far the strongest, with a coefficient about two and a half times the next largest in absolute value (about -1.77 on standardised features against about $+0.70$ for the next feature), and its sign is exactly what the design intends.

What the probe is, and why the adapter is switched off

The probe is a direct-correctness estimator in the style of Kadavath et al. [7]: a small head reads the base model's last-token hidden state on the question and predicts the probability that the model would answer correctly. Its coefficient in the classifier is strongly negative (about -1.77 on standardised features), which is exactly right: a high probe value means the model already knows the answer, so retrieval is unlikely to help and the query should go to zero-shot generation. On its own the probe reaches a mean out-of-fold area under the ROC curve of about 0.77, so it is usefully predictive but imperfect, which is precisely why it is one feature among many rather than the whole decision. One implementation detail is load-bearing: the probe is computed with the low-rank adapter of Chapter 12 *disabled*, so that it reads the frozen base model's self-knowledge on the same distribution the probe was trained on, rather than the shifting fine-tuned distribution. Measuring the base model's knowledge, not the adapter's, keeps the signal stable across cycles.

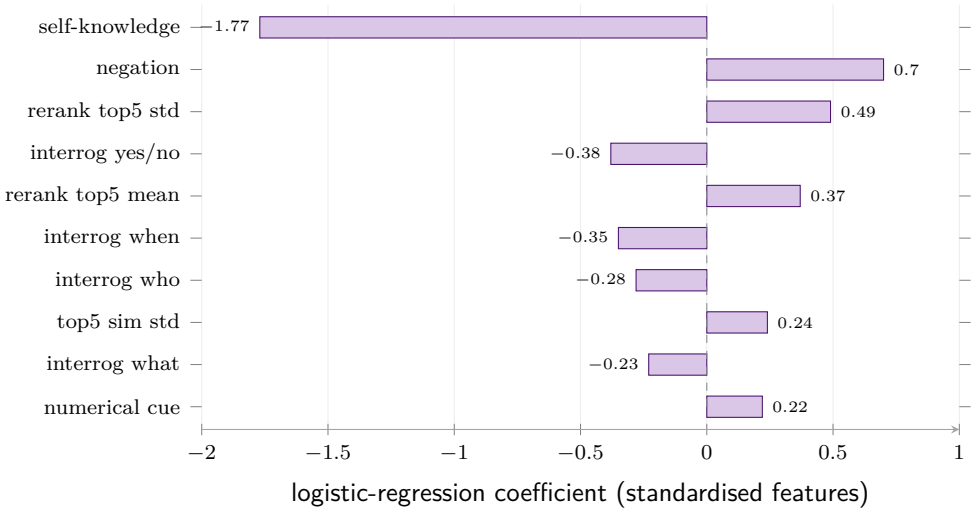


Figure 7.3: The ten features with the largest coefficients. Negative coefficients (left) push toward zero-shot generation; positive coefficients (right) push toward retrieval. The self-knowledge probe p_{IK} dominates, and its strong negative weight encodes the intended rule: the more the model already knows, the less it should retrieve. The negation cue carries the largest positive weight, with the rerank-spread features close behind, escalating to retrieval when the corpus has useful, varied evidence. (Learned coefficients from the deployed classifier.)

7.6 How the classifier was trained

The classifier's leverage comes from its labels, so the labelling deserves care. The question for every training query is not whether the answer is right, but whether *retrieval* produced a better answer than *zero-shot generation*. To label a query, both paths are run: a zero-shot answer and a retrieval-augmented answer are each scored through the verifier of Chapter 9. The query is labelled positive (retrieval helps) when the retrieval answer wins and negative (retrieval hurts) when the zero-shot answer wins.

Train only on the disagreements

Most queries are ties: both paths get it right, or both get it wrong. Those carry no signal about *which path to choose*, and including them would swamp the classifier with uninformative examples and waste expensive verifier calls. So the training set is filtered to the queries where the two paths *disagree*, one right and one wrong. This is both a statistical choice, concentrating the model on the decision boundary that matters, and a cost choice, since only the disagreements need the double verifier scoring. The shipped classifier was trained on roughly 3,200 such disagreement queries.

Training and validation

- Training set: 3,208 disagreement queries (1,846 retrieval-helps, 1,362 retrieval-hurts) drawn from *eight* benchmarks, deliberately broader than the five-benchmark CAEM panel so the classifier learns a transferable retrieval-utility signal.
- Model selection: nested cross-validation, five outer folds and five inner folds, with the regularisation strength and the decision threshold both chosen on held-out inner

folds (final strength $C = 0.5$, threshold $\tau_{\text{RUC}} = 0.375$).

- Discrimination: pooled out-of-fold area under the ROC curve of 0.850, with per-benchmark values ranging from 0.69 on OpenBookQA to 0.94 on TriviaQA.

The nested cross-validation matters because two things are being chosen at once, the regularisation strength and the threshold, and choosing either on the data used to report performance would inflate the result. The outer folds give an honest estimate; the inner folds do the choosing. The threshold landed below one half, at 0.375, because the cost of the two errors is asymmetric and the sweep that maximised net utility preferred to retrieve a little more readily than a symmetric cutoff would.

7.7 Honest limitations

Two caveats keep the classifier in proportion.

Not every coefficient sign matches the naive intuition

A registered audit checks whether the learned coefficients carry the signs the design expects. The self-knowledge probe passes cleanly, and the rerank score passes, but the top-passage similarity does not: its coefficient is mildly negative where a naive reading would expect positive, so the audit records an overall fail on that check. The likely reason is correlation among the retrieval features, with the richer rerank and passage-agreement signals absorbing the useful part of raw similarity and leaving its residual coefficient small and oppositely signed. It is reported transparently rather than hidden, and it does not change the deployed behaviour, since discrimination is measured on held-out folds, not on coefficient signs. It is a reminder that a linear model's individual weights are interpretable only up to the correlations among its inputs.

Discrimination is uneven across benchmarks

The pooled discrimination of 0.850 hides real variation. The classifier separates the two classes very well on TriviaQA and HaluEvalQA, and only modestly on FEVER, OpenBookQA, and CommonsenseQA, where the retrieval-helps and retrieval-hurts queries look more alike on these features. The classifier is therefore most valuable on the open-domain factoid and misconception-bait benchmarks and least decisive on the short, self-contained ones, which is consistent with where the wrong-default failure mode bites hardest.

7.8 The classifier in the lineage of selective retrieval

If an examiner asks: "how is this different from prior selective-retrieval work?"

The idea that retrieval should be conditional is not new, and the thesis does not claim it. A line of work reaches the same diagnosis: that an unconditional retrieval dispatch leaves accuracy on the table relative to a per-query routing decision, including when-not-to-retrieve on PopQA [17], self-knowledge routing [18], Adaptive-RAG [16], and Probing-RAG [19]. CAEM's classifier sits in that lineage with two distinguishing features. It is positioned *downstream* of a memory-coverage and a readiness mechanism, so it refines only the residual queries those mechanisms could not place, rather than gating every query. And its labels are tied to CAEM's own verifier-scored disagreements between the two paths, so it learns retrieval utility as *this* system experiences it, and feeds directly into the tier-conditional self-improvement of Section 7.1, which the selective-retrieval line does not address because those systems do not have a self-improvement loop to protect.

7.9 The two readings, refined

Both of our running queries pass through the classifier, and it sends them opposite ways.

The FEVER claim: redirected to zero-shot

The claim "The French Revolution began before 1792" reaches the classifier through the score fall-through on the empty cycle-zero memory. The query is short, self-contained, and the kind of fact the base model knows, so the self-knowledge probe reads moderately high and the surface-form features mark it as a simple declarative claim. The classifier returns a low p_{RAG}, below 0.375, and the router redirects the default retrieval dispatch to zero-shot generation. The model labels the claim from its own knowledge, which is the right call and avoids dragging in passages that could only muddy a fact it already holds.

The TriviaQA question: confirmed for retrieval

The question " 'If I Ruled The World' comes from which musical?" reaches the classifier through the safety override. Here the self-knowledge probe reads low, the model does not confidently know the answer, and the retrieval-evidence features show that the corpus holds relevant, agreeing passages. The classifier returns a high p_{RAG}, above 0.375, and confirms the grounded dispatch. Retrieval is genuinely the better path here, and the classifier agrees with the safety override rather than countermanding it.

These two cases are the classifier's whole job in miniature: send the question the model already knows to its own parameters, and send the question it does not know to the evidence. Chapter 8 now takes up Stage 4, where the chosen tier finally runs and produces an answer to be judged.

CHAPTER 8

Stage 4: Tier Execution

The router has chosen a tier. Stage 4 is where that choice is finally carried out and an actual answer appears. There are three ways to produce one, and they differ enormously in cost: read a verified answer straight from memory, generate one from the model's own knowledge, or retrieve evidence and generate a grounded answer. This chapter walks all three, the shared reasoning format they obey, and the small escalation valve that connects the two generating tiers.

Three ways to answer, in rising order of effort

Faced with a question, a careful person has three moves. If they have answered it before and remember the answer, they just say it. If they know the area well, they reason it out from what they already know. And if they are unsure, they look it up first and then reason from the source. The three tiers are exactly these moves, made mechanical. The router picked the cheapest move it judged safe; Stage 4 performs it.

8.1 The shared contract: scaffolded chain of thought

Before the tiers themselves, one convention they share. Both generating tiers, and the stored episodes that the memory tier serves, follow the same answer format: a *Reasoning* section followed by a single *Answer* line. This is scaffolded chain-of-thought [11], and the uniformity is deliberate and load-bearing.

Why force the same shape everywhere

The verifier of Chapter 9 reads every answer the same way: it looks for an explicit chain of reasoning and a cleanly extractable final answer. If a fact-checking claim, a trivia question, and a multiple-choice item each came back in a different shape, the verifier would be comparing apples to oranges, and its signals would mean different things on different benchmarks. Forcing one shape everywhere gives the verifier a uniform surface to score, which is precisely what lets a single set of calibrated thresholds govern all five benchmarks. The reasoning scaffold is not just for the model's benefit; it is the contract the rest of the pipeline reads against.

The model is steered into this shape two ways. A system instruction states the format explicitly and makes the Answer line mandatory, and the generation is *prefilled* with the literal token "Reasoning:" so the model begins inside the scaffold rather than deciding whether to use it. A representative prompt skeleton, with the retrieval block that only Tier 3 adds, looks like this.

Listing 8.1: The scaffolded-CoT prompt skeleton shared by Tiers 2 and 3 (Tier 3 adds the Context block).

```
<|im_start|>system
Answer with a Reasoning section, then an Answer line. The Answer line is
    mandatory.
Reasoning: <concise step-by-step thought, 3-4 sentences>
Answer: <final answer in the specified format><|im_end|>
<|im_start|>user
Context:                        # <-- Tier 3 only
[1] <retrieved passage 1>       #     top-5 passages, numbered
...
[5] <retrieved passage 5>
Question: <the query><|im_end|>
<|im_start|>assistant
Reasoning:                      # <-- forced prefix (prefill); the model
    continues here
```

The mandatory Answer line is a hard-won fix

An earlier version of the instruction left the Answer line optional, and the cold-start audit found that on between forty-three and fifty-three percent of samples the model emitted only a Reasoning section and never reached an Answer line, so the user-facing result was empty. The silent cost was worse than it first appeared: on the fact-checking benchmark, about a third of those empty results in fact carried reasoning whose conclusion already matched the gold verdict, so they were correct answers discarded for a structural reason rather than a semantic one. The current instruction makes the Answer line explicitly mandatory and caps the reasoning length so the model reliably reaches it. The lesson generalises: when a downstream component depends on a structural feature of the output, that feature has to be demanded in the prompt, not hoped for. The format is steered further by short per-benchmark worked examples and an answer-format specification, so a fact-check returns a verdict word, a yes-no question returns yes or no, and a multiple-choice item returns a single option letter.

8.2 Tier 1: serve from memory

The cheapest tier is barely a computation at all. When the router commits to Tier 1, the matched episode already holds a verified answer and its trust score, so the tier reads them straight off the entry and returns. There is no generation and, crucially, no verification.

Listing 8.2: Tier 1, in full, from `caem/pipeline.py`.

```
def _tier1(self, search_with_ids):
    entry, entry_id, similarity = search_with_ids[0]
    return entry.answer, entry.u_stored      # stored answer + its trust
        score; no model call
```

Why Tier 1 may skip the verifier

Skipping verification looks dangerous until we recall what a Tier 1 hit means. The episode was admitted to memory only because it passed the four-outcome gate of Chapter 10 on some earlier cycle, so its answer was *already judged trustworthy*, and its trust score was written into the entry at that time. Re-running the nine-signal verifier now would re-derive a verdict the system already holds. So Tier 1 reads the stored verdict instead of recomputing it, returning in the time of a single index lookup with no generation, which makes it by construction far cheaper than the generating tiers; the production design's qualitative target is sub-second serving rather than a measured per-tier latency, since the batched evaluation logs cannot be decomposed into a per-tier figure. The cost of judging a fact is paid once, on the cycle it entered memory, not on every later query that matches it.

A Tier 1 serve does leave one trace: it updates the episode's usage statistics, the retrieval count and running success rate of Section 5.5, so that a heavily served episode's standing can drift with experience and so that a popular episode is flagged for re-checking. Reading the answer is free; recording that it was read is the only bookkeeping.

8.3 Tier 2: zero-shot generation

When the router judges the query familiar enough to generate but has no servable stored answer, it picks Tier 2. The model generates an answer from its own parameters, conditioned only on the query, with no retrieved evidence. This is where the self-improvement loop's gains show up at inference time: a query that needed retrieval in an early cycle can be answered zero-shot once the loop has folded that knowledge into the adapter. Across the deployed trajectory the zero-shot tier answered its own assigned queries at least as accurately as the more expensive retrieval tier answered its assigned queries, with its exact match holding near or above that of the retrieval tier in every cycle (about 0.57 by the third cycle against about 0.48 for the retrieval tier at the cold start). This is a per-tier-on-assigned-queries comparison, not a claim that retrieval is globally inferior, and it is precisely the inference-time signature one expects when the loop has folded retrieval-found knowledge into the model's own parameters.

Listing 8.3: Tier 2 generation, condensed from `caem/pipeline.py`.

```
prompt_text, forced_prefix = build_tier2_prompt(query, tokenizer)    # scaffolded ChatML
full_input = prompt_text + forced_prefix                              # prefill "Reasoning:"

output_ids = model.generate(input_ids, max_new_tokens=512,            # cot_max_new_tokens
                            do_sample=False)                          # greedy, reproducible
continuation = tokenizer.decode(output_ids[0, input_len:])            # only the new tokens
```

```
answer_str = f"{forced_prefix}{continuation}"                    # re-attach "Reasoning:"
```

Three details are worth drawing out. The generation is *greedy*, so the same query under the same weights yields the same answer, which keeps the whole pipeline reproducible. Only the newly generated continuation is decoded, and the forced prefix is re-attached, so the returned string always begins with "Reasoning:" and satisfies the scaffold contract. And the generation budget is capped at five hundred and twelve new tokens, enough headroom for a few-shot example, a short reasoning section, and the Answer line. The result is then handed to the verifier of Chapter 9; unlike Tier 1, a fresh Tier 2 answer has never been judged, so it must be.

8.4 Tier 3: retrieval-augmented generation

The most expensive tier retrieves evidence before generating. It is the architectural floor, the path the router falls back to whenever the cheaper tiers cannot earn the query, and the path the classifier of Chapter 7 reserves for questions where grounding genuinely helps.

The passage store

Tier 3 retrieves from a read-only store of roughly twenty-one million Wikipedia passages, each a hundred-word chunk in the dense-passage-retrieval style [20]. The passages are embedded with the *same* sentence encoder used for episodic memory (Section 3.2), so queries, stored episodes, and corpus passages all live in one 768-dimensional space. The index is the approximate inverted-file product-quantised structure of Section 3.3, sized for the corpus, and it is memory-mapped from disk so that the sixty-four-gigabyte body stays on disk and only the active cells are paged in. Retrieval uses the high-recall probe setting, scanning four times as many index cells as the cheap memory-hit confirmation lookups elsewhere in the system (sixty-four probed cells against sixteen), because on a Tier 3 query recall matters more than the last millisecond of latency.

The generation itself mirrors Tier 2, with one addition: the top five retrieved passages are formatted into a numbered Context block above the question, as in the skeleton of Listing 8.1. The retrieve-then-read structure is the standard retrieval-augmented-generation recipe [10], and the model is asked to reason from the supplied passages rather than from memory alone.

Listing 8.4: Tier 3 retrieve-then-generate, condensed from `caem/retrieval/rag.py`.

```
passages = self._retrieve(query, k=5)                    # top-5 from the 21M-passage index
prompt_text, forced_prefix = build_tier3_prompt(query, passages, tokenizer)  # adds Context block
output_ids = model.generate(input_ids, max_new_tokens=512, do_sample=False)  # greedy, grounded
answer = f"{forced_prefix}{tokenizer.decode(output_ids[0, input_len:])}"
        # scaffold contract
```

Like a Tier 2 answer, a Tier 3 answer is fresh and unjudged, so it passes through the

verifier and the gate, and a Tier 3 answer that verifies well is stored as a new episode, so that a future similar query can be served from the cheaper tiers. This is the mechanism by which the compute ladder *lowers* over time: yesterday's expensive retrieval becomes tomorrow's memory hit.

8.5 The escalation edge

One connection runs directly between the two generating tiers. If a Tier 2 generation comes back empty, or the generation call fails outright, the pipeline does not return an empty answer; it quietly escalates that single query to Tier 3 and marks the result as escalated.

The reported tier still reads 2 after an escalation

The escalation is a robustness valve, not a re-routing. When it fires, the answer comes from Tier 3, but the routing decision is not rewritten: the recorded tier stays at two, with a separate flag noting that the query escalated. Anyone reading the logs should know that an escalated Tier 2 result was in fact produced by retrieval. The valve exists because an empty zero-shot answer is worse than a grounded one, so a failed cheap attempt falls forward to the expensive path rather than failing the query.

8.6 What leaves Stage 4, and where it goes

Table 8.1 collects the three tiers side by side, and the single most important column is the last one. A Tier 1 answer carries a trust score already and goes straight to the output stage. A Tier 2 or Tier 3 answer is a fresh, unjudged string, and it goes to the verifier of Chapter 9 before the system will trust it, store it, or refuse it.

Table 8.1: The three execution tiers. Only the two generating tiers produce a fresh answer that must be verified; the memory tier serves an answer that was already judged on a past cycle.

Tier	Mechanism	Generates?	Verified now?
1 (memory-direct)	read the stored answer and its trust score off the matched episode	no	no (judged on a past cycle)
2 (zero-shot)	generate from the model's own knowledge, query only	yes	yes
3 (retrieval)	retrieve top-5 passages from the 21M-passage corpus, then generate grounded	yes	yes

Tier-execution constants

- Generation: greedy (deterministic) on the frozen backbone plus its adapter; cap 512 new tokens.
- Forced prefix: every generated answer is prefilled with "Reasoning:" to enter the

scaffold.

- Tier 3 retrieval: top 5 passages from ≈ 21 million Wikipedia chunks; one shared 768-dimensional embedding space; high-recall probe setting; index memory-mapped from disk.
- Escalation: an empty or failed Tier 2 generation falls forward to Tier 3, flagged as escalated.
- Verification: Tiers 2 and 3 go to the verifier; Tier 1 does not.

If an examiner asks: "why not verify Tier 1 answers too, to be safe?"

Two reasons, one about correctness and one about cost. On correctness, a Tier 1 answer was admitted to memory only by passing the gate, and the memory is re-judged in full at every *committed* cycle boundary by the retroactive re-verification of Chapter 12, so a stored answer is rarely more than a cycle old without a fresh verdict. On cost, the entire value of the memory tier is that it answers recurring queries for the price of a lookup; verifying every Tier 1 hit would erase that saving and make the cheapest tier nearly as expensive as the others. The design verifies a fact when it enters memory and re-verifies it each cycle, rather than on every serve, which is where the asymmetry of the architecture pays off.

Every tier has now produced an answer, and for the two generating tiers that answer is still unjudged. Chapter 9 opens Stage 5, the nine-signal verifier, the component that decides how much to trust a freshly generated answer.

CHAPTER 9

Stage 5: The Nine-Signal Verifier

A tier has produced a fresh answer. Stage 5 is where the system decides how much to believe it. This is the heart of CAEM's trust machinery and the direct answer to the overconfidence flaw of Chapter 1: rather than read the model's own confidence, which we saw is barely better than chance, the verifier gathers many independent signals about the answer and weighs them together. This chapter covers how each raw signal is produced. The next chapter, Chapter 10, covers how they are calibrated, fused into one number, and turned into a decision.

A panel of specialists, not a single judge

No single test reliably catches a confident falsehood. So the verifier convenes a panel. One specialist asks whether the model was internally sure. Another asks whether the model agrees with itself when made to answer several times. A third checks the answer against retrieved evidence, sentence by sentence. A fourth asks whether the answer even addresses the question. Each specialist is fallible alone, but a confident lie has to fool all of them at once, and that is much harder than fooling any one. The verifier's strength is not any single signal; it is the diversity of the panel.

9.1 Four families, nine headline signals

The signals group into four families, each probing a different way an answer can be wrong. Figure 9.1 is the map to keep in view for the whole chapter.

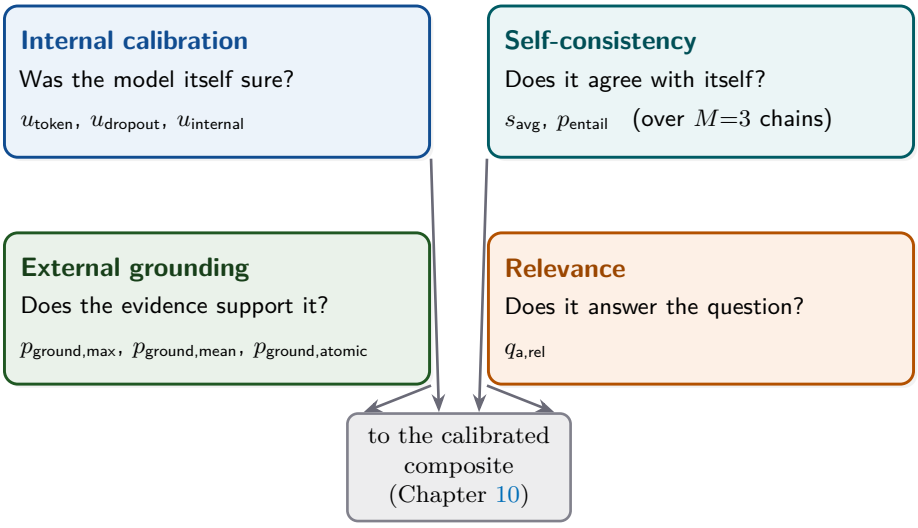


Figure 9.1: The nine headline signals, grouped into four families, each probing a different failure mode. Every family feeds the calibrated composite of Chapter 10. A confident wrong answer must satisfy all four at once to be trusted. (Schematic.)

"Nine signals" is exact: the composite computes more than it uses

The verifier is named for the nine signals of Figure 9.1, and that is exactly how many drive a decision: the calibrated composite of Chapter 10 reads precisely those nine. The verifier nonetheless *computes* more than it uses. Two entity checks, an alias overlap and a head-noun consistency (Section 9.6), were trialed in an earlier revision and then retired from the composite; two original signals, a semantic-entropy measure and a contradiction probability (Section 9.9), were retired as well. All four are still computed and recorded for diagnostics, but none contributes to the trust score in the deployed configuration. So "nine" is not a headline simplification; it is the exact count of signals that move the gate.

9.2 How the signals are produced, cheaply

Several signals would be expensive computed naively, so the verifier produces them from a small number of shared generations. The key economy is that the model is made to answer the same query a few times, and those few generations are reused across multiple signals. Figure 9.2 traces the production.

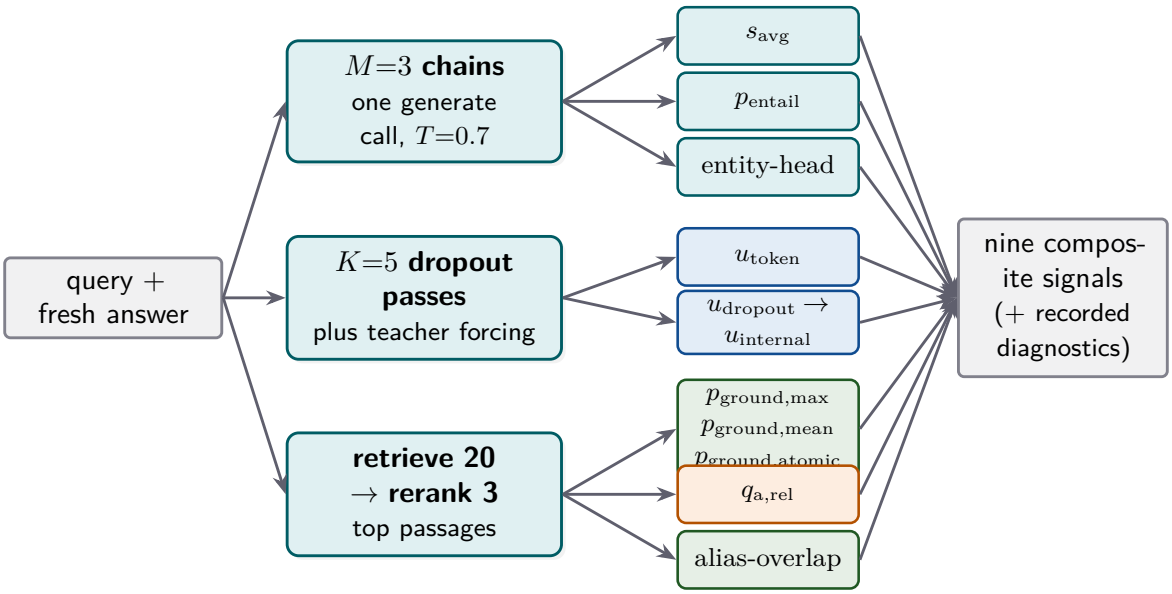


Figure 9.2: The verifier produces all signals from three shared computations: one generation of $M{=}3$ reasoning chains (reused by three signals), $K{=}5$ dropout passes plus a teacher-forcing pass for the internal signals, and one retrieve-and-rerank for the grounding and relevance signals. Generating the three chains once and reusing them is the central economy of the stage. (Schematic.)

The canonical answer claim: one surface for every signal

Before any signal reads the answer, the answer is rewritten into a *canonical claim*, and every signal that touches the answer reads that same canonical form. A multiple-choice letter such as "C" is expanded to its full option text; a bare entity such as "Paris" is wrapped as "Answer: Paris."; an already-declarative answer such as a fact-check

verdict passes through unchanged. The reason is uniformity: the grounding signals, the entailment signal, and the relevance signal must all score the *same* propositional surface, or their numbers would not be comparable across benchmarks and the single calibrated threshold of Chapter 10 could not govern them all. Canonicalisation is the small, unglamorous step that makes the whole panel speak one language.

9.3 Family one: internal calibration

The first family reads the generator itself, at no external cost. It contains three signals. The token confidence u_{token} is the geometric mean of the per-token probabilities of the answer, the same quantity we met in Chapter 4, here computed by teacher-forcing the produced answer rather than from a short decode. The dropout disagreement u_{dropout} measures stability under perturbation: the model is made to answer five times with dropout left switched on, and the signal is the fraction of those five samples that disagree with the plurality answer. The two are combined into a single internal confidence,

$$u_{\text{internal}} \;=\; 0.5\,u_{\text{token}} \;+\; 0.5\,(1 - u_{\text{dropout}}),$$

weighting token reliability and answer stability equally.

Listing 9.1: Dropout disagreement, condensed from `caem/verification/verifier.py`.

```
samples = model.generate(num_return_sequences=5, do_sample=True,     #
    K=5, dropout ON
                         temperature=0.7)                            #
                             model.train() mode
answers = [extract_answer(s) for s in samples]                       #
    short form of each
plurality = most_common(answers)
u_dropout = 1.0 - answers.count(plurality) / 5                       #
    fraction that disagree
```

Two ways to be unsure

The two internal signals catch different doubts. Token confidence is low when the model hedged *within* a single answer, spreading probability over many next tokens. Dropout disagreement is high when the model is unstable *across* answers, landing on different conclusions when nudged. A model can be fluent within one answer yet unstable across attempts, or stable across attempts yet hesitant within each; combining the two catches both. Both are cheap and require no external model, which is why they anchor the panel.

9.4 Family two: self-consistency

The second family makes the model talk to itself. It generates $M = 3$ reasoning chains for the query, sampled at a temperature of 0.7, in a single batched call, and two signals read

those three chains. The semantic consistency s_{avg} is the mean similarity across all unique pairs of chains, measured by embedding each chain and taking cosine similarity; high values mean the model keeps reaching the same place by different routes [12]. The chain-to-answer entailment p_{entail} asks a sharper question: does each chain actually *entail* the served answer? Each chain is run through the natural-language- inference judge against the canonical answer, and the signal is the mean entailment.

Agreement is not the same as support

Why two signals from the same three chains? Because chains can agree with each other yet not support the answer that was served. Imagine three chains that all reason toward one conclusion, but the answer the system actually returned was a different one: s_{avg} would be high (the chains agree) while p_{entail} would be low (they do not entail the served answer). That gap is a real failure mode, an answer disconnected from the reasoning meant to justify it, and it is exactly what p_{entail} exists to catch. Generating the chains once and reading them two ways is both cheaper and more informative than two separate tests.

9.5 Family three: external grounding

The third family is the strongest evidence that an answer is true rather than merely confident: does retrieved evidence support it? The verifier retrieves twenty candidate passages for the query, reranks them with a cross-encoder to the best three, and scores the canonical answer against those three passages with the natural-language-inference judge. Three signals read the result, and they differ in how forgiving they are.

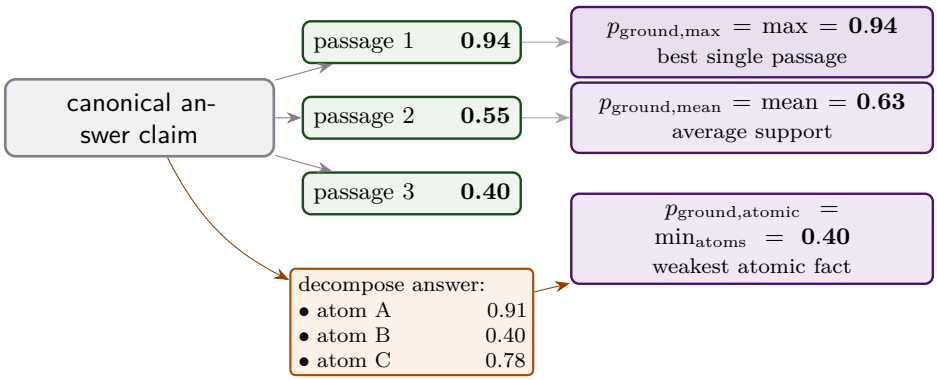


Figure 9.3: Three readings of the same evidence, in rising strictness. $p_{\text{ground,max}}$ takes the single best passage and is the most forgiving; $p_{\text{ground,mean}}$ averages the top three; $p_{\text{ground,atomic}}$ decomposes the answer into atomic facts, scores each against the passages, and reports the *weakest* one. A long answer with one unsupported fact passes the first two and fails the third. (Numbers are illustrative.)

Max, mean, and the weakest link

The maximum grounding $p_{\text{ground,max}}$ is the entailment of the single best-supporting passage, the most forgiving reading; it is also the signal the early-exit gate of Section 9.7

watches. The mean grounding $p_{\text{ground,mean}}$ averages over the three passages, a steadier estimate of overall support. The atomic grounding $p_{\text{ground,atomic}}$ is the strict one: it decomposes the answer into atomic facts in the FActScore style [2], scores each fact against the passages, and returns the *weakest* fact's support, on the principle that one unsupported claim in an otherwise-grounded answer is still a fabrication. Because decomposition is meaningless for very short answers, it is gated by length: answers under twelve tokens skip it and fall back to the mean. The three together span the range from "is there any support at all?" to "is every part supported?"

The two natural-language-inference judges

The grounding and entailment signals all rest on a judge that scores whether a premise entails a hypothesis. CAEM uses two judges and dispatches between them by the length of the hypothesis, as Figure 9.4 shows.

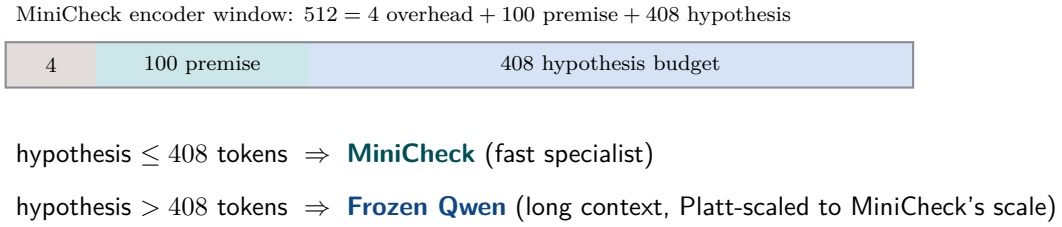


Figure 9.4: Entailment is judged by a fast specialist for short hypotheses and a long-context model for long ones. The split point of 408 tokens is the hypothesis budget left once the specialist's 512-token window reserves four tokens of overhead and a hundred tokens of premise. The long-context judge is Platt-scaled on a shared fold so both judges report on one comparable probability scale. In the deployed run, however, the long-context judge was set aside because its post-hoc probability fit did not clear the required correlation floor, so the shipped verifier ran the fast specialist for every prediction and truncated the rare longer hypotheses at that checker's window; this is recorded among the threats. (Schematic.)

Two further refinements keep the grounding honest. For refutation tasks such as fact-checking, a *directional* scorer rewrites the hypothesis so the judge is asked whether the passages support the *truth* of the claim rather than the claim as stated, which is the correct question when the right answer may be "refutes." And the ensemble of judges is combined conservatively: for entailment the minimum across judges is taken, so every judge must agree before the answer is counted as grounded.

9.6 Family four: relevance, and two entity guards

The fourth family contains a single headline signal, the question-answer relevance $q_{\text{a,rel}}$. A cross-encoder scores whether the answer actually addresses the question, turning its logit into a probability. This closes a subtle failure mode: an answer that is fluent, internally consistent, and even grounded in a retrieved passage, but simply off-topic, answering a different question than the one asked. Grounding alone cannot catch this, because the off-topic answer may be perfectly true; only a direct question-to-answer comparison does.

Two further signals were trialed for this family in an earlier revision. Both are still computed on every query, but a later per-benchmark calibration retired them from the composite, so they survive as recorded diagnostics rather than decision signals; the composite stayed lean at nine.

Alias overlap.
Checks whether the named entities in the answer match the entities in the retrieved passages *up to Wikidata aliases*, so that "William Jefferson Clinton" and "Bill Clinton" count as the same entity. It catches grounding failures that the entailment judge misses on surface form alone.

Entity-head consistency.
Reuses the three self-consistency chains and checks whether they name the *same* head entity. It catches the case where the chains read as similar prose, so s_{avg} is high, yet they actually name different people or places. It costs no extra generation, since the chains already exist.

Why they were trialed, and why the composite stayed at nine

Each was introduced to close a specific, observed failure: a correct entity rejected for using a different surface name, and three chains that looked consistent but disagreed on who they were talking about. On the calibration fold, however, their fitted contribution to the composite turned out negligible, so they were retired from the trust score and kept only as cheap diagnostics. This is the panel philosophy meeting its discipline: a specialist earns a vote in the composite only if the calibration data show it adds discrimination, and these two did not, so the deciding panel stays at nine.

9.7 The confabulation early-exit

Stage 5 contains one hard rule that fires before any fusion: the confabulation early-exit. If, at query time, the model's internal confidence is high while the best passage barely supports the answer, the verifier short-circuits, returning a discard verdict and sending the query to regeneration without computing the composite at all.

Why this rule, and why only at query time

The early-exit encodes the single most dangerous combination of signals: high internal confidence paired with near-zero grounding is the textbook signature of confabulation, an answer the model believes and the world does not support. Catching it with a hard rule, rather than letting the calibrated composite weigh it, guarantees that this specific pathology is always escalated rather than occasionally averaged away. The rule fires only at query time, not during the cycle-boundary re-verification of Chapter 12: after the model has been fine-tuned on its own chains, high internal confidence is partly an artefact of having memorised them, so the signal is no longer trustworthy as a confabulation flag, and re-verification relies on its own grounding-based pruning instead.

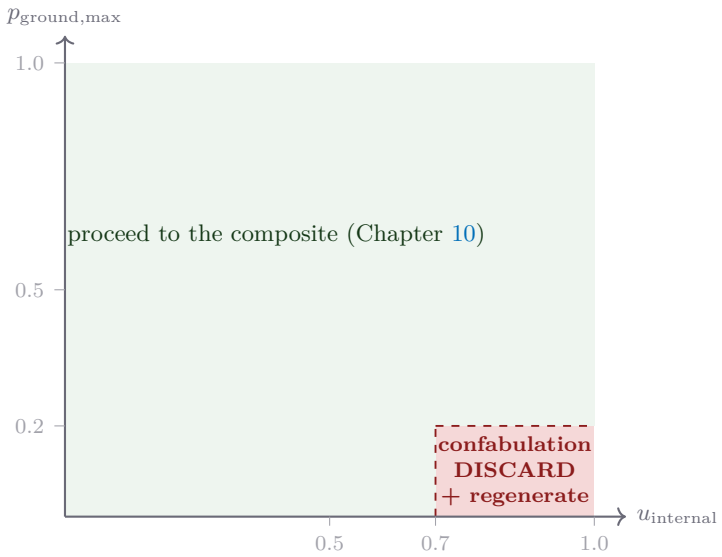


Figure 9.5: The one hard gate inside Stage 5. When internal confidence is high ($u_{\text{internal}} \geq 0.70$) but the best passage barely supports the answer ($p_{\text{ground,max}} \leq 0.20$), the answer is a confident fabrication: the model is sure of something the evidence does not back. The verifier discards it immediately and regenerates, without bothering to compute the rest of the composite. Everywhere else, it proceeds. (Schematic.)

9.8 What the verifier emits

The verifier returns a record carrying all the signals it computes, the nine that feed the composite plus the recorded diagnostics, the canonical answer it scored, the retrieved passages and atomic facts it used, and, after the fusion of Chapter 10, the composite trust score and the gate decision. Every signal defaults to a neutral one-half on failure, so a missing dependency degrades the panel gracefully rather than crashing it or silently scoring zero.

The two readings we have followed since Chapter 2 diverge sharply at this stage, and Figure 9.6 plots their reported signal values side by side.

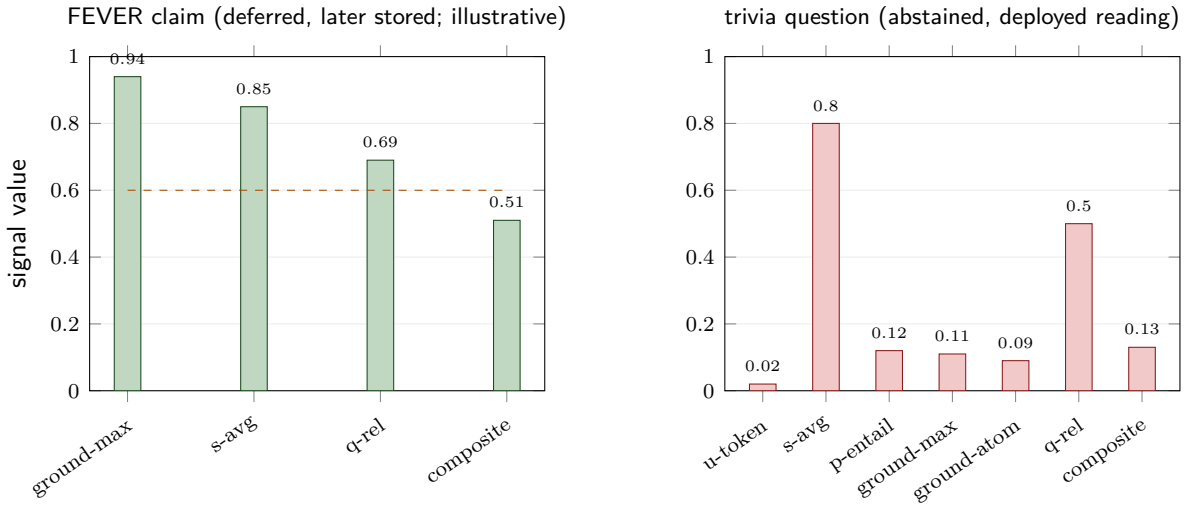


Figure 9.6: Signal readings for the two running episodes. The fact-checking claim (left) is an illustrative hand-authored walkthrough of the defer-then-promote path: it grounds strongly in its best passage and is internally consistent, lifting its composite into the deferral band, from which it is promoted to storage on the next cycle. The trivia question (right) carries the actual deployed cold-start verifier readings for the running episode, and is the chapter's own "agreement is not support" point made visible: the three chains agree with one another ($s_{\text{avg}} = 0.80$) yet do not entail the served answer ($p_{\text{entail}} = 0.12$), and with grounding also near zero the calibrated composite settles at $u_{\text{stored}} = 0.13$, below the deferral cut of 0.45, and with $p_{\text{ground,max}} = 0.11$ also below the grounding floor of 0.20, so the episode is abstained rather than silently discarded. The dashed line marks the storage threshold (Chapter 10).

9.9 Honest limitations

Three caveats keep the verifier in proportion.

The verifier scores the answer, not the reasoning

Every active signal scores the canonical *answer* claim, never the individual steps of the reasoning chain. The verifier therefore cannot distinguish a right answer reached by sound reasoning from a right answer reached by a lucky wrong step, nor can it credit a chain that reasons correctly but states the wrong final answer. The model's logic is judged only insofar as it surfaces in the final answer text. This is a known limitation, logged for future work; the panel is wide across *kinds* of evidence but shallow in the sense that it reads one claim, not a derivation.

Two signals were retired

Two of the original signals no longer drive decisions. A semantic-entropy signal, measuring the diversity of many resampled answers, was retired because its learned weight in the composite shrank to nearly zero, so it added cost without information. A contradiction probability was retired because the default short-hypothesis judge reports only whether a passage *supports* a claim, with no separate contradiction verdict, so the signal was structurally always zero. Both remain in the output record for diagnostics,

but neither affects a decision; CAEM relies on *low grounding* rather than an explicit contradiction veto.

Verifier constants

- Self-consistency chains: $M = 3$, sampled at temperature 0.7, generated once and reused.
- Dropout passes: $K = 5$, dropout active, temperature 0.7.
- Internal confidence: $u_{\text{internal}} = 0.5\, u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$.
- Grounding retrieval: top 20 candidates, reranked to the best 3 passages.
- Atomic decomposition: applied only to answers of at least 12 tokens.
- NLI dispatch: MiniCheck for hypotheses ≤ 408 tokens, Frozen Qwen above.
- Early-exit: $u_{\text{internal}} \geq 0.70$ and $p_{\text{ground,max}} \leq 0.20$ forces a discard at query time.
- Failure default: every signal returns a neutral 0.5 on error.

If an examiner asks: "why so many signals, rather than one strong one?"

Because the literature shows there is no single strong one: single-signal correctness predictors barely beat chance (Chapter 1). The contribution is not a better individual detector but a *panel* spanning four independent failure modes, internal doubt, self-disagreement, ungrounded claims, and off-topic answers, so that a wrong answer must defeat all of them at once. The diversity is the design. And the panel is deliberately conservative at every turn: signals default to neutral on failure, the entailment ensemble takes the minimum so all judges must agree, and a hard early-exit rule guarantees the worst pathology is always escalated rather than averaged away.

The verifier has now produced a vector of calibrated-but-separate signals. Chapter 10 takes up Stage 6: how those signals are each calibrated, fused into a single trustworthy probability, and turned into the four-outcome decision that governs storage.

CHAPTER 10

Stage 6: The Calibrated Composite and Four-Outcome Gate

The verifier of Chapter 9 produced nine numbers. Stage 6 turns those nine numbers into one, the stored trust score u_{stored}, and then turns that one number into an action. This is the chapter where the whole architecture's promise comes due: the number it produces must be an *honest probability*, so that a single fixed threshold can mean the same thing on a fact-check, a trivia question, and a multiple-choice item. We build the number from the ground up, teaching every piece, and then we open the gate that reads it.

Two jobs, in order

Stage 6 has two jobs. The first is *fusion*: take nine signals that live on different scales and point in different directions and combine them into one calibrated probability that the answer is correct. The second is *decision*: compare that probability against fixed cut-points and commit to one of four actions, store, defer, abstain, or discard. The fusion is where the cleverness lives; the decision is deliberately simple, because all the hard calibration work has already been done by the time the gate reads the number.

10.1 Why fusion is hard: scales and sign-flips

The naive way to combine nine signals is a weighted average. It fails here for a concrete reason that is worth seeing before we fix it.

The same signal can mean opposite things on different benchmarks

Consider the question-answer relevance signal $q_{\text{a,rel}}$, which measures how well the answer addresses the question. On an open-ended trivia question, a high relevance is good news: an answer that squarely addresses the question is more likely correct. On a fact-checking claim, where the "answer" is a verdict like *supports* or *refutes*, a very high surface relevance can actually be *bad* news, because the model is echoing the claim's framing rather than judging it. Measured on the calibration data, this signal correlates *negatively* with correctness on the fact-checking benchmark and *positively* on the trivia benchmark. A single fixed weight, positive or negative, must be wrong on one of them. No weighted average can serve both.

The sign-flip, measured

On the cold-start calibration fold, the relevance signal $q_{\text{a,rel}}$ correlates with correctness at a Pearson coefficient of about -0.275 on FEVER, a moderate *negative* relation, and about $+0.068$ on TriviaQA, a *positive* one. The sign itself differs between benchmarks, so the isotonic fit of Section 10.2 learns a *decreasing* curve on FEVER and an *increasing* one on TriviaQA. A single fixed weight, positive or negative, must be wrong on one of them. The flip is not even stable over time: by the third cycle, after the generator had been fine-tuned, the measured FEVER relation had drifted to weakly positive, which is one reason the calibration is re-fitted every cycle (Section 10.5). This single measurement rules out any shared fixed-weight combination and motivates the per-signal, per-benchmark calibration that follows.

The composite solves this in three steps, drawn in Figure 10.1. First, each signal is passed through its own *calibration curve* that converts its raw value into a probability of correctness, learning the right direction automatically. Second, those curves are fitted separately per benchmark and gently blended toward a shared curve to avoid overfitting. Third, the per-signal probabilities are combined into one score through a learned weighting. We take each step in turn.

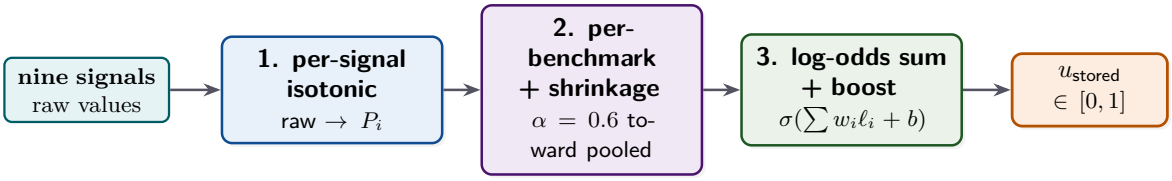


Figure 10.1: The three-step composite. Each raw signal is first mapped to a calibrated probability P_i by its own monotone curve; the curves are fitted per benchmark and shrunk toward a pooled curve; and the per-signal probabilities are combined in log-odds space through a learned weighting and a sigmoid into the single trust score u_{stored}. (Schematic.)

10.2 Step one: turning one raw signal into a calibrated probability

The first step solves a small version of the whole problem: take one signal, say the top-passage entailment, and convert its raw value into a probability that the answer is correct. The tool is *isotonic regression*, and because the rest of the chapter rests on it, we teach it from scratch.

What "calibrated" means here

A raw signal is just a number on some scale; its value of 0.8 has no inherent meaning. We want to replace it with a probability: "answers for which this signal reads 0.8 turn out correct seventy percent of the time, so map $0.8 \mapsto 0.70$." A function that does this faithfully, so that the output really is the empirical fraction correct, is a *calibration curve*. Once every signal has been put through its own calibration curve, all nine speak the same language: probability of correctness.

Isotonic regression and the monotone step function

To learn the curve we use a labelled *calibration fold*: a held-out set of answers for which we know the raw signal value and the ground-truth correctness (one if the answer was right, zero if wrong). Plotting correctness against the signal gives a cloud of zeros and ones. We want a curve through that cloud, and we impose one assumption: the curve must be *monotone*, that is, always non-decreasing (or always non-increasing). The assumption is principled, since a well-designed signal should not reverse direction in the middle of its range, and it is exactly what makes the fit data-efficient.

Isotonic regression, the pool-adjacent-violators idea

Isotonic regression finds the monotone function that best fits the labelled points in the least-squares sense. The classic algorithm is intuitive: sort the points by signal value, walk left to right, and wherever the running fitted values would violate monotonicity (a later average dips below an earlier one), *pool* those adjacent points and replace them with their shared average. Repeated, this produces a *step function*: a piecewise-constant curve that jumps up at a set of breakpoints called *knots* and is flat between them. CAEM stores each signal's curve as its list of knot positions and knot heights, and at run time reads off a value by linear interpolation between the two surrounding knots. The output is clipped to the range [0.02, 0.98] so that no single signal can claim certainty, and a signal needs at least fifty calibration points to earn a curve at all; below that it is dropped and contributes nothing.

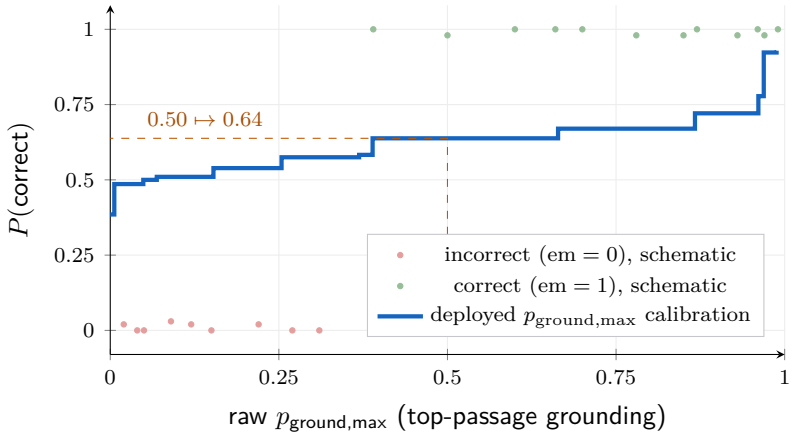


Figure 10.2: The blue step function is the *actual deployed* calibration curve for the top-passage grounding signal $p_{\text{ground,max}}$, read directly from the cycle-three calibration file. Isotonic regression fitted this monotone curve from the labelled calibration fold and CAEM reads it by linear interpolation: a raw grounding of 0.50 maps to a calibrated $P(\text{correct}) \approx 0.64$, and a raw value near 1.0 maps to about 0.93. The curve never reaches the corners because the output is clamped into $[0.02, 0.98]$, so no single signal asserts certainty. The scattered points are schematic, drawn only to show the labelled cloud the curve is fitted through; the per-sample fold values are not stored in the calibration file.

Learning the direction automatically

The sign-flip of Section 10.1 is handled here, for free. Before fitting, the procedure measures the correlation between the raw signal and correctness; if it is positive it fits a non-decreasing curve, and if it is negative it fits a non-increasing one. No hand-coded table of per-signal directions is needed. Figure 10.3 shows the same relevance signal calibrated in opposite directions on the two benchmarks.

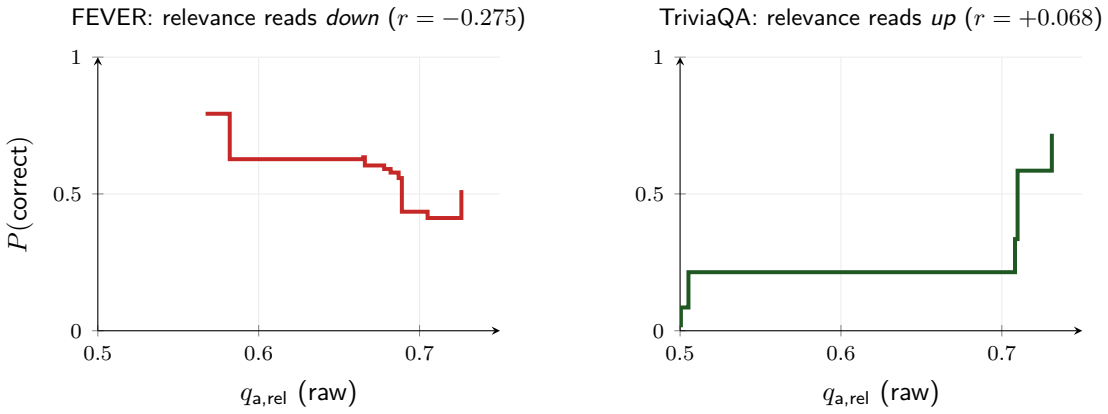


Figure 10.3: The two curves are the *actual deployed* relevance calibrations from the cold-start calibration file, fitted independently per benchmark. On FEVER (left) higher relevance correlates with *lower* correctness (Pearson $r = -0.275$), so isotonic learns a decreasing curve; on TriviaQA (right) it correlates with *higher* correctness ($r = +0.068$), so the curve increases. The relevance score itself lives in a narrow band near 0.5 to 0.73, which is why the axes are zoomed. The auto-detected direction is what lets one signal serve both benchmarks honestly; the jaggedness of the FEVER curve, fitted on only a few hundred points, is exactly what the shrinkage of the next section tames.

Why isotonic, and not the simpler temperature or histogram

Three reasons, each grounded in the report's design rationale. A single-parameter fit like the temperature scaling of Chapter 4 can only soften or sharpen; it cannot bend, and several signals (notably entailment, which saturates near the top of its range) need a genuine bend. A histogram, which just bins the signal and reports each bin's hit rate, wastes the small calibration fold by giving every bin its own independent count; isotonic shares information across neighbouring values through its monotone structure and so needs far less data. And isotonic reads its direction off the data, which is precisely what the sign-flip demands. Temperature scaling is right for the one-dimensional pre-routing confidence; isotonic is right for these nine.

10.3 Step two: per-benchmark curves, shrunk toward a shared one

Fitting one curve per signal across all data would average away the sign-flip. So the composite fits a curve per signal *per benchmark*. But a per-benchmark fold is small, only a few hundred answers, and a flexible curve fitted on little data *overfits*: it chases noise and generalises worse than a coarser fit. The composite controls this with *shrinkage*.

James-Stein shrinkage toward the pooled curve

Two curves are fitted for each signal: a *pooled* curve on all benchmarks together, and a *per-benchmark* curve on each benchmark's slice alone. The deployed curve is a convex blend of the two, taken at every knot,

$$\text{curve}^{b}(x) \;=\; \alpha \cdot \text{curve}^{b}_{\text{local}}(x) \;+\; (1-\alpha)\cdot \text{curve}^{\text{pooled}}(x), \qquad \alpha = 0.6.$$

At $\alpha = 1$ the deployed curve is purely local (maximally adapted, maximally overfit); at $\alpha = 0$ it is the pooled curve (no per-benchmark adaptation). The locked value $\alpha = 0.6$ keeps a moderate per-benchmark adaptation while pulling each local curve most of the way back toward the shared one. This is the James-Stein idea: an estimate from little data is improved by shrinking it toward a pooled prior. The pooled curve also serves as the fallback for any query whose benchmark is unknown at deployment.

Why $\alpha = 0.6$ was chosen

The locked value comes from a cycle-zero diagnostic. Fitting pure per-benchmark curves ($\alpha = 1$) made the weak-signal benchmarks *worse*: their ability to separate correct from incorrect answers, measured by the area under the receiver-operating curve of the stored trust score, fell by roughly five to six points below the pooled fit (from about 0.645 to 0.592 on the misconception probe and from about 0.573 to 0.512 on the multi-step-reasoning probe), because roughly five hundred answers per benchmark is too little for the flexible local curve. Blending at $\alpha = 0.6$ moved those readings back to parity with the pooled fit while preserving a real gain on the strong-signal benchmarks. Shrinkage buys robustness on the benchmarks that need it without surrendering adaptation on the ones that can afford it.

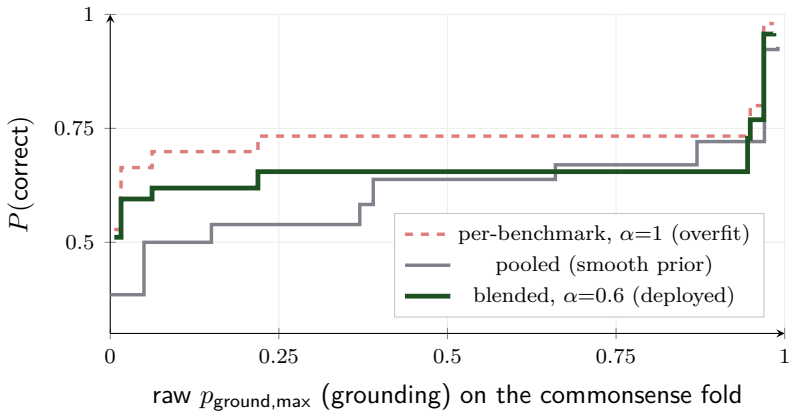


Figure 10.4: All three curves are real, for the grounding signal $p_{\text{ground,max}}$ on the commonsense benchmark. The pure per-benchmark fit (red, dashed, reconstructed from the deployed blend) climbs steeply: on that small fold grounding looks far more informative than it robustly is, since its true correlation there is only about +0.095. The pooled curve (grey), fitted across all benchmarks, rises gently and is well supported. The deployed curve (green) is their $\alpha = 0.6$ blend, sitting between the two and damping the local curve's over-eagerness back toward the pooled prior.

10.4 Step three: combining nine probabilities into one

After steps one and two, each of the nine signals has produced a calibrated probability P_i that the answer is correct. We must fuse the nine into one. Doing this well needs one more idea, the *log-odds*, which we teach now.

Log-odds: the natural currency for combining evidence

A probability P lives in $[0, 1]$, which is awkward for adding evidence: two pieces of strong evidence cannot push past 1. The *odds* $P/(1-P)$ live in $[0, \infty)$, and the *log-odds* $\ell = \log(P/(1-P))$ live on the whole real line, with $\ell = 0$ meaning "even chance" ($P = 0.5$), positive meaning "likely correct," and negative "likely wrong." Log-odds are the right currency because independent pieces of evidence *add* in log-odds space, so accumulating evidence becomes plain addition rather than an awkward combination of bounded fractions. So the composite converts each P_i to its log-odds ℓ_i, adds them up, and converts the total back to a probability with the sigmoid $\sigma(z) = 1/(1+e^{-z})$, the inverse of the log-odds map.

The log-odds sum, with a learned boost

The composite score is

$$u_{\text{stored}} = \sigma\Big(b + \sum_{i=1}^{9} w_i \ell_i\Big), \qquad \ell_i = \log\frac{P_i}{1-P_i},$$

where P_i is the calibrated probability from signal i. With every weight $w_i = 1$ and intercept $b = 0$ this is the plain log-odds sum, which treats the nine signals as independent evidence. CAEM goes one step further and *learns* the weights w_i and intercept b, which is the *boost layer*.

The boost layer is a logistic regression

The boost layer is a logistic regression, and since the term has not yet been taught, here it is in full.

What a logistic regression is, and how it is trained

A logistic regression predicts a probability from a set of input features. It forms a weighted sum of the features plus a constant, $z = b + \sum_i w_i x_i$, and squashes that sum through the sigmoid to get a probability $\sigma(z)$. *Training* it means choosing the weights w_i and the constant b that make those predicted probabilities match the known labels as closely as possible, measured by the *log-loss* (the negative log-likelihood, which rewards confident-correct predictions and punishes confident-wrong ones). The fit is found by standard convex optimisation; there is one global best answer. Here the features x_i are the nine per-signal log-odds ℓ_i, and the labels are the calibration fold's correctness flags, so the boost learns *how much to trust each signal* relative to the others.

Regularisation, and why it is set very strong

Left unconstrained, a logistic regression on correlated features can learn large, unstable weights that overfit the calibration fold. *Regularisation* penalises large weights; its strength is set by a constant C, where a *smaller* C means a *stronger* penalty and weights kept closer to zero. CAEM fits the boost with $C = 0.01$, a deliberately strong penalty. The reason is that the per-signal isotonic curves have already done the heavy calibration; the boost is only a gentle correction for residual correlations among signals (for instance, the two internal-confidence signals partly measure the same thing). Strong regularisation keeps the boost close to the plain log-odds sum, correcting only what the data clearly support and refusing to chase noise on a small fold.

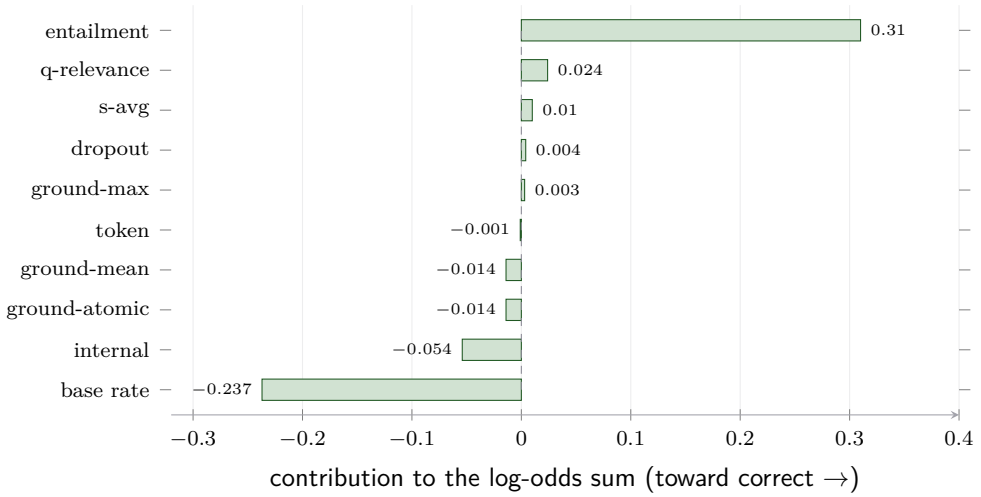


Figure 10.5: The *actual* per-signal log-odds decomposition for a real FEVER claim, "Drake Bell is a model," computed from the deployed cycle-three calibration. The top-passage entailment is the one strong vote that the answer is correct (+0.310), but it is almost entirely cancelled by the FEVER base-rate prior carried in the boost intercept (−0.237) and the low internal-confidence signal (−0.054); grounding lands near neutral because the retrieved passage barely supports the claim. The ten contributions sum to a hair-positive +0.032, and the sigmoid maps that to $u_{\text{stored}} = 0.508$, which the gate reads as a deferral rather than a store.

10.5 How the composite is fitted: the training phase

Everything so far has described how the composite *runs*. But nothing in it is hand-set: every knot height of every isotonic curve, and every weight of the boost layer, is *learned from data*. That learning happens in a phase kept strictly separate from the phase in which the composite is used, and the separation is the key to understanding when and how the composite is trained.

Two phases: fit-time and predict-time

The composite has a *fit-time* and a *predict-time*, and they never overlap. At fit-time the curves and weights are *estimated* from a batch of labelled examples, the way any supervised model is trained. At predict-time, when a real query arrives, those curves and weights are *frozen*: a signal value is simply looked up on its stored curve and the stored weights are simply applied. No fitting happens while answering a query. The composite is trained in batches and applied one query at a time, exactly mirroring the two clocks of the whole system in Chapter 2.

The training data: a labelled calibration fold

The composite is fitted on a *calibration fold*: a held-out set of roughly fifteen hundred answers, each one carrying two things, the nine raw signal values the verifier computed for it, and a *correctness label*, written em, that is one if the answer was actually right and zero if it was wrong (on TruthfulQA the correctness is decided by the language-model judge of Chapter 14 rather than by string match). In machine-learning terms the nine signals are the *inputs* and em is the *target*: the composite learns the mapping from signals to the probability that $\text{em} = 1$. The fold is held out in the strict sense that the generator never trains on it; it exists only to teach the calibration what each signal's values are worth.

Fitting the curves, then the weights

Fitting proceeds in the same three steps the composite later runs, but now estimating rather than applying.

1. **Each isotonic curve is fitted** by taking that signal's (value, em) pairs across the fold and running the pool-adjacent-violators procedure of Section 10.2 to get the monotone step function and its knots. The direction is read from the sign of the signal-to-correctness correlation, and a signal with fewer than fifty usable pairs is dropped. This is done per benchmark and per pooled union, then blended by the shrinkage of Section 10.3.
2. **The boost weights are fitted** by building a table whose rows are the fold's answers and whose columns are that answer's nine per-signal calibrated log-odds (computed by passing each signal through the curve just fitted). A logistic regression is then trained on this table against the em labels, choosing the weights and intercept that minimise the log-loss under the strong regularisation $C = 0.01$. At least fifty complete rows are required, or the boost is skipped and the plain log-odds sum is used.
3. **The result is serialised**: the curves (as knot lists) and the boost weights are written to a calibration file, which is all that predict-time needs.

Listing 10.1: Fitting the composite, condensed from `caem/verification/cal_prob_composite.py`.

```
for sig in signals:                                  # one isotonic curve
    per signal
```

```
    vals, labels = collect(fold, sig)                    # (signal value, em) pairs from the fold
    if len(vals) < 50: continue                          # too little data -> drop the signal
    direction = "increasing" if corr(vals, labels) >= 0 else "decreasing"  # auto-direction
    curve[sig] = isotonic_fit(vals, labels, direction)   # pool-adjacent-violators -> knots

X = [[log_odds(curve[s].predict(row[s])) for s in signals] for row in fold]  # per-signal log-odds
y = [row["em"] for row in fold]                          # correctness labels
boost = LogisticRegression(C=0.01).fit(X, y)             # minimise log-loss -> weights + intercept
save(curve, boost.coef_, boost.intercept_)               # serialise to composite_calibration.json
```

When it is fitted: once at cold-start, then every cycle

The composite is fitted for the first time at *cold-start* (cycle zero), on the cycle-zero calibration fold, and the file it produces is the one the system ships with. It is then *re-fitted at every cycle boundary that commits*, on the same fold *re-scored under the updated model*. Figure 10.6 draws the two lanes.

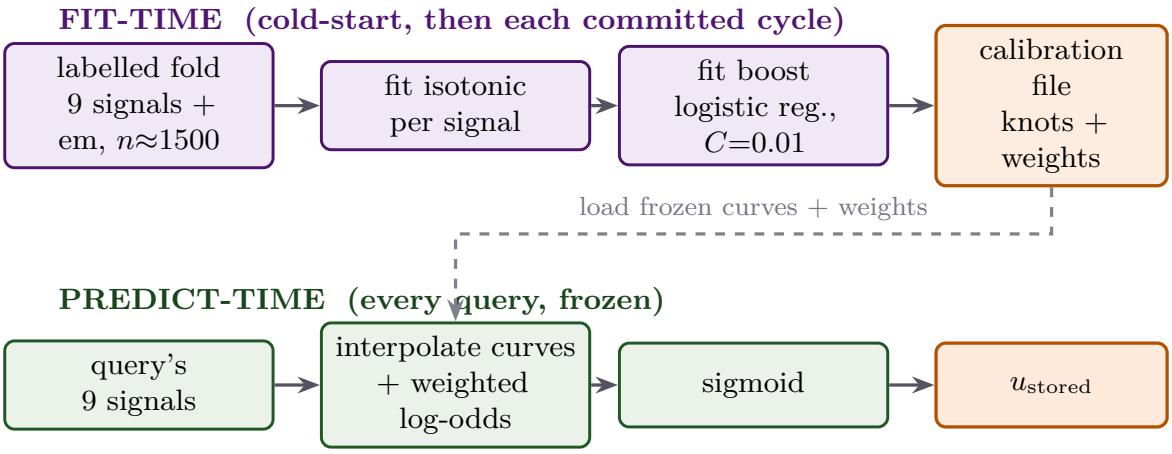


Figure 10.6: The two phases. At fit-time (top) the labelled calibration fold is used to estimate the isotonic curves and then the boost weights, which are saved to a calibration file; this happens once at cold-start and again at every committed cycle boundary. At predict-time (bottom) a real query's nine signals are run through the *frozen* curves and weights to produce u_{stored}. The training never touches a live query, and a live query never re-trains the composite. (Schematic.)

Why the fold is fixed but re-scored each cycle

The calibration fold is the *same* fifteen hundred questions every cycle; it is not resampled. What changes is the *model that answers them*. Because the self-improvement loop of Chapter 12 fine-tunes the generator each cycle, the same question now produces

different signal values and possibly a different correctness label, so the fold is *re-scored* under the updated model before the composite is re-fitted on it. Re-fitting is necessary precisely because the model drifts: if the curves stayed frozen while the signal distributions shifted underneath them, a stored score of 0.60 would slowly stop meaning a sixty-percent chance of correctness, and the fixed gate thresholds would lose their meaning. Holding the fold fixed while re-scoring it ensures that any change in the fitted calibration reflects the model's movement, not noise from a different sample. The composite and the pre-routing temperature are, in fact, the only parts of the architecture re-fitted each cycle; everything else in the configuration is locked.

The composite is supervised; the signals it calibrates are not its labels

A common confusion is to think the verifier's signals *are* the training labels. They are not. The signals are the *inputs*; the single training label per answer is its ground-truth correctness em, obtained by scoring the fold's answers against gold. The composite is an ordinary supervised model learning "given these nine signal readings, how often was the answer actually right?" That is why a *labelled* fold is required, and why the quality of u_{stored} ultimately rests on having honest correctness labels on the calibration fold.

10.6 The whole computation, end to end

A borderline FEVER claim becomes $u_{\text{stored}} = 0.508$

Take a real FEVER claim the deployed system actually scored, "Drake Bell is a model." Its nine raw signals enter step one and each is mapped through its own FEVER-calibrated, shrunk curve to a probability of correctness: the top-passage entailment reads moderately high (the retrieved passage does engage the claim), the relevance maps just above one half, the grounding signals land almost exactly at one half because the passage barely supports the specific assertion, and the internal-confidence signal maps below one half because the model was unsure. Step three converts each to log-odds and forms the boosted weighted sum, drawn in Figure 10.5: the single strong upward vote from entailment is almost entirely cancelled by the FEVER base-rate prior carried in the intercept and by the downward internal-confidence vote, leaving a hair-positive total of +0.032. The sigmoid turns that into $u_{\text{stored}} = 0.508$. The number is honest: it says this answer is just above an even chance of being correct, which is exactly the regime where the system should hesitate rather than commit. The gate reads it as a deferral, and the claim is parked for reconsideration at the next cycle boundary.

Listing 10.2: The composite score, condensed from `caem/verification/cal_prob_composite.py`.

```
log_odds = boost_intercept                    # b (0 when boost not
    fitted)
```

```
for sig, curve in calibrations.items():          # the nine per-signal curves
    p = curve.predict(signal_values[sig])         # raw value -> calibrated P_i (isotonic)
    p = clip(p, 0.02, 0.98)                       # bound any single signal
    ell = log(p / (1 - p))                        # P_i -> log-odds
    log_odds += boost_weights.get(sig, 1.0) * ell  # weighted (boost) sum
u_stored = 1.0 / (1.0 + exp(-log_odds))           # sigmoid back to [0, 1]
```

Per-benchmark dispatch, and the cold-start fallback

At run time the composite is handed the query's benchmark tag. If a per-benchmark calibration exists for that benchmark, the score is computed through its curves and weights; otherwise the pooled fallback is used. On the very first cycle, before any calibration fold has been scored, no curves exist yet, so the verifier bootstraps with a simple fixed weighted sum and switches to the calibrated composite as soon as the cycle-zero fold is fitted. Thereafter the curves are refit at every cycle boundary (Chapter 12), which is what keeps u_{stored} meaning the same thing as the model underneath it changes.

10.7 The four-outcome gate

Now the second job. The gate reads u_{stored} and commits to one of four actions. It is deliberately a short cascade of fixed thresholds, because the number it reads is already a calibrated probability.

The decision tree

After the early-exit, the gate is four lines, read top to bottom, first match wins:

- $u_{\text{stored}} \geq 0.60 \Rightarrow$ **store** the answer as a verified episode.
- $0.45 \leq u_{\text{stored}} < 0.60 \Rightarrow$ **defer**: hold it in a buffer for reconsideration at the next cycle boundary.
- $u_{\text{stored}} < 0.45$ and $p_{\text{ground,max}} < 0.20 \Rightarrow$ **abstain**: the answer is both low-confidence and ungrounded, so return an honest refusal.
- otherwise $\Rightarrow$ **discard**: drop the answer silently.

The storage and deferral cut-points are *precision targets*, read as "store only when the calibrated chance of correctness is at least sixty percent." Because u_{stored} is a calibrated probability, that sentence means the same thing on every benchmark.

Listing 10.3: The gate, condensed from `caem/verification/verifier.py`.

```
if u_stored >= 0.60:          # store_threshold
    return "STORE"
if u_stored >= 0.45:          # defer_threshold
    return "DEFERRED"
```

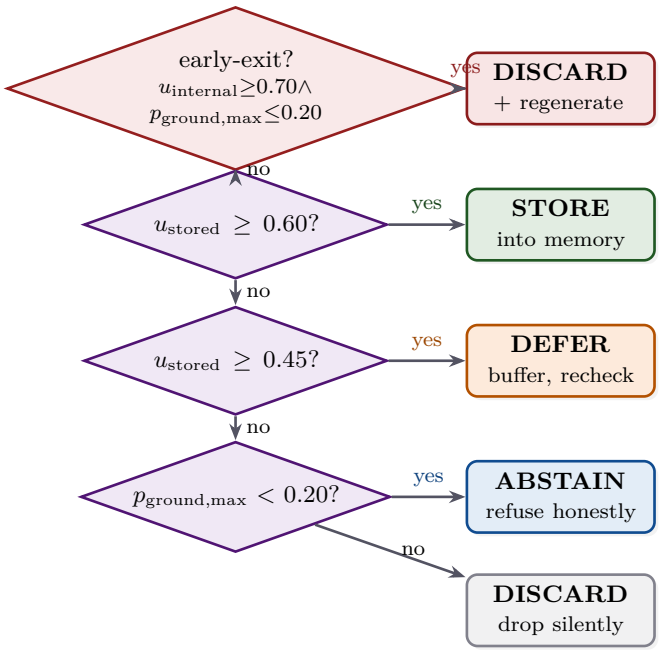


Figure 10.7: The decision cascade. The confabulation early-exit of Chapter 9 is checked first and forces a discard-and-regenerate when the model is internally sure but ungrounded. Otherwise the calibrated score is compared against the storage cut-point 0.60 and the deferral cut-point 0.45; below deferral, an answer with essentially no grounding is turned into an honest abstention, and anything else is discarded silently. (Schematic.)

```
if p_ground_max < 0.20:          # abstain_pground_ceiling
    return "ABSTAIN"             # low confidence AND ungrounded -> honest
        refusal
return "DISCARD"
```

Why four outcomes and not two

A plain accept/reject gate would throw away two distinctions the architecture depends on. The first is *defer*: an answer just below the storage line is promising but not yet trusted, so rather than discard it the system parks it and lets a later, better model re-judge it. This is exactly how the FEVER claim at $u_{\text{stored}} = 0.508$ is later promoted to memory (Chapter 12). The second is the split between *abstain* and *discard* among the low-confidence answers. If the answer is also ungrounded (its best supporting passage barely entails it, $p_{\text{ground,max}} < 0.20$), the honest move is to tell the user "I do not know" rather than emit a guess; that is abstain. If instead there is some grounding but the composite still rejects it, the answer is dropped silently as a discard. The four outcomes are the difference between a system that merely filters and one that knows the difference between "wrong" and "unknown."

10.8 Why one fixed threshold governs five benchmarks

The deepest payoff of the chapter is also the simplest to state. The gate uses one pair of cut-points, 0.60 and 0.45, for every benchmark. That is only sound because all the

benchmark-specific work happened upstream: each signal was calibrated per benchmark, and the composite produced a probability whose meaning is benchmark independent. A storage score of 0.60 certifies the same sixty-percent calibrated chance of correctness whether the query was a fact-check or a trivia question. Push the calibration into the signals and the threshold can be a constant; skip the calibration and the threshold would have to be retuned per benchmark, which is exactly the brittleness the composite exists to remove.

If an examiner asks: "wasn't there a conformal-prediction gate here?"

An earlier design used a split-conformal gate, which offers a formal coverage guarantee by calibrating a cut-point on a held-out fold. It was removed. The guarantee relies on the calibration fold and the evaluation stream being *exchangeable*, a technical condition meaning their order carries no information, and a cycle-zero diagnostic found that condition violated in this setting: the self-improvement loop shifts the answer distribution between the fold and the stream, so the coverage guarantee no longer held. Rather than ship a guarantee that does not hold, the design reverted to the fixed-threshold gate above, whose meaning is maintained instead by re-fitting the calibration every cycle. Reporting the removal is the honest choice; a guarantee that fails its premise is worse than an explicit threshold that does not claim one.

Composite and gate constants

- Composite signals: nine (the verifier's other computed signals do not enter).
- Per-signal calibration: isotonic, output clipped to $[0.02, 0.98]$, minimum 50 samples.
- Shrinkage toward pooled curve: $\alpha = 0.6$.
- Boost: logistic regression on per-signal log-odds, regularisation $C = 0.01$.
- Combination: $u_{\text{stored}} = \sigma(b + \sum_i w_i \ell_i)$.
- Gate: store ≥ 0.60, defer ≥ 0.45, abstain if below 0.45 and $p_{\text{ground,max}} < 0.20$, else discard.
- Early-exit (first): $u_{\text{internal}} \geq 0.70$ and $p_{\text{ground,max}} \leq 0.20$ forces discard-and-regenerate.

The composite has now turned nine signals into one honest probability, and the gate has turned that probability into one of four actions. Chapter 11 takes up Stage 7, the output contract: how each of the four decisions is rendered to the user, including how an abstention becomes a calibrated refusal and how the served answer carries its confidence with it.

CHAPTER 11

Stage 7: The Output Contract

Six stages have decided what to answer and how much to trust it. Stage 7 is the last one in the per-query pipeline, and it is the only one the user ever sees. Its job is narrow but important: take the four-outcome decision of Chapter 10 and turn it into what is actually shown, and crucially, ensure the answer never travels alone. This is the stage where the calibration mismatch of Chapter 1, the confident lie wearing the same clothes as the confident truth, is finally repaired at the surface the user touches.

The decision becomes a presentation

By the time a query reaches Stage 7, the hard thinking is done: there is a generated string and a verdict (store, defer, abstain, or discard). Stage 7 does not re-judge anything. It decides what string to display, whether to show it at all, and what reliability signal to attach. The guiding principle is that a user should never receive a fluent answer with no indication of how much to trust it; the answer and its confidence are delivered together, as one object.

11.1 Two answers, kept separate

The pipeline carries the answer in *two* fields, and the separation is deliberate. One field, the *raw answer*, is the model's literal output, kept verbatim. The other, the *display answer*, is the curated string a user would see. They are computed from the same generation but serve opposite masters.

Why scoring and display must not share a string

Evaluation needs the model's *literal* prediction: to score exact match honestly, the harness must see exactly what the model produced, including a wrong answer, so that a wrong answer counts as wrong. The user, by contrast, should see a *curated* string: the final answer with its reasoning scaffold stripped away, or an honest refusal, or nothing at all. If display and scoring shared one field, then either the user would see raw scaffold text, or the scorer would be fed a cleaned-up string that no longer matches what the model actually said. Keeping the raw answer for the scorer and the display answer for the user lets each be exactly what it should be.

11.2 Mapping the decision to a display string

The display answer is computed by one small function that reads the verdict and nothing else. Its logic is the whole contract.

Listing 11.1: The display mapping, condensed from `caem/pipeline.py`.

```
if verifier_output is None:                # Tier 1 hit, or a pipeline edge case
    return strip_scaffold(answer)          # serve the (stored) answer as-is
if decision == "ABSTAIN":
    return "I do not know."                # the calibrated refusal
if decision == "DISCARD":
    return ""                              # suppress: the prediction is not safe to show
return strip_scaffold(answer)              # STORE / DEFERRED: show the final answer
```

So the four verdicts map to three surface behaviours. A **stored** or **deferred** answer is shown, because the system trusts it enough to serve (and, for a deferred answer, also holds it for reconsideration). An **abstained** query returns the fixed refusal string "I do not know.", the principled failure mode in a factual setting where a confident wrong answer costs more than an honest refusal. A **discarded** answer is suppressed entirely, because the verdict was that the prediction is not safe to expose. A Tier 1 memory hit, which skipped the verifier by design, serves its stored answer directly.

Scaffold stripping keeps display and scoring aligned

The generated string is the scaffolded form "Reasoning: ... Answer: Y" from Chapter 8. The display function strips this down to just the final answer Y, and it does so through the *same* extraction routine the exact-match scorer uses. This shared routine is not an accident: it guarantees that the answer the user sees and the answer the scorer grades are extracted identically, so a reported score always corresponds to what was actually shown. If the string carries no scaffold markers (a direct answer, or a legacy format), it is passed through unchanged.

11.3 The answer never travels alone: the confidence envelope

A bare string, even the right one, would reproduce the original problem: the user could not tell a trusted answer from a guess. So a deployment frontend wraps the display answer in a *confidence envelope* built from the same verdict and the calibrated score u_{stored}. The envelope carries six things alongside the answer.

A continuous score.
The calibrated u_{stored} rendered as a percentage, for example "72%", read directly as the chance the answer is correct.

A semantic tag.
One of *verified*, *provisional*, *conflicting*, or *insufficient*, naming *which* situation produced the verdict rather than just how confident it was.

An ordinal label.
One of *high*, *moderate*, *low*, or *very low*, an at-a-glance reliability hint.

An icon.
A check, a tilde, a warning, or a cross, for instant visual reading.

A show-or-hide flag.
Whether to display the answer at all, hidden on an abstention.

A caveat.
An optional hedge sentence appropriate to the verdict.

Table 11.1: How each verdict is rendered. The four verdicts map to four distinct user-facing presentations, not to four points on a single confidence scale.

Verdict	Tag	Label	Icon	Shown to the user
STORE	verified	high	check	the answer, with its confidence percentage
DEFERRED	provisional	moderate	tilde	the answer, with a "re-checking later" caveat
DISCARD	conflicting	low	warning	a hedge that the evidence found disagrees
ABSTAIN	insufficient	very low	cross	"I do not know." (the answer is hidden)

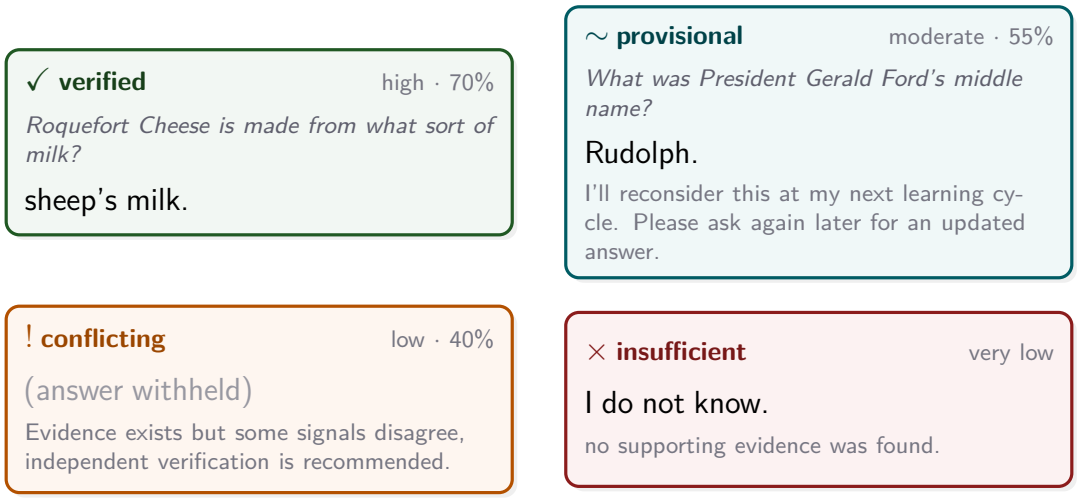


Figure 11.1: The four verdicts rendered as user-facing cards. The *verified* and *provisional* cards reproduce real deployed cycle-three records; the *conflicting* and *insufficient* cards are representative of their verdict. A *verified* answer carries its confidence; a *provisional* answer is shown with a promise to re-check it; a *conflicting* verdict withholds the answer and explains that the evidence disagreed; an *insufficient* verdict returns an honest refusal. The answer and its reliability signal always arrive together.

11.4 Conflicting is not the same as insufficient

The most important subtlety in the envelope is that the two low-confidence verdicts, discard and abstain, are *different failure modes*, not two points on one confidence scale. This was decided back at the gate (Chapter 10) and it surfaces here.

The split is by grounding, not by score

Recall the gate: among answers below the deferral cut-point, the split between abstain and discard is made by the grounding signal $p_{\text{ground,max}}$, not by u_{stored}. An **abstain** (tag *insufficient*) means there was essentially no supporting evidence at all, so the honest move is to say nothing. A **discard** (tag *conflicting*) means evidence existed but the signals disagreed about it, so the answer is withheld with a different explanation. A consequence worth stating plainly: a discarded answer can carry a *higher* u_{stored} than an abstained one. In the deployed cycle-three run this is not a corner case but the norm: the discard and abstain calibrated-confidence distributions overlapped almost entirely, with the chance that a randomly chosen discarded answer scored higher than a randomly chosen abstained one close to one half (near 0.46). The two verdicts therefore genuinely cannot be ordered on a single confidence axis; the split is by grounding, not by score. The two tags are not "a bit unsure" versus "very unsure"; they are "the evidence conflicts" versus "there is no evidence." Naming the failure mode, rather than only its severity, is what lets the user respond appropriately, by rephrasing in one case and by trusting the refusal in the other.

11.5 Deferral speaks in the system's own clock

The deferred verdict has a caveat unlike the others, because deferral is a promise about the future. A provisional answer is shown with a note that the system will reconsider it, and the phrasing reflects how CAEM actually updates: its cycle boundary is triggered by *query volume*, not by the wall clock, so the default caveat is the cadence-neutral "I'll reconsider this at my next learning cycle." A deployment that updates on a known schedule can override this with a concrete cadence, nightly or hourly or continuously, to give the user a real expectation. The deferral message is the only place the user-facing surface acknowledges that the system learns over time, which is the subject of the next chapter.

11.6 Where the calibration mismatch is repaired

Chapter 1 opened with the core hazard: a model states a falsehood with the same confidence it states a truth, and a reader has no way to tell them apart. Stage 7 is the surface where that hazard is closed. Every served answer arrives with a calibrated probability and a named reliability tag; a low-confidence answer is either hedged, withheld, or replaced by an honest refusal; and the confidence number means the same thing on every benchmark because of the calibration of Chapter 10. The confident lie no longer wears the same clothes as the confident truth, because the clothes now include an honest label of how much to trust what was said.

If an examiner asks: "why four outcomes rather than answer-or-refuse?"

Because two of the four outcomes carry information a binary surface would discard. *Defer* separates the genuinely uncertain answer, which a later cycle's better model may resolve, from the confidently wrong one; collapsing both into "refuse" would throw

away a recoverable answer. And the *conflicting*-versus-*insufficient* split separates "the evidence disagrees" from "there is no evidence," two situations that call for different user responses. The four-outcome contract is what lets the system distinguish *wrong*, *unknown*, and *not yet decided*, rather than flattening all three into a single refusal.

The output contract

- Two answer fields: raw (verbatim, for scoring) and display (curated, for the user).
- Display map: store/defer → stripped answer; abstain → "I do not know."; discard → withheld.
- Scaffold stripping shares the exact-match scorer's extraction routine.
- Envelope: calibrated percentage, semantic tag (verified / provisional / conflicting / insufficient), ordinal label, icon, show-or-hide flag, caveat.
- Discard versus abstain is split by grounding ($p_{\text{ground,max}}$), not by u_{stored}.

The per-query pipeline is now complete: a query has been triaged, routed, answered, verified, scored, and presented with its confidence. Everything so far has happened without the model learning anything. Chapter 12 opens Part III and the second clock, Stage 8, the cycle-boundary self-improvement loop, where the verified answers this pipeline produced are folded back into the model and the whole system gets better.

PART III

The Self-Improvement Loop

CHAPTER 12

Stage 8: The Cycle-Boundary Self-Improvement Loop

Everything in Part II happened without the model learning anything. A query was triaged, routed, answered, verified, scored, and presented, and through all of it the weights, the calibration, and the memory were only *read*. This chapter is the other clock: the slow, periodic loop that runs at *cycle boundaries* and is the only place the model actually changes. It is CAEM's answer to the third structural flaw of Chapter 1, staticness, and it is where the verified answers the pipeline produced are folded back into the model so that next cycle's pipeline is better than this one's.

The system grades its own homework, then studies it, carefully

The per-query pipeline does the system's homework all day and files the answers it trusts into memory. At a cycle boundary the system stops taking new questions, opens that file of verified answers, and *studies them*: it fine-tunes itself on its own best work. But studying can go wrong, so before keeping the lesson it checks that it has not forgotten anything it used to know. If it has, it tears up the lesson and keeps yesterday's self, while keeping the new file of answers. One clock answers; the other clock learns, under a guard that refuses to let learning become forgetting.

12.1 Two clocks, in full

Chapter 2 introduced the two clocks; now we run the slow one in detail. The fast clock is the per-query pipeline, ticking thousands of times. The slow clock is the cycle: one tick processes a whole batch of queries through the fast clock and then runs a single training-and-maintenance block. The realised run had six ticks of the slow clock, cycles zero through five.

Why training is periodic, not per query

Training on every query would be both ruinously expensive and dangerous: a gradient step per answer would let one bad answer move the weights immediately, with no chance to verify it first. Training never, and the system is static. CAEM takes the middle path: it *accumulates* verified answers during the fast clock and trains on them in a *batch* at the cycle boundary, after they have all passed the gate. This is what lets the loop train only on answers it has had a chance to trust, and what lets a single guard check the whole update before committing it.

12.2 What one cycle does, in order

A cycle boundary is not one action but an ordered block, and the order is load-bearing. Figure 12.1 lays out the full sequence; the sections that follow open each part.

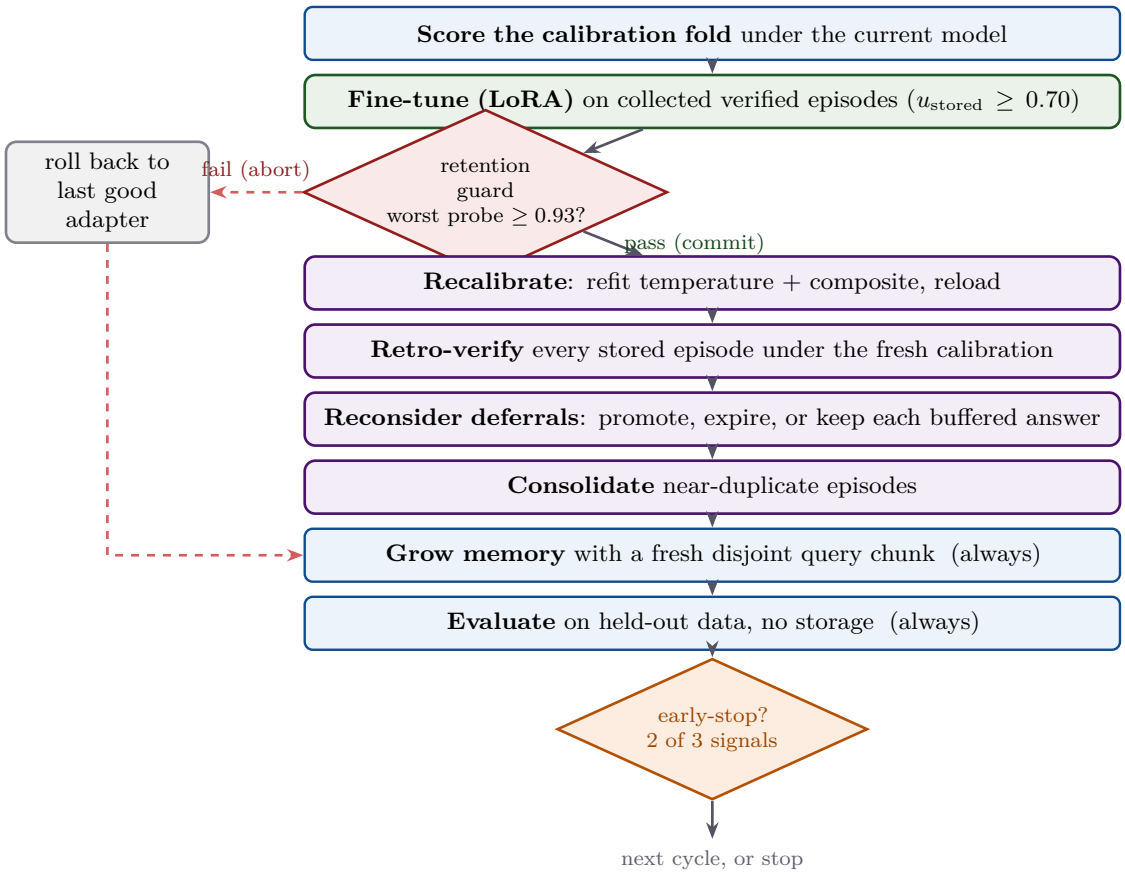


Figure 12.1: The cycle-boundary block. The model is fine-tuned on the verified episodes, then the retention guard decides commit or abort. On a commit (green) the four commit-gated steps run in order (purple): the calibration is refit, the memory re-verified, the deferred buffer reconsidered, and near-duplicate episodes consolidated; on an abort (red) the adapter rolls back to the last good state and all four commit-gated steps are skipped. Growing the memory with fresh data and evaluating on held-out data run regardless (blue). Finally the early-stop gate decides whether to begin another cycle. (Schematic.)

12.3 What it trains on: the verified episodes

The training data is the system's own memory, filtered. Only episodes whose stored trust score u_{stored} is at least 0.70 are collected, a threshold deliberately *higher* than the 0.60 storage cut of Chapter 10: storage admits an answer to memory, but training demands a stricter, cleaner subset. Of those, only episodes from the three *training* benchmarks are eligible; the two transfer benchmarks are held out so that generalisation is measured on data the model never trained on.

The training target is the reasoning, not just the answer

For each eligible episode the training target is its *reasoning chain*, not merely its final answer. The prompt is the question in the scaffolded chat format of Chapter 8, and the target the model learns to produce is the verified "Reasoning: ... Answer: Y" completion, with the prompt tokens masked out of the loss so only the completion is learned. This is *distillation*: the model is taught to reproduce, from its own parameters, the grounded reasoning it previously needed retrieval to produce. Chains that are empty, too short, or detected as repetitive loops fall back to the bare answer or are filtered out, so the loop never trains on degenerate text.

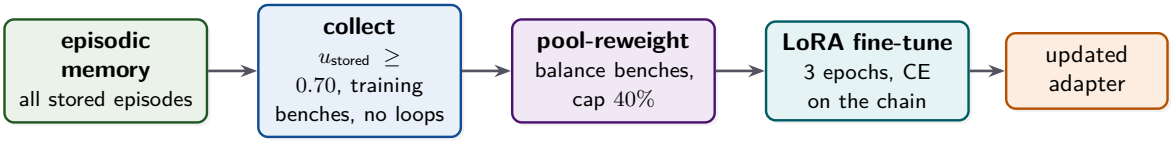


Figure 12.2: Where the training set comes from. The memory is filtered to high-trust, training-benchmark, loop-free episodes; the survivors are re-weighted to balance the benchmarks; and the model is fine-tuned to reproduce their verified reasoning chains. The output is an updated low-rank adapter. (Schematic.)

Pool reweighting: keeping one benchmark from eating the pool

Left alone, one benchmark can dominate the training pool. In an earlier version of the system, fact-checking episodes came to make up the entire pool and the model collapsed on the other benchmarks. Pool reweighting prevents this with four mechanisms working together.

The four reweighting mechanisms

- **Temperature-mixed proportions** ($T = 2.0$): the per-benchmark shares are softened, turning a tenfold gap in raw counts into roughly a threefold gap in training weight.
- **Upsample cap** ($3\times$): a scarce benchmark's samples are replicated at most three times, so balance never comes from runaway duplication.
- **A floor** (50 samples): every benchmark gets at least this many, even if smoothing would round it to nothing.
- **A hard ceiling** (40%): no single benchmark may exceed forty percent of the final pool, with the excess water-filled to the under-represented ones.

A cold-start fallback seeds any benchmark with no verified episodes from a small set of gold-labelled examples, so the loop can train even on the first cycles.

12.4 The fine-tune: a small adapter, a frozen model

The actual training touches only the low-rank adapter of Section 3.6. The backbone stays frozen; only the rank-thirty-two adapter on the seven projections, about two percent of the parameters, receives gradients. The model is trained to reproduce the collected chains by cross-entropy on the completion tokens.

Listing 12.1: The fine-tune, condensed from `caem/training/self_improvement.py`.

```
loader = DataLoader(QADataset(train_pairs), batch_size=4, shuffle=True)
    # micro-batch 4
optimizer = AdamW(adapter_parameters, lr=2e-4)
    # LoRA learning rate
for epoch in range(3):
    # epochs_per_cycle
    for step, batch in enumerate(loader):
        loss = cross_entropy(model(batch).logits, batch.labels)
            # prompt masked -100
        # LoRA path: no L2 anchor -- the frozen base bounds drift on its
            own
        loss.backward()
        if (step + 1) % 4 == 0:
            # grad_accum -> eff. batch 16
             clip_grad_norm(adapter_parameters, 1.0); optimizer.step();
                optimizer.zero_grad()
```

Why there is no anchor term on the LoRA path

A full fine-tune of every weight would risk drifting far from the starting model, so the classic remedy adds an *anchor* penalty to the loss, $(\lambda/2)\|\theta - \theta_{\text{prev}}\|^2$, pulling the new weights toward the previous cycle's. On the deployed low-rank path this term is *switched off*, and deliberately so: the backbone is frozen and the adapter is tiny, so the model simply *cannot* drift far, the bounded adapter is the anchor. The explicit penalty survives only in the full-fine-tune fallback (with $\lambda = 0.01$), kept for ablation. Choosing a small adapter over a penalised full update is the cleaner way to bound forgetting, and it is why the loss in the listing is plain cross-entropy.

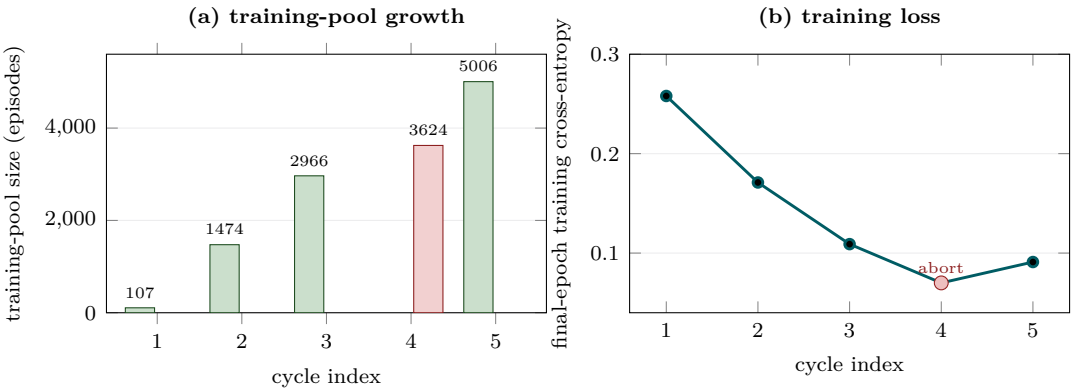


Figure 12.3: The training trajectory of the actual deployed run. Panel (a): the training-pool size grows monotonically across the committed cycles (107, 1474, 2966, 5006 at cycles one, two, three, and five), with the aborted cycle four (red) drawing on 3624 episodes. Panel (b): the final-epoch training cross-entropy falls steadily (0.258, 0.171, 0.109) and reaches its lowest value of the entire run, 0.070, at cycle four (red), before rising to 0.091 at cycle five. Cycle four thus reached the lowest training loss of the run yet breached the retention floor, the textbook signature of an over-fit update that the guard correctly caught and rolled back.

12.5 The retention guard: commit or abort

A fine-tune can improve the target benchmarks while quietly eroding unrelated abilities, which is the catastrophic forgetting of Chapter 1. The retention guard is the check that catches this before the update is kept.

Pristine probes, worst-case ratio, and the floor

At cycle zero, before any training, the system measures its accuracy on three *retention probes*, held-out tasks chosen to span different abilities: a four-option general-knowledge test, an open-text factoid test, and a five-option commonsense test, two hundred questions each. These cycle-zero scores are the *pristine baseline*, anchored once and reused forever as the denominator (it is even persisted to disk so a resumed run does not re-anchor against an already-trained model). After each fine-tune the three probes are re-run, and each post-training score is divided by its pristine value to get a retention ratio. If the *worst* of the three ratios falls below the floor $\rho_{\min} = 0.93$, the cycle *aborts*. The rule is a logical AND over the probes: all three must hold above the floor for the cycle to commit, so a collapse on any single ability vetoes the update.

Asymmetric rollback: weights revert, memory persists

On an abort the adapter is rolled back to the previous cycle's snapshot, and the commit-gated steps below are skipped. But the *episodic memory is not rolled back*: every episode admitted during the aborted cycle survives. This asymmetry is the core safety property of the whole architecture. Memory growth is monotone and high-precision by construction, since each episode passed the gate, so it is always safe to keep; a weight update is a global change with no such guarantee, so it is the thing

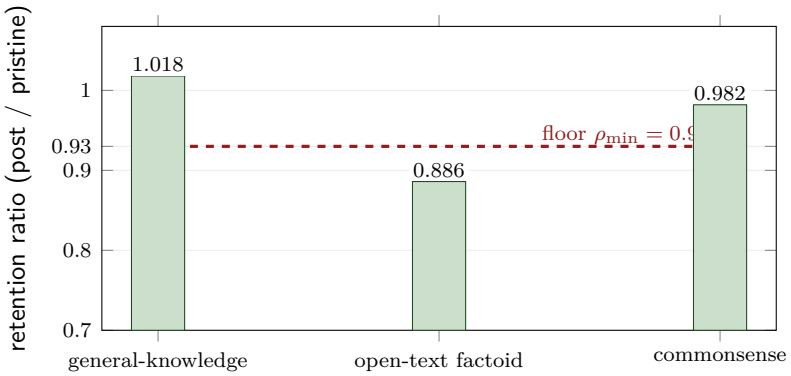


Figure 12.4: The three retention probes measured at the cycle-four boundary, read from the run log. General knowledge actually improved slightly (ratio 1.018) and commonsense held at 0.982, both above the floor, but the open-text factoid probe fell to 0.886, below the 0.93 floor. Because the worst probe breaches the floor, the cycle aborts: the freshly trained adapter is discarded and the previous cycle's adapter is restored. The memory accumulated this cycle is kept.

thrown away. The realised run hit this fork for real at cycle four: the open-text factoid probe fell to 0.886, below the floor, the adapter reverted to its cycle-three state, and the memory carried forward untouched. The next cycle then re-attempted training from the last good adapter against fresh data.

12.6 The rest of the boundary, and what is gated on committing

If the cycle commits, four more things happen, in order, and each depends on the model having actually improved.

Recalibrate.
The calibration fold is re-scored under the new model, then the temperature of Chapter 4 and the composite of Chapter 10 are re-fitted and reloaded. This is the per-cycle refit of Section 10.5: it keeps u_{stored} honest as the model moves underneath it.

Re-verify the memory.
Every stored episode is re-scored through the freshly recalibrated verifier. An episode whose trust has risen above the storage threshold is promoted, and one that has fallen below a floor is pruned. In the deployed run a typical committed cycle re-scored several thousand stored episodes and pruned only a small single-digit-percentage tail; at the third cycle, for instance, all 5358 stored episodes were re-scored, 352 were pruned, and 5006 survived. This is how the deferred fact-checking claim of $u_{\text{stored}} = 0.508$ from Chapter 2 is finally promoted to memory: the better model scores it higher, and it crosses the line. Re-verification runs *after* recalibration, deliberately, so it scores through the new curves rather than stale ones. The aborted cycle skipped re-verification entirely, because re-scoring through unchanged weights would be a no-op.

Reconsider deferrals.
The deferred buffer of borderline answers is re-examined and each entry is promoted, expired, or kept.

Consolidate.
Near-duplicate episodes are merged into a single representative.

An abort skips the commit-gated half, but not the whole cycle

All four of the above, plus the re-verification, are gated on the cycle committing; an abort skips them, because re-scoring and re-verifying through an unchanged model would add nothing. But the cycle still does its *unconditional* work even on an abort: it grows the memory with a fresh, content-hash-disjoint chunk of new questions (with storage on), runs the held-out evaluation (with storage off), checkpoints, and consults the early-stop gate. An aborted cycle loses its weight update, not its progress.

12.7 When the loop stops

The loop is configured for ten cycles, but it does not always use them. After a burn-in, an early-stop gate watches three signals: whether the joint quality score has stopped rising, whether the storage rate has saturated, and whether general-knowledge retention is holding. When two of the three agree that the system has reached equilibrium, the loop stops early. In the deployed run the two signals that fired were the saturated storage rate and the holding general-knowledge retention, while the joint-quality saturation signal had not yet tripped; two of the three was enough to stop at the first eligible cycle after the burn-in. The realised run stopped this way at cycle five, having committed cycles one, two, three, and five and aborted cycle four, a commit fraction of four in five. The full realised trajectory is the subject of Chapter 15.

12.8 Why this closes the static flaw

Verification that compounds instead of evaporating

Chapter 1 argued that the deepest waste in a static model is that verification does not compound: effort spent confirming an answer today changes nothing tomorrow. The cycle loop is the mechanism that makes it compound. Every verified answer is filed in memory; every cycle the model studies that file and folds the knowledge into its parameters; and the retention guard ensures each such step is a net gain rather than a trade. The receipt that this is distillation and not mere caching is that, after the loop runs, the model answers from its own parameters, on the zero-shot tier, questions it previously could only answer with retrieval, and it does so at a higher accuracy than the retrieval tier reached at cold-start: in the deployed run the learned zero-shot tier reaches an exact match of about 0.57 on its assigned queries by the third cycle, above the roughly 0.48 the retrieval tier managed on its assigned queries at the cold start. The loop turns one-shot verification into a permanent capability.

Cycle-loop constants

- Training set: episodes with $u_{\text{stored}} \geq 0.70$, training benchmarks only, loops filtered.
- Pool reweighting: temperature 2.0, upsample cap 3×, floor 50, ceiling 40%.
- Fine-tune: LoRA rank 32, $\alpha = 64$, 3 epochs, micro-batch 4, effective batch 16, learning rate 2×10^{-4}, AdamW, no anchor term.
- Retention guard: three probes (200 each), worst-ratio floor $\rho_{\min} = 0.93$; abort restores the adapter, memory persists.
- Boundary: recalibrate, re-verify, reconsider, consolidate (all commit-gated); grow and evaluate (unconditional).
- Horizon: up to 10 cycles; early-stop on two of three equilibrium signals (realised stop at cycle five).

If an examiner asks: "what shows the loop helps rather than just drifts?"

Three guards make the answer empirical, not hopeful. The training set is gated at a strict trust threshold, so the loop studies only high-quality answers. The retention guard measures held-out ability after every update and rolls back any update that regresses it, so a drifting cycle is caught and undone rather than committed. And the memory is curated against the improving model each cycle, so its quality is maintained rather than left to decay. The realised commit fraction of four in five, with one genuine abort, is the direct evidence that the guard is active and that the committed cycles are the ones that earned their keep.

The per-query pipeline and the cycle loop are now both fully open. Chapter 13 steps back to the theory: the data-purity property that lets an imperfect verifier yield a clean memory, and the monotonicity and convergence results that the loop's behaviour is meant to exhibit.

PART IV

Theory and Evaluation

CHAPTER 13

The Theory: Purity, Monotonicity, Convergence

The architecture chapters established *how* CAEM works. This chapter asks *why it should work at all*, and answers the question Chapter 1 left open: if the verifier is imperfect, why does the memory not slowly fill with the verifier's mistakes? The answer is a small set of results about the *stored pool*. We teach each one from the ground up, state precisely what it does and does not claim, and pair it with what the realised run actually showed.

The one question the theory must answer

Every gate in CAEM is fallible. The verifier is better than chance but not perfect, so it will sometimes admit a wrong answer. The natural worry is a feedback spiral: the verifier's errors enter memory, the loop trains on them, the model degrades, and the errors compound. The theory's job is to show that this does not happen, that under a mild condition the stored pool ends up *cleaner* than the verifier itself, and that the system contracts toward a stable, high-purity state rather than spiralling. The whole chapter is the unpacking of that promise.

13.1 What the theorems are about, and what they are not

Before any result, one scope statement, because it governs how every claim should be read.

These are pool-purity guarantees, not model-accuracy guarantees

The theorems bound the *purity of the stored memory pool*: the fraction of stored episodes whose answer is correct. They do *not* bound the exact-match accuracy of the fine-tuned model. The two are different quantities flowing through different channels: the pool purity is a property of what the gate admits, while the model's accuracy is a property of what gradient descent does with the training pool. A cycle in which the model's accuracy on some benchmark dips is therefore *not* a violation of these theorems; it is simply outside their scope. Keeping this distinction sharp is what lets the theory make clean, provable claims about the memory while the messier question of the trained model's accuracy is handled empirically in Chapter 15. When this chapter says "purity," it always means the stored pool, never the model.

Pool purity, defined

Let $\mathcal{M}_t$ be the set of episodes in memory at the end of cycle t. Its *purity* is

$$\pi_t \;:=\; \Pr\left(\text{answer correct} \,\middle|\, \text{episode} \in \mathcal{M}_t\right),$$

the chance that a randomly drawn stored episode is right. Purity one means a spotless pool; purity equal to the base model's accuracy means the gate added nothing. The theory is the study of how π_t behaves across cycles.

13.2 The data-purity result: why an imperfect verifier is enough

This is the heart of the chapter and the direct answer to Chapter 1's objection. It rests on two quantities. The *base accuracy* p is how often the generator is right before any filtering. The *balanced accuracy* α is how good the gate is at telling right from wrong, averaged over the two cases, with $\alpha = 1/2$ meaning a coin flip and $\alpha = 1$ a perfect judge.

The purity identity

If the gate admits an answer, what is the chance it is actually correct? By Bayes' rule, writing the gate's true-positive and true-negative rates both as α for clarity,

$$\pi \;=\; \frac{p\,\alpha}{p\,\alpha \;+\; (1-p)(1-\alpha)}.$$

Read the denominator as "ways to be admitted": a correct answer admitted (rate $p\,\alpha$) plus a wrong answer wrongly admitted (rate $(1-p)(1-\alpha)$). The purity is the first term's share. The crucial algebraic fact is that whenever $\alpha > 1/2$, this π is strictly greater than p: **the pool is purer than the unfiltered generator**, and for a strong enough generator it is purer than the verifier's own accuracy. A gate only modestly better than a coin flip still manufactures a clean pool.

Why a so-so judge produces a clean pool

The trick is that the gate is applied to a *population*, and base rates do the heavy lifting. Suppose the generator is right 60% of the time and the gate is right 70% of the time. Among the answers the gate admits, the correct ones were both common to begin with and likely to be passed, while the wrong ones were less common and likely to be caught. The arithmetic above turns those two advantages into an admitted-pool purity of about 78%, higher than either the generator's 60% or the gate's 70%. This is the base-rate argument Chapter 1 promised, and it is why CAEM does not need a perfect verifier, only one that beats a coin flip on the claims it scores.

The cross-cycle floor

The identity is the expected purity for one admission. The first theorem turns it into a guarantee that holds *every* cycle, accounting for the fact that the pool is the union of newly

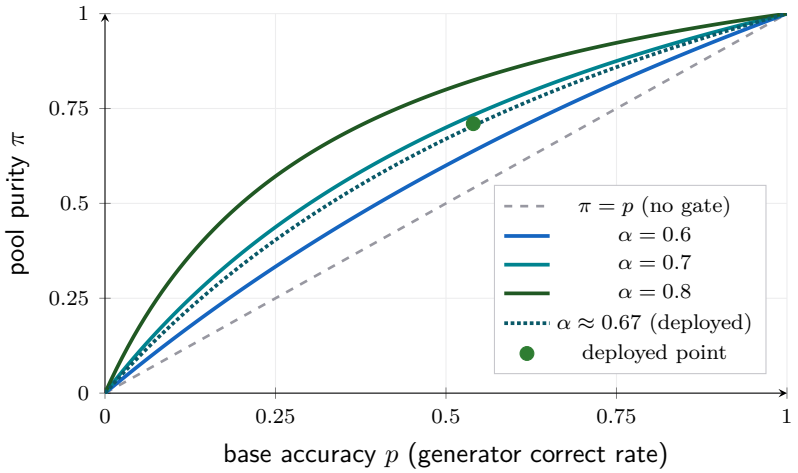


Figure 13.1: Pool purity π as a function of base accuracy p, for three gate qualities α. Every curve lies *above* the diagonal $\pi = p$ whenever $\alpha > 1/2$: the gate makes the pool purer than the unfiltered generator. A gate that is only modestly better than chance ($\alpha = 0.6$) still lifts a 60%-accurate generator to a pool purity near 0.69, and a $\alpha = 0.7$ gate lifts it to about 0.78, above the gate's own accuracy. The dotted curve at $\alpha \approx 0.67$ and the marked operating point are the deployed cold-start gate: at the realised base rate $p \approx 0.54$ the identity predicts a pool purity near 0.71, which sits just under the realised calibration-fold purity band of 0.72 to 0.75, so the deployed run lands in the band the identity anticipates. (Curve family illustrative; the marked point and the $\alpha \approx 0.67$ curve are from the deployed cold-start fold.)

admitted episodes and old episodes that survived re-verification.

Pool purity invariant

Let π_{store} be the gate's calibration-fold precision at the storage threshold, and π_{retro} the precision of episodes that survive the retroactive re-verification prune. Then for every cycle,

$$\pi_{t+1} \;\geq\; \min(\pi_{\text{store}},\, \pi_{\text{retro}}).$$

The reasoning is a one-line convexity argument: next cycle's pool is the disjoint union of survivors (each at least π_{retro} pure) and new admissions (each at least π_{store} pure), and the purity of a union sits between the two, hence above the smaller. The bound is one-sided and can be re-checked from each cycle's own calibration fold, which makes it a falsifiable contract rather than an aspiration.

The floor, and what the run showed

The registered floor is $\pi_{\text{store}} \approx 0.64$ at the storage threshold $\tau_{\text{store}} = 0.60$, with the retroactive prune at $\tau_{\text{retro}} = 0.50$ supplying the second floor. Across the realised six cycles the pooled calibration-fold purity stayed in the band 0.72 to 0.75, clearing the floor by eight to eleven points at every cycle. The guarantee held, with margin to spare. A reading below the floor would have falsified either the exchangeability assumption or the prune contract; none was observed.

The floor is a pooled guarantee, and one benchmark dips below it

The floor is stated on the *pooled* training-panel axis, not per benchmark, because the assumption it rests on (that the calibration fold and the candidate pool are interchangeable) holds on the pooled axis but not necessarily on each benchmark's slice. On the realised run the FEVER per-benchmark reading dips just below 0.64 at cycles one through four (in the band 0.61 to 0.63) and recovers at cycle five. This is reported transparently because it is real, but it is not a falsification: it is a per-benchmark diagnostic outside the pooled scope the theorem actually claims. Stating the scope precisely is what lets the dip be informative rather than alarming.

13.3 Monotonicity: a better generator stores more

The second theorem connects the loop's effect on the generator to the gate's behaviour. As the self-improvement loop sharpens the generator, the calibration-fold separation between correct and incorrect answers, measured by Cohen's d (a standard effect size: the gap between the two groups' means in units of their spread), widens. The theorem asks what that does to the storage rate σ, the fraction of answers admitted.

Sign of the storage-rate response

Modelling the storage score as approximately Gaussian within each correctness class, the direction in which the storage rate moves as the separation d grows is

$$\operatorname{sign}\left(\frac{\partial \sigma}{\partial d}\right) \;=\; \operatorname{sign}\left(p_{+}\phi_{+} - p_{-}\phi_{-}\right),$$

where $\phi_{\pm}$ are the densities of the correct and incorrect classes at the threshold. In words: when the operating point sits in the high-precision regime, where the correct class has more mass at the threshold than the incorrect class, a better generator (larger d) admits *more* answers, $\sigma_{t+1} \geq \sigma_t$. Improving the model and holding the threshold fixed lets more of its now-better answers clear the bar.

The realised receipt is honest about being partial

This is the theorem whose empirical support is weakest, and the thesis says so. Of the three observable cycle-to-cycle transitions, one aligns strongly with the prediction, one is too small to read, and one moves the opposite way. The reverse-sign transition is not treated as a refutation but is attributed to two effects the simple Gaussian model leaves out: the composite is re-fitted each cycle (so the storage score is not a fixed function), and the within-class spread contracts across cycles (so the shared-variance assumption is approximate). A clean test would freeze the composite for a cycle and let only the generator move, which is registered as future work. Reporting a load-bearing theorem with a partial receipt, and naming exactly why, is the honest posture.

13.4 Convergence: the gap contracts to a stable band

The third theorem describes the long-run trajectory of purity. Writing π_∞ for the asymptotic purity and $G_t = |\pi_t - \pi_\infty|$ for the remaining gap, the gate-plus-loop update behaves as a contraction.

Contraction envelope, and why it is reframed

Under a stationarity assumption on the per-cycle storage rate and separation, the gap shrinks geometrically in expectation,

$$\mathbb{E}[G_t] \;\le\; (1-r)^t\, G_0, \qquad r \in (0,1).$$

Each cycle admits fresh high-purity episodes and re-scores old ones under the improved generator, and the expected purity moves a fixed fraction r of the way toward its limit. But six cycles is far too short to estimate a rate r cleanly, so the claim the thesis actually registers is the weaker, testable one: *bounded-band stationarity*, that π_t stays inside a narrow band rather than diverging. On the realised run the calibration-fold purity traced a band of width below 0.03, consistent with contraction and inconsistent with a spiral; the sharper geometric-rate test is deferred to a longer horizon.

The idealised limit, and why reality sits below it

A companion identity shows that, in an idealised world where the verifier's quality stays fixed above one half and nothing else interferes, the purity recurrence climbs strictly toward one: keep filtering with a better-than-chance gate and the pool tends to perfection. CAEM does not reach that ideal, and the theory says why. Three deliberate regularisers pin the realised ceiling strictly below one: the frozen backbone and bounded adapter cap how far the model can move, the retention guard turns any regressive cycle into a no-op, and the finite capacity of the generator and the retrieval corpus impose a hard floor on error. The gap between the realised plateau and the theoretical one is the price of these safety mechanisms, not a failure of the mathematics.

13.5 The Tier-1 floor: memory hallucination is bounded by pool impurity

The first corollary cashes the purity guarantee out into a user-facing rate. The memory-direct tier serves a stored answer verbatim, so its error rate is exactly the pool's impurity. Hence

$$\limsup_{t\to\infty}\ \mathrm{CHM}_1(t) \;\le\; \max\bigl(1-\pi_{\mathrm{store}},\, 1-\pi_{\mathrm{retro}}\bigr),$$

where CHM_1 is the memory tier's hallucination metric and lim sup is the long-run worst case (it tolerates bounded wobble rather than demanding a smooth descent).

A capability bound, not a trajectory

This corollary must be read as a statement of *capability*, not a measured curve. By design the evaluation set is held disjoint from the training stream, so the memory tier fires on almost nothing during evaluation, seven times across nine thousand held-out queries, every one returning the correct stored answer. The asymptotic hallucination rate of a tier that rarely fires is not observable on this horizon. So the corollary says "the memory tier cannot hallucinate faster than the pool is impure," a sound bound, and the realised receipt is the targeted routing probe rather than a trajectory. The conservative ceiling of about 0.36 from the cold-start fold comfortably overshoots the realised pool impurity of roughly 0.25, which the retroactive prune tightens further.

13.6 Self-correction: bad episodes are pruned in finite time

The most reassuring corollary addresses the feedback-spiral worry head on. Each stored episode is re-scored at every cycle boundary by the same verifier against the improved generator. Treat surviving a re-verification pass as a noisy yes/no test of the episode's correctness.

Why a wrong episode cannot survive forever

For a truly correct episode, each pass is more likely to keep it than drop it; for a wrong one, more likely to drop it than keep it, provided the verifier's balanced accuracy stays above one half. The evidence from repeated passes *multiplies*: the likelihood ratio in favour of correctness, given survival of k passes, grows without bound, so the chance a survivor is actually correct tends to one. Concretely, a wrong episode's per-cycle prune probability is at least $\delta = 2\alpha - 1 > 0$, so its expected survival time is at most $1/\delta$ cycles, a finite number. Hallucinations do not accumulate; they are flushed.

The directional receipt

The clean prediction (every wrong episode pruned eventually) is asymptotic and not testable in six cycles, so the registered receipt is directional: the per-cycle prune rate should decline as the pool's quality rises. It did, monotonically, from thirteen percent at cycle one to nine, then seven, then four percent at cycle five, as the verifier found fewer low-quality entries to remove. The trend is the supported reading; an exact rate is deferred to a longer run.

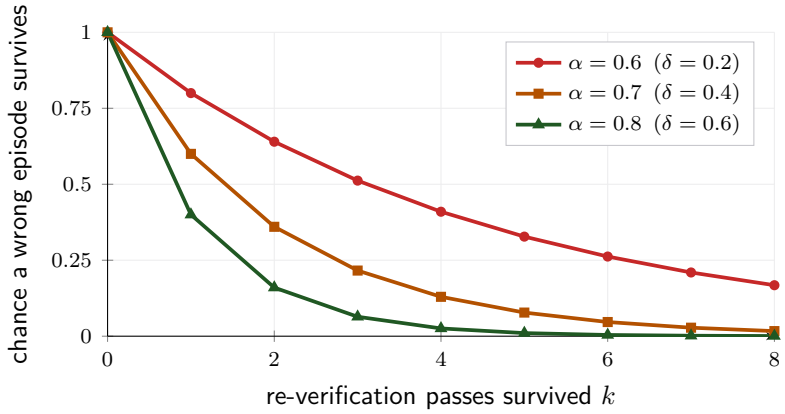


Figure 13.2: (Illustrative.) A wrong episode's chance of surviving k re-verification passes decays geometrically, at a rate set by the verifier's margin over chance $\delta = 2\alpha - 1$. Even a modest gate ($\alpha = 0.6$) drives the survival chance below ten percent within ten passes; a stronger gate flushes errors in a handful. This is why repeated re-verification prevents a feedback spiral rather than causing one. The deployed run carries no per-episode k-pass survival record; the directional receipt that the chapter does report, the per-cycle prune rate declining from about thirteen to four percent, is a distinct aggregate quantity rather than a measurement of these curves.

13.7 Stochastic equilibrium: learning under a guard that sometimes says no

The last corollary folds the retention guard of Chapter 12 into the convergence picture. Because a cycle commits only when the worst retention probe holds above the floor, the commit events form a coin-flip sequence with some success rate π_{commit}, and the contraction runs only on the cycles that commit. Three consequences follow.

The commit rate is a real fraction.
On any trajectory where the guard ever fires, π_{commit} lies strictly between zero and one. The realised sequence of commit decisions was keep, keep, keep, abort, keep, a commit fraction of four in five, with worst-probe ratios 1.006, 0.949, 0.976, 0.886, 0.958 against the floor 0.93.

Calendar progress slows, but does not stop.
The contraction envelope on the calendar timeline becomes $(1 - \pi_{\text{commit}} \cdot r)^t$: aborted cycles simply do not advance the model, so convergence is paced by the commit rate.

Memory grows regardless.
The fresh-data path runs on every cycle, committed or not, so the memory and the per-tier coverage grow monotonically even across an abort. This is the asymmetric rollback of Chapter 12 stated as an invariant: weights advance only on commits, but memory advances always.

13.8 The results, assembled

The results lock together, as Figure 13.3 shows. The purity identity and its cross-cycle floor establish that the pool stays clean; monotonicity describes how the gate responds as the generator improves; convergence bounds the trajectory to a stable band; the Tier-1 floor turns pool purity into a user-facing rate; self-correction guarantees errors are flushed; and stochastic equilibrium accounts for the guard. Together they answer Chapter 1's objection in full: an imperfect verifier is enough, errors do not compound, and the system contracts toward a stable, high-purity state.

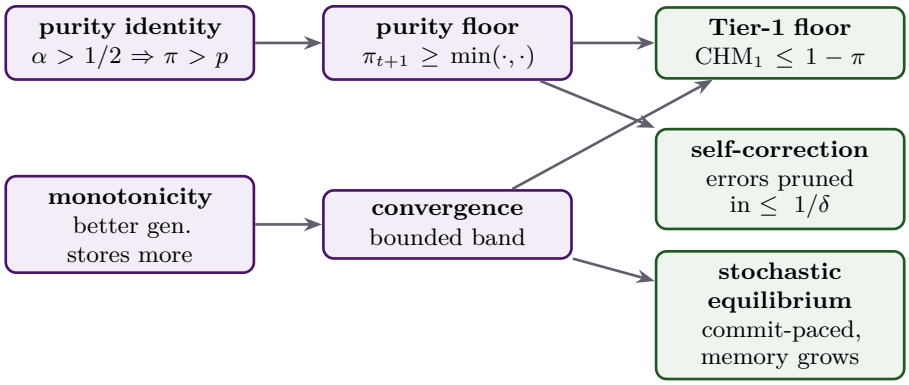


Figure 13.3: The dependency map. The purity identity (an imperfect gate yields a clean pool) underpins the cross-cycle floor; the floor and the convergence bound feed the Tier-1 hallucination ceiling and the finite-time self-correction; monotonicity and convergence, qualified by the retention guard, give the stochastic equilibrium. Purple nodes are the three load-bearing theorems; green are the corollaries. (Schematic.)

If an examiner asks: "do these theorems prove CAEM reduces hallucination?"

They prove a precise, bounded thing and not a sweeping one. They establish that the stored *memory pool* stays clean under a verifier only modestly better than chance, that it cannot spiral, and that the memory tier's error is bounded by the pool's impurity. They do *not* prove that the fine-tuned model's accuracy rises on every benchmark; that is a property of the training gradient, outside the pool-level scope, and is settled empirically in Chapter 15. The honest framing is that the theory secures the data foundation, a clean, self-correcting memory, and the empirical chapters measure what the model built on that foundation actually achieves. Several results carry partial or asymptotic receipts at the six-cycle horizon, and each is reported with its scope rather than overclaimed.

The theory in constants

- Precondition: verifier balanced accuracy $\alpha > 1/2$ on the claims it scores.
- Purity floor: $\min(\pi_{\text{store}}, \pi_{\text{retro}})$, registered at ≈ 0.64; realised band 0.72 to 0.75.
- Tier-1 hallucination ceiling: $\max(1 - \pi_{\text{store}}, 1 - \pi_{\text{retro}})$; realised pool impurity ≈ 0.25.
- Self-correction: wrong episode expected survival $\leq 1/\delta$ cycles, $\delta = 2\alpha - 1$; prune

rate fell 13% → 4%.

- Commit fraction: $\pi_{\text{commit}} = 4/5$ realised; convergence paced by $(1 - \pi_{\text{commit}} r)^t$.
- Scope: all results bound the *stored pool*, not the model's exact-match accuracy.

The theory secures the foundation. Chapter 14 turns to measurement: the five benchmarks, the eight external baselines, and the metrics, the exact-match and composite-hallucination axes, that the next chapters use to report what the realised system achieved.

CHAPTER 14

Benchmarks, Baselines, and Metrics

The previous thirteen chapters built the machine and proved properties about it. This chapter does something different: it lays out the *instruments* that measure whether the machine works. Three instruments, in order. The **benchmarks** decide what we ask the system. The **baselines** decide what we compare it against. The **metrics** decide how we score what comes back. Chapter 15 and Chapter 16 will report what these instruments measured; this chapter is about the instruments themselves, because a result is only as trustworthy as the ruler that produced it.

Why a chapter on measurement

It is easy to make a hallucination-reduction system look good by choosing a favourable benchmark, a weak baseline, or a metric that rewards the behaviour the system happens to produce. CAEM commits in advance to a fixed panel of benchmarks, a panel of eight baselines that each isolate one competing idea, and a metric suite that scores honesty and accuracy separately. The discipline is the point: every number in the next two chapters is produced by an instrument defined here, on data the system never trained on, under a protocol that is identical for CAEM and for every baseline it is compared against.

14.1 The benchmark panel

CAEM is evaluated on a fixed panel of five question-answering benchmarks. They are not interchangeable: each stresses a different failure surface, and they are split into two roles that the whole continual-learning story depends on.

Training benchmarks and transfer benchmarks

A self-improving system has an obvious way to cheat: it can get better at exactly the tasks it trains on while quietly getting worse everywhere else. To catch this, the panel is split. Three benchmarks are *training* benchmarks: the self-improvement loop is allowed to learn from their answers. Two are *transfer* benchmarks: they are evaluated every cycle but the loop never trains on them, so they measure whether an improvement generalises or is just memorisation of the training distribution. A gain that shows up only on the training benchmarks is suspect; a gain that also shows up on the held-out transfer benchmarks is real.

Benchmark	What it tests	Answer surface	Role
FEVER [21]	factual claim verification	supports / refutes / not enough info	training
TriviaQA [22]	open-text factoid recall	free-text entity (many aliases)	training
CommonsenseQA [23]	everyday commonsense	five-option multiple choice	training
TruthfulQA [1]	common-misconception probing	free-text answer	transfer
StrategyQA [24]	implicit multi-step reasoning	yes / no	transfer

Table 14.1: The evaluation panel. Three training benchmarks feed the self-improvement loop; two transfer benchmarks are held out to measure generalisation. All five are evaluated at every cycle.

What each benchmark stresses

The five are chosen to span distinct surfaces so a single metric cannot be gamed. *FEVER* is a fact-checking benchmark with a bounded three-label space; the answer is a verdict, not free text, so the failure mode is mislabelling. *TriviaQA* is open-text factoid recall with many acceptable aliases per question, so the failure mode is fabrication of a plausible-but-wrong entity. *CommonsenseQA* is a five-option multiple-choice benchmark on everyday reasoning with a bounded label space. *TruthfulQA* is adversarial: its questions are written so that the most fluent answer is a common human misconception, so it specifically rewards a system that knows when not to commit. *StrategyQA* asks yes/no questions whose reasoning chain is implicit and must be decomposed, stressing multi-step inference. The two transfer benchmarks (TruthfulQA, StrategyQA) are exactly the ones the loop never sees during training.

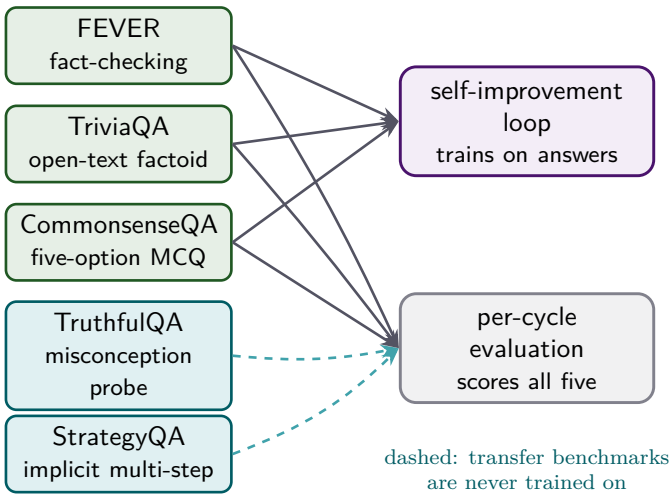


Figure 14.1: The panel split. The three training benchmarks (green) feed the self-improvement loop and are also evaluated; the two transfer benchmarks (orange) are evaluated every cycle but never enter training, so they measure generalisation rather than memorisation. (Schematic.)

The panel was pruned for a principled reason

The five-benchmark panel is what survived a cold-start screening. Three further benchmarks were considered and removed before the main run, and the reason ties directly back to the

data-purity theorem of Chapter 13.

Three benchmarks were removed, and why

The purity theorem requires the verifier to clear a balanced-accuracy floor on the claims a benchmark produces; below that floor, storing is worse than not storing. A cold-start screening measured this and rejected three benchmarks. A multi-hop benchmark's base correctness was about 0.09, which demanded a verifier discrimination far beyond what any signal achieved, so its episodes were *mathematically unstoreable*. An open-domain benchmark sat at about 0.16 with a verifier balanced accuracy below one half on every calibration fold, a structural-failure benchmark. A four-option science benchmark was dropped as redundant with the five-option commonsense benchmark already in the panel. All three are kept on disk as registered exclusion evidence; none enters the active trajectory. The lesson is that the panel is not a convenience sample, it is the set of benchmarks on which the architecture's own precondition is met.

Six disjoint pools, and why leakage would invalidate everything

Within each benchmark, every sample is assigned to exactly one of six pools, and the assignment is enforced to be leak-free. This is the single most important piece of evaluation hygiene in the whole project.

The six pools

Each benchmark's samples are partitioned into six disjoint pools: a *seed* pool that bootstraps the cold-start memory, a *purity* pool that measures the storage precision, a *calibration* pool on which the composite of Chapter 10 is fitted, per-cycle *training* chunks that the self-improvement loop consumes (one disjoint chunk per cycle), an *evaluation* pool that produces the per-cycle trajectory numbers, and a held-out *test* pool reserved for the final generalisation table. A sample that appears in training or calibration must never reappear in evaluation or test, otherwise the reported accuracy and calibration would be inflated by memorisation rather than measuring generalisation. Transfer benchmarks populate only the evaluation and test pools; their training, seed, calibration, and purity pools are empty by construction.

Content-hash deduplication and the leakage guards

Disjointness is guaranteed by a content hash, not by a dataset's native identifiers, because source datasets reuse question text under drifting identifiers across versions. Each sample's pool key is the first sixteen hexadecimal characters of a SHA-256 digest of its question text, so two questions with identical text collide into the same key regardless of how the dataset labelled them. Before any pool is used, an assertion flattens all six pools and raises an error if a single content key appears in more than one; a second assertion checks that no key appears in two different benchmarks. Any duplicate halts the run immediately. The deduplicate-then-slice order is what lets the evaluation and test numbers be read as generalisation rather than recall.

Pool sizes in the realised run

In the realised trajectory each training benchmark allocated roughly one thousand seed candidates, five hundred calibration samples, five hundred purity samples, a per-cycle training chunk (two thousand for the two large benchmarks, seven hundred for the smaller commonsense benchmark), three hundred evaluation samples, and up to five hundred test samples. Each transfer benchmark allocated only its three hundred evaluation samples and its test samples. The headline figure to carry is the *evaluation fold*: three hundred samples per benchmark across five benchmarks, so every cycle's trajectory number is computed over fifteen hundred held-out questions. The calibration fold that the composite is fitted on is a separate five hundred per training benchmark, fifteen hundred in total, and is disjoint from the evaluation fold.

The retention probes

One more measurement surface deserves its own mention, because it is what the retention guard of Chapter 12 reads. It is deliberately *not* one of the five panel benchmarks.

Three held-out probes, anchored once

The retention guard scores the fine-tuned model on three probes that span different abilities: a general-knowledge multiple-choice probe, a held-out open-text factoid probe, and a held-out commonsense multiple-choice probe, two hundred questions each. Their cold-start scores are anchored once, to about 0.61, 0.395, and 0.825 respectively, and that anchor is reused as the denominator for every later cycle. A cycle aborts if any probe's ratio against its anchor falls below the floor $\rho_{\min} = 0.93$. The probes are held out from both training and the panel evaluation folds, so a high retention ratio cannot be bought by overfitting the panel. This is the multi-modal guard that fired at the fourth cycle of Chapter 15, when the open-text factoid probe ratio fell to 0.886.

14.2 The baselines: what we compare against

A trajectory number in isolation means nothing. The question is always *compared to what*. CAEM answers this with a panel of eight baselines, and the panel is constructed so that each baseline isolates exactly one idea that a sceptic might propose as the *real* source of any gain.

Each baseline removes one explanation

Suppose CAEM reports a gain over a plain language model. A reviewer can offer many deflating explanations: maybe it is just the prompt, maybe just chain-of-thought, maybe just retrieval, maybe just fine-tuning. The baseline panel is built to answer each of these in turn. Every baseline runs on the *same* frozen backbone as CAEM, so no comparison is contaminated by a stronger model; each adds exactly one of the competing ingredients; and the gap between CAEM and each baseline is the value of everything CAEM adds beyond that ingredient. No single baseline combines all the ingredients, which is precisely the gap the architecture occupies.

	Baseline	What it isolates
B1	zero-shot	the empirical floor: the bare backbone with the same answer format, no reasoning trigger, no retrieval, no memory, no verifier
B2	chain-of-thought	the effect of a zero-shot reasoning trigger over the bare format [11]
B3	retrieval-augmented	unconditional grounding: every query retrieves, no router, no verifier [20]
B4	reasoning + retrieval	a reasoning trigger layered on top of retrieval
B5	five-shot reasoning	in-context demonstrations instead of fine-tuning [11]
B6	vanilla fine-tuning	plain supervised fine-tuning with no retention guard, no memory, no verifier
B7	active retrieval	look-ahead-triggered retrieval during generation [25]
B8	semantic-entropy	sampling-uncertainty abstention from a single model family [14]

Table 14.2: Each baseline runs on the same frozen backbone as CAEM and isolates one competing explanation for a gain. B1 is the floor; B3 and B7 isolate retrieval; B6 isolates training-time learning; B8 isolates sampling-based abstention.

The matched protocol

The comparison is fair only if every system is scored under identical conditions, so three things are pinned. First, the *backbone*: every baseline uses the same frozen three-billion-parameter instruction model CAEM uses, so a gain is never attributable to a bigger model. Second, the *evaluation fold*: baselines are scored on the byte-identical, content-hash-matched evaluation samples CAEM saw, produced by the same seeded split. Third, the *answer format*: every baseline emits its answer through the same reasoning-then-answer scaffold CAEM uses, so the answer extractor reads the same shape from all systems. With these pinned, any difference in score is attributable to mechanism, not to setup.

The format floor: why "raw" exact match lies

An earlier protocol let the generation-only baselines emit bare prose with no explicit answer line. The answer extractor then found nothing to extract, and strict exact match read close to zero *even when the prose contained the correct answer*. On the open-text factoid benchmark the bare zero-shot model was right in its prose about fifty-nine percent of the time yet scored near zero on extracted exact match. The fix is the matched protocol above: every baseline is required to commit to an explicit answer line under the same instruction CAEM follows, so its exact match reflects what it knew rather than how it happened to format the output. The book reports the matched-protocol numbers throughout, because a baseline made artificially weak is as dishonest as a system made artificially strong.

Scoring the baselines through the same verifier

A baseline owns no verifier, so on its own it produces no hallucination signals and no calibrated trust score. To place every system on the same honesty axis, each baseline's question-and-answer pairs are replayed through the *locked* CAEM verifier and composite, which recomputes the grounding, entailment, relevance, and consistency signals and the four-outcome decision for that answer. The same locked instrument scores CAEM and every baseline, so any difference in the hallucination metric is a property of the generator's output, not of a different ruler. This is the standard practice for cross-system hallucination comparison: fix the judge, vary the system under test.

If an examiner asks: "how much of the gain is just retrieval?"

This is exactly what the panel is designed to answer, and the answer is built into its structure. The contrast against the zero-shot floor (B1) isolates the joint contribution of routing, verification, memory, and self-improvement together. The contrast against unconditional retrieval (B3) isolates what the architecture adds *beyond* grounding every query. The contrast against vanilla fine-tuning (B6) isolates what the deployment-time machinery adds beyond training-time learning. And the difference between the B1 contrast and the B6 contrast decomposes the gain between deployment-time mechanisms and training-time mechanisms. No reviewer can attribute the result to retrieval alone, because B3 and B4 already hold retrieval fixed; the remaining gap is everything else.

14.3 The metrics

The final instrument is the scorer. CAEM reports a small suite of metrics, and the central design decision is that *accuracy and honesty are scored on separate axes*. A system can be more accurate, or more honest, or both, and conflating the two is how the field has historically rewarded confident wrongness.

Exact match and token overlap

Exact match, per task family

Exact match is the headline accuracy axis and the basis for every paired comparison. It is computed per task family after a shared normalisation that lowercases, strips punctuation, removes the articles *a*, *an*, and *the*, and collapses whitespace, matching the standard convention. On the fact-checking benchmark it is label-exact over the three verdicts; on the open-text factoid benchmark it is a match against any of the gold aliases, with token-level overlap reported alongside for partial credit; on the two multiple-choice and yes/no benchmarks it is label-exact. The extractor reads the final answer out of the reasoning-then-answer scaffold before scoring, so a correct answer that follows its reasoning is still counted.

Listing 14.1: Exact match and answer extraction, condensed from `eval/metrics.py`.

```
def normalise(text):                                  # SQuAD / TriviaQA convention
    text = text.lower().translate(PUNCT_TABLE)
    return " ".join(t for t in text.split() if t not in {"a", "an",
        "the"})

def extract_cot_answer(text):                         # pull the answer out of the
    scaffold
    if "Answer:" in text:
        return text.split("Answer:")[-1].strip()
    m = search(r"the answer is[:\s]+(.+?)(?:[.!?]|$)", text)    #
        natural-language CoT
    return m.group(1).strip() if m else text.strip()

def exact_match(prediction, gold):
    return float(normalise(prediction) == normalise(gold))
```

The misconception benchmark needs a language-model judge

On the misconception benchmark, surface-overlap scoring is actively misleading. The benchmark provides reference answers, and a token-overlap score rewards an answer for echoing the misconception's wording even when it asserts the false belief. A bare model that fluently restates the common misconception can therefore score high on overlap while being exactly wrong. For this benchmark the book uses a language-model judge that reads the question and answer and rules on truthfulness, not token overlap. The judged correctness is far lower, and far more honest, than the overlap score: the same answers an overlap metric scores around 0.80 are judged correct only around 0.43. Wherever the misconception benchmark's accuracy appears in this book, it is the judged value, never the overlap value.

The composite hallucination metric

Accuracy says whether the answer was right. It says nothing about *how* the system failed when it was wrong. The composite hallucination metric is the honesty axis, and it is a structured decomposition rather than a single rate.

Eight failure modes, equally weighted

The composite hallucination metric is the equal-weighted mean over an eight-axis failure-mode taxonomy. The eight axes are: confident confabulation (wrong while the calibrated trust score is at least one half), factual fabrication (wrong while the weakest atomic fact is ungrounded), logical fabrication (wrong while the reasoning does not entail the answer), off-topic answering (wrong while the answer is irrelevant to the question), defensive evasion (wrong while emitting a stock "cannot determine" phrase), template leakage (a placeholder string leaked from the prompt), false refusal (the answer was actually correct yet the system abstained or discarded it), and length-padded over-generation (wrong while the answer is implausibly long). A ninth axis, factual contradiction, is part of the taxonomy but excluded from the default denominator,

because the deployed natural-language inference judge produces no contradiction class and the corresponding rate is structurally zero. The subtypes overlap deliberately, so the metric is a mean prevalence across failure modes, not the fraction of samples with any failure.

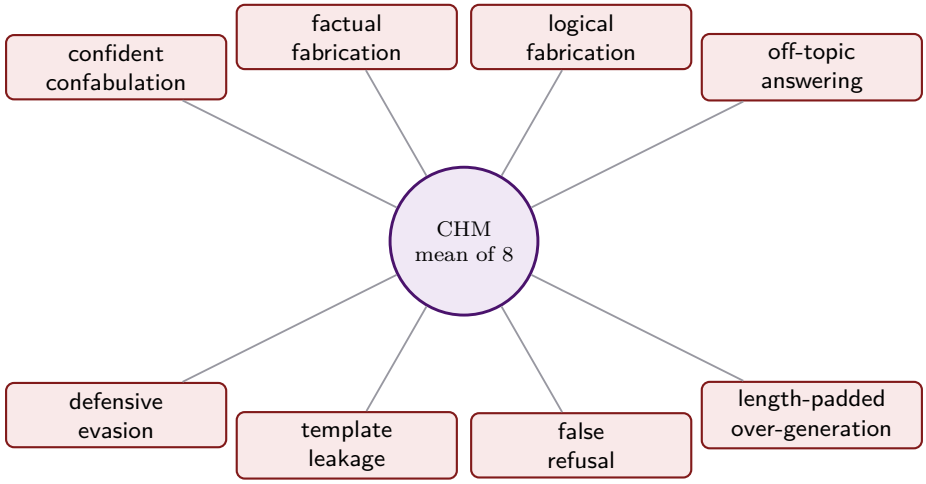


Figure 14.2: The composite hallucination metric is the equal-weighted mean of eight measured failure-mode rates. Each axis is a distinct way to be wrong; weighting them equally avoids privileging one definition of hallucination over another. A ninth axis (factual contradiction) is in the taxonomy but is structurally zero under the deployed judge and excluded from the denominator. (Schematic.)

Listing 14.2: The composite hallucination metric, condensed from `eval/metrics.py`.

```
SUBTYPES = ["confident_confabulation", "factual_fabrication",    # 8
    measured axes
            "logical_fabrication", "off_topic", "defensive_evasion",
            "template_leak", "false_refusal", "over_long"]       #
                contradiction excluded

def chm(samples):
    rates = hallucination_subtypes(samples)        # one prevalence per
        axis
    return sum(rates[s] for s in SUBTYPES) / len(SUBTYPES)    #
        equal-weighted mean
```

A confident confabulation scores on one axis

Take a wrong answer the system nonetheless stored, with a calibrated trust score of 0.62. It is incorrect and its trust is above one half, so it fires the confident confabulation axis, the costliest failure: the model was wrong and apparently sure. If its weakest atomic fact were also ungrounded it would additionally fire factual fabrication, and a single bad answer can light up several axes at once. The metric counts each axis's prevalence across the fold and averages the eight, so a system that trades confident confabulations for honest abstentions lowers one axis without inflating the others, and

its composite falls. That is exactly the trade the gate of Chapter 10 is built to make.

What fires each axis

Each axis is a prevalence: count the samples on the fold that trip its condition, divide by the fold size, and that fraction is the axis rate. Every condition is the same right-or-wrong verdict (defined just below) paired with one verifier signal or one text pattern. Table 14.3 is the exact trigger for each axis, transcribed from the scoring routine in `eval/metrics.py`. Two conditions are worth singling out before the table. Template leakage carries no correctness guard at all: a leaked placeholder is a defect whether or not the answer happened to be right. False refusal is the only axis that fires on a *correct* answer, because refusing to serve an answer you actually held is its own failure mode.

Axis	**Fires on a sample when**
Confident confabulation	the answer is *wrong* and $u_{\text{stored}} \geq 0.50$
Factual fabrication	the answer is *wrong* and $p_{\text{ground,atomic}} < 0.30$
Logical fabrication	the answer is *wrong* and $p_{\text{entail}} < 0.30$
Off-topic answering	the answer is *wrong* and $q_{\text{a,rel}} < 0.50$
Defensive evasion	the answer is *wrong* and the text matches an evasion phrase (“cannot determine”, “the context does not mention”)
Template leakage	the text matches a leaked placeholder such as “ <concise factual answer>”, *regardless of right or wrong*
False refusal	the answer is *correct* yet the gate tag is abstain or discard
Length-padded overgeneration	the answer is *wrong* and exceeds 600 characters
(excluded) Factual contradiction	wrong and the contradiction signal exceeds 0.30; structurally zero under the deployed judge, so kept out of the denominator

Table 14.3: The exact trigger for each axis, from `eval/metrics.py`. A single bad answer can satisfy several rows at once, which is why the eight rates are averaged rather than summed.

Where right and wrong come from

Almost every row of Table 14.3 hinges on whether the answer was right or wrong, so that verdict is the load-bearing input. It is a single binary field, exact match, attached to each sample: one means correct, zero means wrong, and a missing value counts as wrong. The axes simply read it as “wrong” (EM = 0) or “correct” (EM = 1).

How that one bit is computed is not uniform across the panel, and the differences matter. Before anything is compared, the final answer is extracted from the reasoning trace (the text after an “Answer:” marker, or the phrase “the answer is X”, else the whole string), and both sides are normalised the SQuAD way: lowercased, stripped of punctuation, with the articles *a*, *an*, and *the* removed and whitespace collapsed. Then the match rule depends on the benchmark, as in Table 14.4.

Benchmark	How exact match is decided
FEVER	extract the three-class label (supports / refutes / not-enough-info); one if it equals the gold label
TriviaQA, Natural Questions	one if the normalised answer equals *any* of the gold aliases (there can be ten to forty per question)
StrategyQA	extract yes or no; one if it equals the gold boolean
ARC-Challenge	extract the option letter; one if it equals the gold letter
TruthfulQA	one if the best ROUGE-L overlap with a gold answer exceeds 0.15; in the thesis this proxy is replaced by a judge model's verdict, which is the figure that should be reported
ASQA	one if the best ROUGE-L overlap exceeds 0.20 (longer golds, higher bar)
Generic open-ended QA	one if the normalised answer equals the single gold string

Table 14.4: The right-or-wrong bit is computed differently for each benchmark, from `eval/harness.py`. This is the verdict every hallucination axis then reads.

The TruthfulQA verdict is not the surface one

On TruthfulQA the offline match rule is a ROUGE-L overlap threshold, which rewards sharing surface tokens with a gold answer even when the claim is false. The honest hallucination figure for TruthfulQA uses a judge model's correctness verdict in place of that overlap, and the composite is recomputed against it. Feed the wrong field into the right column of Table 14.3 and every TruthfulQA axis inherits the error.

One sample, end to end

Take a TriviaQA item whose gold aliases are {Drake Bell, Jared Drake Bell}. The model emits "The show aired on Nickelodeon. The answer is Josh Peck." Extraction keeps "Josh Peck", normalisation gives "josh peck", and that matches no alias, so exact match is zero: the sample is *wrong*. The verifier reported, for this answer, a calibrated trust of 0.508, a weakest-atomic-fact grounding of 0.22, an entailment of 0.40, a question-answer relevance of 0.71, a gate tag of discard, and a length of fifty-three characters. Walking Table 14.3: confident confabulation fires (wrong and $0.508 \geq 0.50$); factual fabrication fires (wrong and $0.22 < 0.30$); logical fabrication does not (0.40 is not below 0.30); off-topic does not (0.71 is not below 0.50); evasion and template leakage find no matching phrase; false refusal needs a *correct* answer; and over-generation needs more than six hundred characters. So this one sample lights two of the eight axes, and counts in both at once. (Illustrative signal values.)

Now scale to the fold. Suppose across the benchmark the eight axis rates come out as 0.150, 0.110, 0.090, 0.070, 0.020, 0.000, 0.010, and 0.040. The composite is their flat average, $(0.150+0.110+0.090+0.070+0.020+0.000+0.010+0.040)/8 = 0.490/8 = 0.061$. Because the axes overlap, the share of samples with *any* defect is higher than 0.061; the composite is a mean of failure-mode prevalences, not that union. (Illustrative rates.)

The composite evaluation score

The accuracy axis and the honesty axis can move in opposite directions, and so can calibration and retention. A single headline scalar is useful only if it refuses to let one strong axis hide a collapsed one.

A geometric mean of five axes

The composite evaluation score is the geometric mean of five axes, each in the unit interval with one as best: accuracy (pooled exact match), epistemic quality (one minus the composite hallucination metric), retention (the worst of the three probe ratios, capped at one), calibration (one minus twice the calibration error, floored at zero), and verifier reliability (the verifier's balanced accuracy rescaled so chance maps to zero). The mean is geometric, not arithmetic, on purpose: a near-zero axis drags the whole score down, so a system cannot buy a good headline by excelling on four axes while catastrophically forgetting on the fifth. Each axis is clamped just above zero so a single uncomputed axis cannot zero the score outright. The score is a visualisation and headline tool only; the book always reports the five component axes beside it so the reader can reconstruct it.

Listing 14.3: The composite evaluation score, condensed from `eval/metrics.py`.

```
def ces_score(acc, epi, ret, cal, ver, eps=0.01):    # five orthogonal
    axes in [0,1]
    axes = [min(max(a, eps), 1.0) for a in (acc, epi, ret, cal, ver)]
    prod = 1.0
    for a in axes:
        prod *= a
    return prod ** (1.0 / len(axes))                  # geometric mean:
        any collapse hurts
```

Two conventions for the score, do not conflate them

The five-axis score is fully defined only for a cyclic, calibrated, verifier-gated system: retention, calibration, and verifier reliability are not properties a single-pass baseline even has. So there are two conventions. The *system-internal* convention reports the full five-axis mean for CAEM across cycles, and is the one to read as the headline. A *cross-system* convention compares CAEM and the baselines on only the two shared axes, accuracy and epistemic quality, and defaults the other three to one for the baselines purely so the arithmetic runs. Under that second convention a baseline can show a higher number than CAEM, but only because three of its axes were set to one by fiat rather than measured. That is a bookkeeping artefact, never a finding; the baselines were not measured on retention, calibration, or verifier reliability because they do not implement them.

Significance, and the rule for licensing a claim

Paired tests, corrected across the panel

Because every system is scored on the identical evaluation samples, the comparisons are paired, and paired tests are used throughout. On the exact-match axis the test is McNemar's test with a continuity correction, the maximum-likelihood test for matched binary outcomes [26]. On the hallucination axis, which is continuous, the test is the paired Wilcoxon signed-rank test. Both report a bias-corrected bootstrap ninety-five percent interval on the delta beside the p-value, because a p-value alone does not convey effect size. Within each benchmark the eight baseline contrasts are corrected for multiple comparisons by the Holm procedure [27], which controls the family-wise error rate at higher power than a plain Bonferroni correction.

When a claim is allowed to be called supported

A single significant p-value is not enough to license an architectural claim, because with enough samples a trivially small difference becomes significant. The book uses a conjunction: a contrast counts as supporting only if its Holm-corrected p-value is below five percent *and* the bootstrap interval on the exact-match delta lies entirely above two percentage points. Meeting one of the two is partial support; meeting neither is unsupported. This pre-registered rule is what separates a result worth building an architecture on from a statistical curiosity.

The decision distribution and the abstention lens

Four outcomes, and what abstention buys

Because CAEM can decline to answer, accuracy alone misreads it. The four-outcome decision distribution (store, defer, discard, abstain) is reported alongside accuracy, and the per-outcome accuracy is strictly ordered: stored answers are the most accurate, abstentions the least, with defer and discard between. The honest reading of the misconception benchmark depends on this. A bare model is not licensed to abstain, so it answers every adversarial question and pays the price in confident wrongness; CAEM abstains on roughly a quarter to a third of those questions. Its strict exact match therefore looks lower, while its hallucination metric on the same benchmark falls: the answer set is becoming more honest even as the strict-correctness rate declines. Exact match is the wrong lens for honesty, which is precisely why the book reports the honesty axis separately.

14.4 Reading the panel: what good looks like

The full trajectory is the subject of Chapter 15 and the ablations are the subject of Chapter 16. But the instruments are easier to trust once we have seen them produce a headline, so this section reads one real comparison end to end.

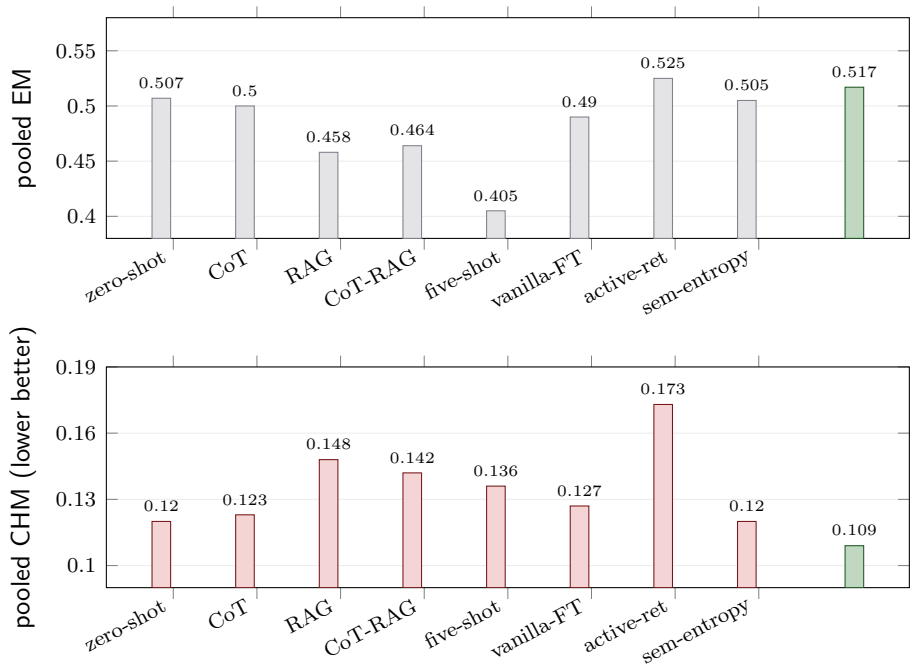


Figure 14.3: The actual deployed comparison at the final cycle. Top: pooled exact match across the five-benchmark fold; CAEM sits in the upper band, at parity with the strongest baseline (active retrieval edges it by under one point). Bottom: pooled composite hallucination metric, where CAEM is the lowest (most honest) of every system, beating active retrieval by about thirty-seven percent relative. The headline is read on both axes at once: accuracy parity with the best baseline, and a clear honesty win over all of them.

The headline, on two axes

At the final cycle CAEM reaches a pooled exact match of 0.517 and a pooled hallucination metric of 0.109 over the five-benchmark fold. Of the eight baselines, the best on accuracy is active retrieval at 0.525, edging CAEM by under one point, but its hallucination metric is 0.173, the worst of any system. CAEM is the lowest on the hallucination axis of every system measured. At its best cycle the accuracy reads 0.544, ahead of all eight. The honest verdict is therefore precise: accuracy at parity with the strongest baseline, and a decisive, statistically significant advantage on confident wrongness, which is the quantity the architecture was built to reduce.

Decomposing the gain

A single CAEM-versus-floor number bundles two very different contributions: what the architecture buys before any training, and what the self-improvement loop adds on top. The book separates them with a three-way decomposition, because they answer different questions.

Architecture, self-improvement, and the honesty story

The decomposition reads off three contrasts. The architecture contrast (the zero-shot floor against cold-start CAEM, before any training) lifts pooled exact match by about three and a half points on the full panel and nearly nine points on the



Figure 14.4: The actual three-way decomposition of the exact-match gain over the zero-shot floor. "Architecture" is the cold-start contrast (three tiers, verifier, and routing, before any fine-tuning); "self-improvement" is the slice the cyclic loop adds; "full" is the two combined. On the five-benchmark panel almost all the accuracy gain is architectural and the self-improvement slice is roughly flat, because the loop's value is on the honesty axis instead, which Figure 14.5 decomposes on the same three contrasts. On the three training benchmarks the architectural gain is much larger (about nine points).

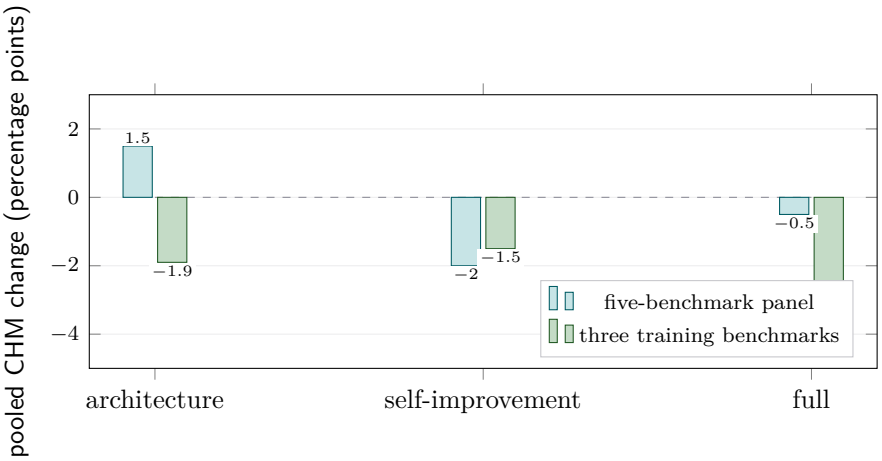


Figure 14.5: The actual three-way decomposition of the composite hallucination metric over the zero-shot floor, the honesty companion to Figure 14.4; **lower is better, so a bar below the dashed line is an improvement**. The contrasts are the same: architecture is the cold-start step, then the self-improvement slice, then the two combined. The reading is the mirror image of the accuracy figure. On the full five-benchmark panel the architecture step *raises* the hallucination metric slightly (a bar above the line), because cold-start retrieval amplifies the adversarial misconception benchmark, and the self-improvement step is what pulls it back down. On the three training benchmarks both steps lower it and the combined reduction is about three and a half points (close to a quarter relative). Where the accuracy figure showed the architecture doing the work and the loop flat, this figure shows the loop doing the honesty work.

training benchmarks: the three tiers, the verifier, and the router account for most of the headline accuracy gain on their own. The self-improvement contrast (cold-start CAEM against the third cycle, architecture held fixed) is roughly flat on exact match. This is not a disappointment, it is the expected result, because the loop's job is honesty, not raw accuracy, and Figure 14.5 shows this directly. On the full panel the architecture step actually *raises* the pooled hallucination metric slightly, from 0.120 to 0.135, because cold-start retrieval amplifies the adversarial misconception benchmark; the self-improvement step is what pulls it back, from 0.135 to 0.115, about fifteen percent. On the three training benchmarks both steps reduce it, and the combined drop is about a quarter. The loop folds retrieval-found knowledge into the model and trims confident wrongness; it was never going to move strict exact match much, and the metric suite is designed to show that distinction rather than hide it.

If an examiner asks: "so the loop barely helps accuracy?"

Correct, and the book says so plainly rather than burying it. The self-improvement loop is close to flat on pooled exact match, and that is the honest reading of the decomposition. Its contribution is on the honesty axis: about a fifteen percent reduction in the hallucination metric across the same cycles on the full panel, close to a quarter on the training benchmarks, a growing share of queries answered cheaply from the zero-shot tier as knowledge is distilled into the weights, and a memory that survives every retention-guarded rollback. Reporting a flat accuracy slice next to a real honesty gain is exactly the separation of axes this chapter argued for. A metric suite that reported only accuracy would have called the loop worthless; the suite the book uses shows what it actually bought.

With the instruments defined, we can now read what they measured. Chapter 15 walks the realised six-cycle trajectory benchmark by benchmark and cycle by cycle, and Chapter 16 removes one architectural piece at a time to price each component against the panel and metrics established here.

CHAPTER 15

The Realised Trajectory: Cycles 0 to 5

Chapter 14 defined the instruments. This chapter reads them across the run the system actually performed: six cycles, numbered zero through five, on the five-benchmark panel. Everything here is measured, not designed. Where the earlier chapters said what *should* happen, this one says what *did*, and it does not hide the places where the two diverge.

What a cycle is, and what we are watching

Cycle zero is the cold start: the full architecture is in place, but the model has not yet been fine-tuned on its own answers. Each later cycle fine-tunes on the verified episodes accumulated since the last boundary, checks retention, and re-scores memory. We watch five things across the six cycles: pooled accuracy and honesty, the per-benchmark spread, how the four-outcome gate shifts, how work moves between the three tiers, and what the retention guard does when an update goes wrong. The single most important habit to keep is the one Chapter 14 argued for: read accuracy and honesty on separate axes, because the loop moves them differently.

15.1 The shape of the run

The run committed three cycles, aborted one, and committed a final one. That sentence is the spine of the whole chapter.

Cycle	Pooled EM	Pooled CHM	Worst probe	Floor	Outcome
C0 (cold start)	0.541	0.135	—	—	baseline
C1	0.543	0.112	1.006	0.93	commit
C2 (best)	0.544	0.107	0.949	0.93	commit
C3	0.532	0.115	0.976	0.93	commit
C4	0.532	0.115	**0.886**	0.93	**abort**
C5 (final)	0.517	0.109	0.958	0.93	commit

Table 15.1: The realised trajectory. Pooled exact match and the composite hallucination metric are unweighted means over the five panel benchmarks (the misconception benchmark scored by the language-model judge). The *worst probe* column is the lowest of the three retention ratios at that cycle; the cycle aborts when it falls below the floor $\rho_{\min} = 0.93$, which happened at C4. C2 is the within-trajectory best cycle, C5 the final one.

Read the two axes separately

On the accuracy axis the run is almost flat: pooled exact match stays between 0.517 and 0.544 across all six cycles, peaking at C2. On the honesty axis the run does real work: the hallucination metric falls from 0.135 at the cold start to 0.107 at C2 and

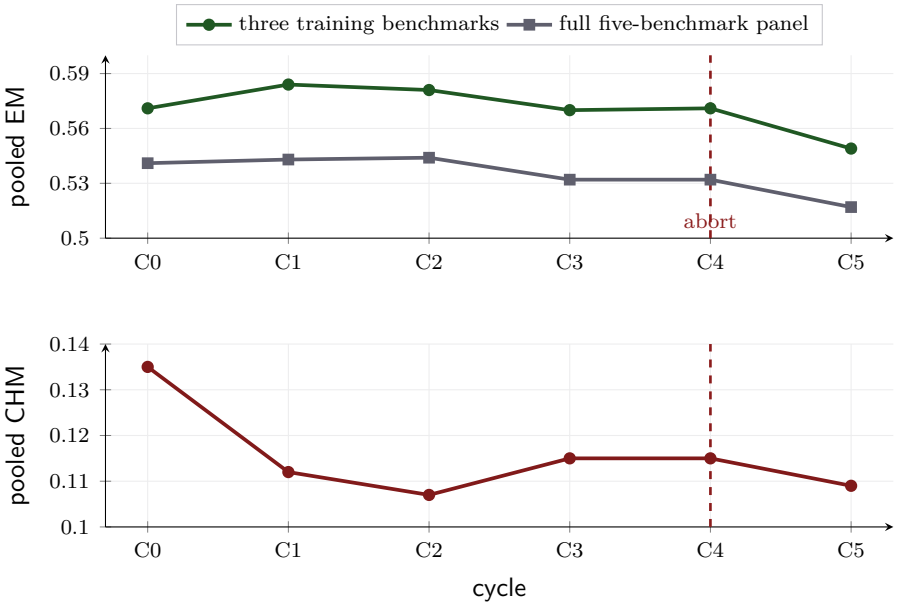


Figure 15.1: The actual deployed trajectory. Top: pooled exact match holds in a narrow band (full panel 0.517 to 0.544), with the training benchmarks running a few points higher. Bottom: the hallucination metric drops sharply from the cold start to the first committed cycle and then holds low. The dashed line marks C4, where the retention guard aborted; its eval ran on the rolled-back C3 model, which is why C3 and C4 read almost identically.

holds near 0.11 thereafter, a reduction of roughly a fifth that the accuracy trace alone would never reveal. This is the single most important reading of the whole trajectory: the self-improvement loop is a honesty engine, not an accuracy engine, and a chapter that reported only exact match would have called it a failure.

15.2 Accuracy, benchmark by benchmark

The flat pooled accuracy hides a real split, and the split is exactly along the training-versus-transfer line that Chapter 14 drew.

Four steady, one declining

Against the zero-shot floor of Chapter 14, four of the five benchmarks improve or hold. The open-text factoid benchmark rises from a floor of 0.293 to a trajectory band around 0.46 to 0.49; the fact-checking benchmark rises from 0.453 into the 0.48 to 0.53 band; the commonsense benchmark holds high, in the 0.72 to 0.76 band, a little above its 0.703 floor; the implicit multi-step benchmark holds around 0.64. The single exception is the misconception benchmark, which falls every cycle, from 0.380 to 0.300, ending below its own 0.433 zero-shot floor. That one benchmark is the entire reason the pooled five-benchmark accuracy looks flat while the three training benchmarks sit a few points higher. The honest move is not to drop the benchmark but to read its decline with the right metric, which the chapter does in Section 15.6.

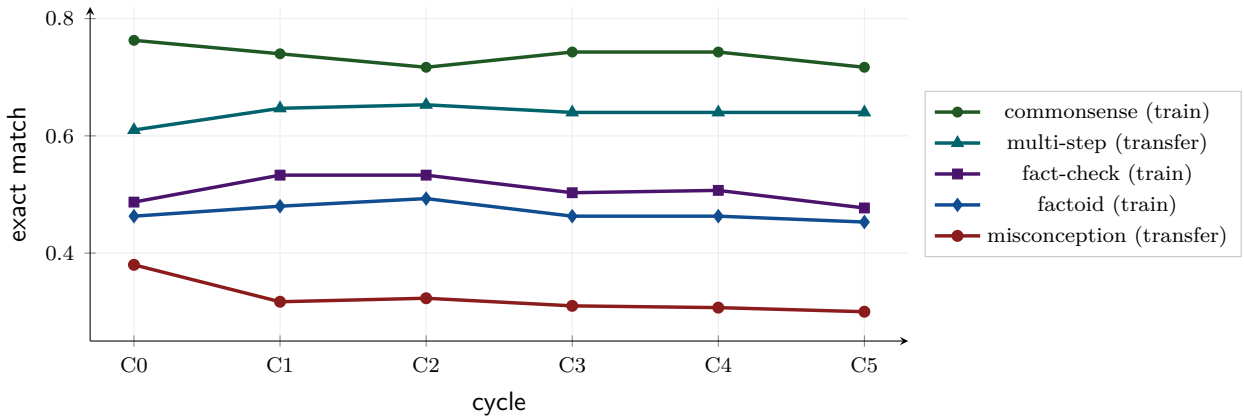


Figure 15.2: The actual per-benchmark exact match across the six cycles (misconception benchmark scored by the language-model judge). Four of the five benchmarks hold steady or rise; the misconception benchmark falls monotonically from 0.380 to 0.300, which is the one trace that drags the pooled average down. Section 15.6 shows why that fall is honesty, not failure.

15.3 How the gate shifts

The four-outcome gate of Chapter 10 is not static. As the model improves, the mix of store, defer, discard, and abstain moves, and the direction of the movement is itself a measurement of the system's growing confidence.

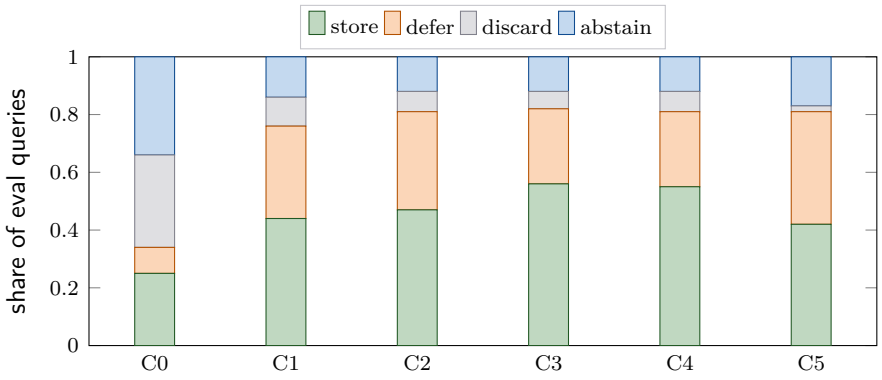


Figure 15.3: The actual decision distribution across the six cycles, pooled over the five-benchmark fold. At the cold start the gate is cautious: a third of queries are discarded and a third abstained. By the third cycle the store and defer shares dominate (store peaks near 0.56) while discard collapses from 0.32 to about 0.06, the signature of a model the gate increasingly trusts.

From cautious to decisive, and the per-outcome ordering

At the cold start the gate parks or refuses two thirds of its answers: store 0.25, defer 0.09, discard 0.32, abstain 0.34. By the third committed cycle it has turned decisive: store 0.56, defer 0.26, discard 0.06, abstain 0.12. Two movements drive this. The discard share collapses from about a third to a sixteenth as the model produces fewer answers that are low-confidence yet grounded enough not to abstain; and the store-plus-defer

mass grows as more answers clear the trust thresholds. The per-outcome accuracy stays strictly ordered at every cycle, stored answers most accurate and abstentions least, which is exactly the ordering a calibrated gate must produce. The final cycle relaxes slightly, store 0.42 and defer 0.39, as the post-abort model becomes a little less certain, a movement the next sections explain.

15.4 The three tiers in motion

The clearest evidence that the loop is doing something real is not in the pooled accuracy but in how the three tiers perform and how work moves between them.

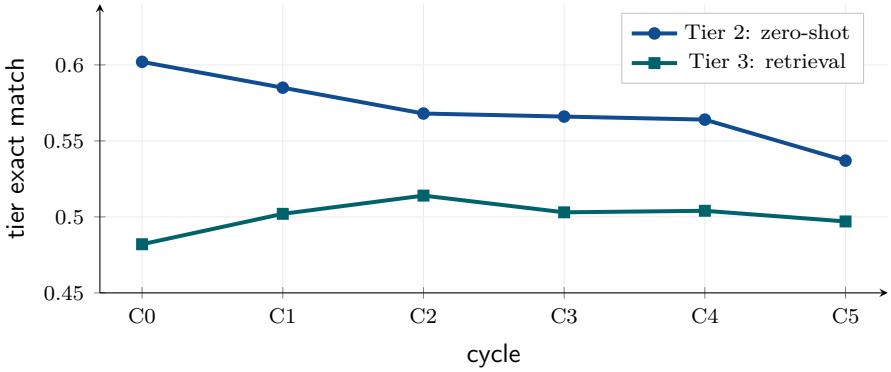


Figure 15.4: The actual per-tier exact match across the run. The cheap zero-shot tier (Tier 2) answers its assigned queries *more* accurately than the expensive retrieval tier (Tier 3) at every single cycle, opening at a twelve-point gap (0.602 against 0.482) at the cold start. The gap narrows as the loop runs but never reverses, which is the inference-time signature of self-improvement folding retrieval-found knowledge into the model's own parameters. The memory-direct tier (Tier 1) is omitted here: it fires on about one query in a thousand and returned the correct stored answer every time it did.

The selection effect, stated honestly

The per-tier reading must be stated with care, because it is partly a selection effect. The router sends a query to the zero-shot tier precisely because it judges the query answerable from parameters, so that tier's queries are easier on average. The honest claim is therefore the conditional one: *the zero-shot tier answers the queries it is assigned at least as accurately as the retrieval tier answers the queries it is assigned*, at every cycle, while costing a fraction as much. It is not the claim that retrieval is worse in general. Even stated conditionally the receipt is strong: a tier that needs no retrieval matching a tier that does is exactly what a system that has distilled its retrieved knowledge into its weights should show. The memory-direct tier fires on roughly one query in a thousand because the evaluation fold is content-hash disjoint from everything the system trained or stored on, so near-verbatim repeats are rare by construction; when it does fire it is a precision-first cache, correct on every realised firing.

15.5 Retention, and the cycle that was thrown away

Three cycles committed cleanly. The fourth did not, and what the system did about it is the safety property the whole architecture exists to guarantee.

The cycle-four abort

At the fourth cycle boundary the retention guard ran its three probes against their cold-start anchors. The general-knowledge probe came back at 1.018, slightly improved; the commonsense probe at 0.982; but the held-out open-text factoid probe fell to 0.886, below the floor $\rho_{\min} = 0.93$. Because the guard aborts on the worst probe, the cycle was thrown away: the freshly fine-tuned adapter was discarded and the model rolled back to its third-cycle weights. This is why C3 and C4 read almost identically in Table 15.1: the C4 evaluation was run on the restored C3 model. The fifth cycle then re-attempted training from that last good adapter and committed. The retention figure of Chapter 12 draws this firing.

Why the memory did not roll back with the weights

The abort reverts the weights but *not* the memory. Every episode admitted during the fourth cycle survived, and the store kept growing across the boundary. This asymmetry is deliberate and is the core safety property: memory growth is monotone and high-precision by construction, since each episode passed the gate, so it is always safe to keep, whereas a weight update is a global change with no such guarantee, so it is the thing thrown away. A rollback therefore costs a cycle's training but never a cycle's remembered knowledge. The reconsideration of deferred episodes is one of the steps gated on committing, so it is skipped on an abort, which is correct: re-scoring through unchanged weights would be a no-op.

15.6 The honesty lens on the misconception benchmark

The one benchmark whose accuracy fell every cycle is also the one that proves the metric suite of Chapter 14 was necessary. Read with exact match alone, it looks like a regression. Read with the honesty axis, it is the opposite.

Falling accuracy, rising honesty

On the misconception benchmark the judged exact match falls from 0.380 at the cold start to 0.300 at the final cycle. Over the same cycles its abstention rate stays high, in the band of roughly a quarter to a third of all questions (one hundred of three hundred at the cold start, sixty-nine at the end), where the zero-shot floor abstains on none. And its hallucination metric *falls*, from about 0.150 to about 0.104. The three numbers together tell one story: the declining exact match tracks an increase in honest abstention, not a loss of knowledge. The benchmark is built so the fluent answer is a common misconception, and a dense retrieval substrate tends to amplify that framing rather than refute it, so the system increasingly declines rather than

commit to a confident falsehood. A bare model is not licensed to abstain, pays no abstention penalty, and scores a higher strict exact match by stating the misconception outright. Exact match rewards that; the honesty axis does not.

If an examiner asks: "isn't a falling benchmark just a worse system?"

Not on this benchmark, and the separation of axes is what lets us say so without hand-waving. The misconception benchmark is adversarial by design: its questions are written so the most fluent completion is false. A system that answers all of them scores a higher exact match precisely by asserting falsehoods confidently, which is the behaviour the whole project is built to suppress. CAEM instead abstains on a quarter to a third of them, so its exact match falls while its confident-wrongness metric on the same benchmark falls too. The architectural remedy for the lost accuracy is the retrieval-utility classifier of Chapter 7, which routes these queries away from the amplifying retrieval tier; the trajectory here is the un-routed behaviour the classifier was built to correct, reported honestly rather than quietly dropped.

15.7 Inside the loop, and when it stopped

Underneath the eval numbers the loop itself left a clean record: a training pool that grew, a training loss that fell, a memory that accumulated, and a principled reason it stopped at five rather than running to its configured ten.

Pool, loss, memory, and the stop

The per-cycle training pool grew as memory accumulated: roughly one hundred episodes at the first cycle, then about fifteen hundred, three thousand, and five thousand by the fifth. The final-epoch training loss fell across the committed cycles, from about 0.26 to about 0.11, with the aborted fourth cycle reaching the lowest loss of the run, about 0.07, the textbook signature of an over-fit update that the retention guard correctly caught (Chapter 12 draws this). The episodic store grew monotonically across every boundary, including the aborted one, from a few hundred episodes to roughly ten thousand by the final cycle, and the per-cycle re-verification pruned a declining tail, about thirteen percent at the first committed cycle down to about four percent at the last, as the pool's composition shifted toward higher-confidence survivors. The loop stopped at the fifth cycle, not the configured tenth, because two of its three early-stop signals fired together: the storage rate had saturated and general-knowledge retention was holding, which is enough to halt at the first eligible cycle after the burn-in.

Why stopping early is a feature, not a shortfall

A self-improvement loop that ran forever would eventually overfit its own outputs, and the fourth cycle is the warning shot: lowest training loss, failed retention. The early-stop gate reads the same evidence and halts while the system is still healthy, rather than grinding through five more cycles of diminishing returns and rising forgetting risk.

The realised five cycles are not a truncated ten-cycle plan; they are the loop recognising that it had extracted the gains its own answers could supply and stopping before it started to harm itself.

15.8 What the theory predicted, and what the run did

Chapter 13 proved properties about the memory pool; the trajectory is where those properties either held or did not. They held, with one honest exception.

The theory met the run

The data-purity floor held: the calibration-fold pool purity stayed in the band 0.72 to 0.75 across every committed cycle, clearing the registered floor near 0.64 by eight to eleven points. The self-correction prediction held: the per-cycle prune rate declined monotonically, from about thirteen percent to about four percent, as wrong episodes were caught and removed faster than they accumulated. The Tier-1 floor held: the memory-direct tier returned the correct answer on every one of its seven firings across nine thousand evaluation queries. The convergence prediction held in the weak sense the theory allowed: four of five committed transitions kept retention above the floor and the fifth was caught and reverted. The one property the run only partially supported is monotone improvement, because the composite is re-fitted each cycle and so violates the fixed-function premise the proof assumes; the chapter reports this as a partial receipt rather than a clean confirmation, exactly as Chapter 13 flagged.

The trajectory is the architecture's behaviour with every piece switched on. To price each piece, Chapter 16 switches them off one at a time and measures what the panel and the metrics of Chapter 14 report when a component is removed.

CHAPTER 16

Ablations: What Each Piece Is Worth

Chapter 15 showed the architecture with every piece switched on. This chapter asks the harder question: what is each piece actually worth? The honest answer is built from the contrasts the run already supplies, one component-level switch that was genuinely toggled, and a clear ledger of which lesions remain to be run. We do not pretend to a brute-force sweep that was not performed, and the reason that is defensible is the subject of the first section.

Ablation by measurement, not by brute force

The textbook way to price a component is to remove it, re-run everything, and measure the drop. That is expensive: a system with a dozen parts would need a dozen full re-runs. CAEM takes a different route wherever it can. Many components leave a *direct trace* in the trajectory already measured: the value of the router is the per-tier accuracy and tier shares of Chapter 15; the value of self-improvement is the cold-start-versus-trained contrast; the value of the forgetting guard is what it did when it fired. Where a component's worth is already visible in a number we have, re-running a lesion to re-measure it would be redundant. Where it is not, a lesion is warranted. This chapter prices each piece by the cheapest sound evidence available, and is explicit about the one lesion whose clean counterfactual is still owed.

16.1 The ablation registry, and the claim it defends

CAEM ships a small, deliberate ablation registry rather than a large sweep. The registry pairs each lesion with the specific thesis claim it is meant to defend, and prunes any lesion whose claim is already settled by a direct metric.

Four entries, each tied to a claim

The registry holds a full-system reference and three lesions. The reference, named *full*, applies no mutation and is the anchor every delta is measured against. The three lesions are: *no-self-improvement*, which runs the full pipeline with no fine-tuning so the system stays at its cold-start weights; *no-forgetting-guard*, which sets the retention tolerance to zero so every update commits regardless of how much it forgets; and *no-retroverify*, which skips the per-cycle re-scoring of stored memory. Each defends a numbered claim: no-self-improvement and no-retroverify defend the self-improvement claim, no-retroverify additionally defends the memory-purity claim, and no-forgetting-guard defends the no-catastrophic-forgetting claim. The memory-routing claim is defended without a lesion at all, by the per-tier shares and accuracies already in Chapter 15.

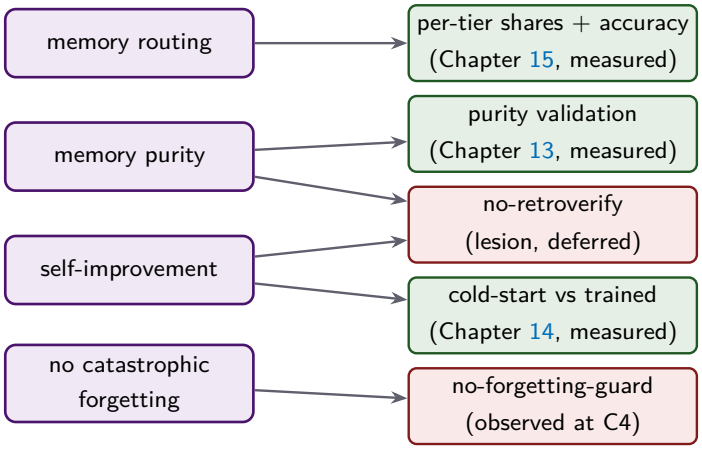


Figure 16.1: The ablation registry maps each thesis claim to its cheapest sound evidence. Three claims are settled by a metric already in hand (green): routing by the per-tier trajectory, purity by the validation measurement, self-improvement by the cold-start-versus-trained contrast. Two lean on a lesion (red): no-forgetting-guard, observed naturally when the guard fired at the fourth cycle, and no-retroverify, whose clean counterfactual is registered but not yet run. (Schematic.)

Why the sweep is small

An earlier design listed more than twenty candidate lesions. They were pruned against two questions: is the claim already defended by a direct metric, and is the evidence redundant with a lesion already kept? A lesion that removed the storage gate was dropped because the purity validation measures the gate's effect directly; a lesion that removed the cheap memory tier was dropped because the per-tier share trajectory already shows its usage; lesions on individual verifier signals were dropped because the signal correlation matrix shows each carries non-redundant information. What survived is the minimal set whose claims no existing number answers. This is a deliberate choice to spend evidence where it is missing, not to manufacture re-runs where it already exists.

16.2 Ablating the whole architecture, and the self-improvement slice

The largest ablation is the one already drawn in Chapter 14: the eight-baseline panel is the architecture removed in pieces, and the decomposition is self-improvement removed on its own.

The panel is a component-removal ladder

Read as ablations, the baselines of Chapter 14 price the architecture from the bottom up. The zero-shot floor is everything removed: no routing, no retrieval, no memory, no verifier, no fine-tuning. Unconditional retrieval adds back grounding and nothing else. Vanilla fine-tuning adds back training-time learning and nothing else. The gap from the floor to full CAEM is therefore the joint worth of routing, verification, memory,

and self-improvement; and the gap from vanilla fine-tuning to full CAEM isolates the deployment-time machinery from the training-time machinery. On the pooled panel the floor-to-cold-start step, which adds the three tiers, the verifier, and the router before any training, is worth about three and a half points of exact match (nearly nine on the training benchmarks), as Figure 14.4 showed.

The no-self-improvement lesion is the cold-start cycle

One of the three registered lesions needs no separate run, because it already exists in the trajectory. The no-self-improvement lesion is defined as the full pipeline with fine-tuning switched off, which is exactly the cold-start cycle of Chapter 15. Pricing self-improvement is therefore the cold-start-versus-third-cycle contrast, and its reading is the one Chapter 14 and Chapter 15 already established: the loop is close to flat on pooled exact match and worth about a fifteen percent reduction in the hallucination metric, a quarter on the training benchmarks. The lesion confirms what the metric suite was built to surface: self-improvement buys honesty, not raw accuracy, and removing it costs honesty rather than accuracy.

16.3 The retrieval-utility classifier, switched off

One component was toggled cleanly on and off under a matched protocol: the retrieval-utility classifier of Chapter 7. It is the chapter's one true component-level ablation, and it is also the one with the most to teach, because the result is asymmetric.

Benchmark	EM off → on	CHM off → on (% rel)	retrieval share off → on
fact-checking	0.503 → 0.503 (+0.0)	0.117 → 0.120 (+2.5)	92.3% → 70.7%
open-text factoid	0.437 → 0.463 (+2.7)	0.123 → 0.121 (−2.0)	95.7% → 68.0%
commonsense	0.603 → **0.743** (**+14.0**)	0.131 → **0.086** (**−34.4**)	57.7% → 20.3%
implicit multi-step	0.617 → 0.640 (+2.3)	0.119 → 0.128 (+8.1)	100% → 76.7%
misconception	0.210 → **0.310** (**+10.0**)	0.116 → 0.119 (+2.9)	99.0% → **40.0**%
pooled	**0.474** → **0.532** (**+5.8**)	**0.121** → **0.115** (**−5.2**)	**88.9**% → **55.1**%

Table 16.1: The actual on-versus-off contrast for the retrieval-utility classifier at the third cycle, matched backbone and evaluation fold (misconception exact match by the language-model judge). With the classifier off, every query that fails the cheap-path gates routes unconditionally to retrieval; with it on, those queries route to retrieval only when the classifier predicts retrieval will help. The asymmetric-rescue pattern is in the bold cells.

Asymmetric rescue, and why it is the right shape

The classifier lifts pooled exact match by 5.8 points and lowers the pooled hallucination metric by about five percent, but the pooled numbers understate the mechanism. The gains are concentrated where retrieval was hurting: the commonsense benchmark gains fourteen points of exact match and loses a third of its hallucination metric, and the misconception benchmark gains ten points, while the fact-checking benchmark, where retrieval genuinely helps, is held flat. The retrieval share collapses from eighty-nine

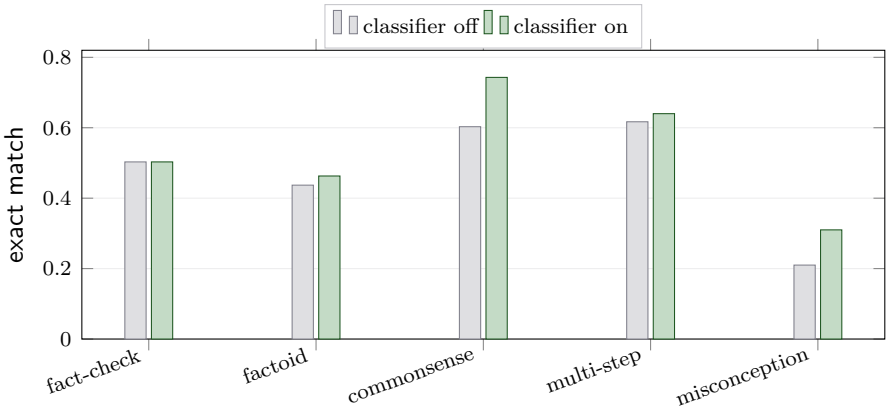


Figure 16.2: The actual per-benchmark exact match with the retrieval-utility classifier off and on. The gains are concentrated, not uniform: the two benchmarks where unconditional retrieval was hurting (commonsense +14.0 points, misconception +10.0 points) are rescued, while the fact-checking benchmark, where retrieval helps, is held exactly flat. The classifier learns where grounding earns its cost and routes accordingly, collapsing the pooled retrieval share from 88.9% to 55.1%.

percent to fifty-five percent pooled, and by fifty-nine points on the misconception benchmark specifically. This is the right shape because it shows the classifier is not simply turning retrieval down everywhere; it is learning the per-benchmark sign of retrieval's value and acting on it, preserving grounding where it pays and withdrawing it where it amplifies the wrong answer.

If an examiner asks: "is this a clean classifier-only ablation?"

No, and the chapter says so in the same breath as the result. The on-arm shipped the classifier alongside concurrent pipeline corrections and a re-fitted composite, so the contrast is a *bundled* on-versus-off comparison, not a single-variable isolation. The honest claim is that the bundle, with the classifier as its load-bearing addition, produces the asymmetric rescue; the arm-isolating re-run that would attribute the gain to the classifier alone is registered as future work. Reporting the confound next to the number is the difference between an ablation and a result dressed up as one. The mechanism, routing retrieval away from the benchmarks it hurts, is exactly what the classifier was designed to do, which is why the bundled gain is attributed to it even without the clean isolation.

16.4 The forgetting guard, ablated by the run itself

The no-forgetting-guard lesion would set the retention tolerance to zero and let every update commit. It was not run as a separate sweep, but the run performed the experiment for us, once, at the fourth cycle.

The fourth cycle is the guard's natural ablation

At the fourth cycle the freshly fine-tuned model reached the lowest training loss of the entire run, about 0.07, yet its held-out open-text factoid probe fell to 0.886, below the floor. The guard aborted the cycle and rolled the weights back. With the forgetting guard ablated, that same update would have committed: the model with the best training loss and a failed retention probe would have become the next cycle's starting point. The guard is therefore worth exactly the degradation it refused to accept: an update that looked best by training loss and worst by held-out retention. This is the cleanest possible illustration of why training loss is not the quantity to trust, and why the guard reads a held-out probe instead. The clean lesion, which would let the loop run several cycles without the guard and measure the cumulative forgetting, would change which cycles commit and so needs its own run; it is registered but not yet performed. What the run shows is the guard catching a real degrading update on the one occasion it appeared.

16.5 The lesion still owed, and the honest ledger

One registered lesion has neither a direct metric nor a natural observation, and the chapter does not pretend otherwise.

No-retroverify was not run

The no-retroverify lesion skips the per-cycle re-scoring of stored memory. Unlike no-self-improvement, it cannot be read off an existing cycle, because its effect is a *divergence* that compounds over cycles: without re-verification the stored pool would slowly accumulate episodes that a better model would no longer trust, and only a parallel run under the lesion would measure how far the purity drifts. That run was not performed. What the trajectory does show is the work re-verification did when it ran: it pruned a declining fraction of the pool each cycle, about thirteen percent at the first committed cycle down to about four percent at the last, which is the count of episodes the re-scoring removed. That is evidence the mechanism is active and that its workload shrinks as the pool's quality rises, but it is not the counterfactual purity-without-it. The clean lesion is registered future work, and the book reports it as owed rather than quietly folding the prune-rate trace into a claim it cannot support.

The ledger, read honestly

Pulling the pieces together: the three-tier architecture earns the accuracy (about three and a half pooled points over the floor, nearly nine on the training benchmarks); the self-improvement loop earns the honesty (roughly a fifteen percent reduction in confident wrongness while accuracy stays flat); the retrieval-utility classifier earns a further five-and-a-half points of accuracy by routing retrieval only where it helps, though that figure is bundled with concurrent fixes; the forgetting guard earns its keep on the one cycle it caught a degrading update; and re-verification is demonstrably active

Component	**Evidence**	**What it is worth**
Three-tier architecture	baseline panel, cold start	+3.4 pp pooled EM (+8.8 training); routing earns the accuracy
Self-improvement loop	cold-start vs trained	flat EM, about −15% CHM (honesty, not accuracy)
Retrieval-utility classifier	on/off, bundled	+5.8 pp EM, −5.2% CHM, retrieval share −33.8 pp
Forgetting guard	cycle-four abort	caught a lowest-loss, failed-retention update
Retroverification	prune-rate trace (counterfactual owed)	removed 13%→4% of the pool per cycle

Table 16.2: What each piece is worth, by the cheapest sound evidence available. Three components are priced by a metric already in hand, one by a clean on/off toggle (bundled), one by a natural observation, and one by an active-but-not-yet -counterfactually-isolated trace. The exact-match and hallucination deltas are pooled over the five-benchmark panel.

but its clean counterfactual is owed. No single number captures the system, which is the whole reason the metric suite of Chapter 14 reports several. The architecture and the loop divide the labour between them: one makes the system accurate, the other makes it honest, and the components around them keep both properties from eroding.

This closes the evaluation part. The architecture has been built stage by stage, proven where proof was possible, measured across a real trajectory, and priced component by component. Chapter A collects every registered constant the book has used into one registry, and Chapter B gathers the symbols and terms, so that any number or name in the preceding chapters can be traced to its definition.

PART V

Reference

APPENDIX A

The Registered-Constant Registry

Every number this book relied on is collected here, grouped by the component it governs, with the chapter that introduces it. The registry has one purpose: to make the architecture auditable. A constant that appears in a results chapter should be traceable to a single registered value, and that value should be the one in the live configuration, not a number invented for exposition. The tables below are read directly from the deployed configuration; where a symbol was used in the text it is repeated here beside its value.

Why constants are registered, not tuned per result

A system with dozens of thresholds invites a quiet form of overfitting: nudge a cut-point until the headline improves, then report the headline. CAEM resists this by *registering* its constants before the trajectory runs and holding them fixed across every cycle and every benchmark. The two exceptions, the calibration that is re-fitted each cycle and the per-benchmark calibration curves, are themselves learned under a fixed procedure rather than hand-tuned. This appendix is the ledger that makes that discipline checkable: one value per constant, fixed for the run.

A.1 Backbone and generation

Constant	Value	Where
Generator backbone (frozen)	Qwen-2.5-3B-Instruct	Chapter 3
Decoding	greedy (no sampling)	Chapter 3
Maximum new tokens (both paths)	512	Chapter 8
Pre-routing decode length	32 tokens	Chapter 4

Table A.1: Backbone and generation constants.

A.2 Episodic memory

Constant	Value	Where
Embedding model	all-mpnet-base-v2	Chapter 3
Embedding dimension	768, unit-normalised	Chapter 3
Memory capacity	1,000,000 episodes	Chapter 5
Capacity-prune trigger	0.95 full	Chapter 5
Novelty gate	cosine 0.95	Chapter 5
Consolidation clustering threshold	cosine 0.92	Chapter 5
Consolidation maximum trust spread	0.15	Chapter 5
Value weighting (importance/recency)	0.70/0.30	Chapter 5
Importance sub-weights	0.40/0.30/0.20/0.10	Chapter 5
Recency decay λ	0.01	Chapter 5
Retroverification prune floor	0.50	Chapter 5

Table A.2: Episodic-memory constants. The novelty gate (0.95) and the consolidation clustering threshold (0.92) are distinct and must not be conflated.

A.3 Retrieval and routing

Constant	Value	Where
Passage corpus	$\approx$ 21M passages, 64 GB	Chapter 8
Index structure	IVF-PQ, 32768 cells	Chapter 3
Probe (grounded tier / cheap lookup)	64/16 cells	Chapter 8
Retrieved passages	top 5	Chapter 8
Maximum retrieved context	384 tokens	Chapter 8
Pre-routing confidence blend	0.60 token +0.40 internal	Chapter 4
Calibration temperature range	$[e^{-3}, e^{3}] = [0.05, 20.1]$	Chapter 4
Safety floor ϕ_{safe}	0.38	Chapter 4
Routing blend λ	0.70	Chapter 6
Tier-1 combined threshold	0.90	Chapter 6
Tier-2 similarity threshold	0.75	Chapter 6

Table A.3: Retrieval and routing constants.

A.4 Retrieval-utility classifier

Constant	Value	Where
Routing threshold τ_{RUC}	0.375	Chapter 7
Regularisation strength	$C = 0.5$	Chapter 7
Cross-validation	nested 5×5	Chapter 7
Self-knowledge probe coefficient	-1.77 (standardised)	Chapter 7

Table A.4: Retrieval-utility classifier constants.

A.5 The nine-signal verifier

Constant	Value	Where
Self-consistency chains M	3 at temperature 0.7	Chapter 9
Dropout passes K	5 at rate 0.1	Chapter 9
Internal confidence	$0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$	Chapter 9
Grounding retrieve / rerank	$20 \to 3$ passages	Chapter 9
Atomic-decomposition minimum length	12 tokens	Chapter 9
Entailment judge split	408 tokens (512−4−100)	Chapter 9
Confabulation early-exit	$u_{\text{internal}} \geq 0.70 \wedge p_{\text{ground,max}} \leq 0.20$	Chapter 9

Table A.5: Verifier constants.

A.6 Composite and four-outcome gate

Constant	Value	Where
Calibrated signals	9	Chapter 10
Per-signal isotonic clip	$[0.02, 0.98]$	Chapter 10
Minimum samples per signal	50	Chapter 10
Shrinkage toward pooled α	0.6	Chapter 10
Boost regularisation	$C = 0.01$	Chapter 10
Storage threshold τ_{store}	0.60	Chapter 10
Deferral threshold τ_{defer}	0.45	Chapter 10
Abstain grounding ceiling	$p_{\text{ground,max}} < 0.20$	Chapter 10

Table A.6: Composite and gate constants.

A.7 Self-improvement and retention

Constant	Value	Where
Adapter rank r / scale α	32/64	Chapter 12
Adapter dropout	0.05	Chapter 12
Adapted projections	7 (attention + feed-forward)	Chapter 12
Trainable fraction	$\approx 2\%$ of parameters	Chapter 12
Learning rate	2×10^{-4}	Chapter 12
Epochs per cycle	3	Chapter 12
Micro-batch / effective batch	4/16	Chapter 12
Training trust cut	$u_{\text{stored}} \geq 0.70$	Chapter 12
Pool reweighting temperature	2.0	Chapter 12
Pool reweighting cap / floor	$3 \times$ /50	Chapter 12
Pool cold-start seed / max share	100/0.40	Chapter 12
Weight-anchor strength (fallback)	$\lambda = 0.01$	Chapter 12
Retention floor $\rho_{\min}$	0.93	Chapter 12
Retention probes	3×200 questions	Chapter 14
Early-stop rule / burn-in	2 of 3 signals / 5 cycles	Chapter 12
Cycles configured / realised	10/6 (C0–C5)	Chapter 15

Table A.7: Self-improvement loop and retention-guard constants.

A.8 Benchmarks, evaluation, and metrics

Constant	Value	Where
Evaluation panel	5 benchmarks (3 train +2 transfer)	Chapter 14
Evaluation fold	300/benchmark (1500 total)	Chapter 14
Calibration fold	500/training benchmark (1500)	Chapter 14
Content key	SHA-256, first 16 hex chars	Chapter 14
Semantic-entropy baseline samples	10 at temperature 1.0	Chapter 14
Hallucination metric	mean of 8 measured subtypes	Chapter 14
Subtype firing thresholds	0.50/0.30/0.30/0.50/600 chars	Chapter 14
Evaluation score	geometric mean of 5 axes ($\varepsilon = 0.01$)	Chapter 14
Significance (accuracy / honesty)	McNemar / Wilcoxon, Holm-corrected	Chapter 14
Claim-licensing rule	Holm $p < 0.05$ and interval above 0.02	Chapter 14

Table A.8: Benchmark, evaluation, and metric constants.

What is fixed, and what is fitted

Of everything tabulated above, exactly two things change during a run, and both change under a fixed procedure rather than by hand: the pre-routing temperature is re-fitted each cycle, and the per-signal calibration curves and their boost weights are re-fitted each committed cycle on the re-scored calibration fold. Every other value in this registry is locked before the trajectory begins and held constant across all six cycles and all five benchmarks. That separation, learned calibration over fixed structure, is what lets a single pair of gate thresholds mean the same thing on every benchmark, and it is the discipline this appendix exists to document.

APPENDIX B

Glossary of Symbols and Terms

This glossary collects the notation and vocabulary the book uses. The first section lists every mathematical symbol with its meaning and the chapter that introduces it; the second defines the architectural terms in plain language. Nothing here is new: it is a single place to look up a symbol or a name encountered anywhere in the preceding chapters.

B.1 Symbols

Symbol	Meaning	Where
Confidence and uncertainty		
u_{pre}	Pre-routing confidence: a cheap readiness score read before any tier runs.	Chapter 4
u_{token}	Token-level confidence: the geometric mean of the per-token probabilities of the generated answer.	Chapter 9
u_{dropout}	Dropout disagreement: instability of the answer across resampled forward passes.	Chapter 9
u_{internal}	Internal confidence: $0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$, the model's self-assessed certainty.	Chapter 9
c_{conv}	Encoder convergence ratio: how settled the late-layer representation is, a component of u_{pre}.	Chapter 4
s_{avg}	Self-consistency: semantic agreement across the M resampled reasoning chains.	Chapter 9
Grounding and relevance		
p_{entail}	Reasoning-to-answer entailment: whether the chain logically supports the answer.	Chapter 9
$p_{\text{ground,max}}$	Top-passage grounding: entailment of the single best supporting passage (the most forgiving reading).	Chapter 9
$p_{\text{ground,mean}}$	Mean grounding: average support across the retrieved passages.	Chapter 9
$p_{\text{ground,atomic}}$	Atomic grounding: support of the weakest atomic fact in the answer (the strictest reading).	Chapter 9
$q_{\text{a,rel}}$	Question-answer relevance: how well the answer addresses the question.	Chapter 9
Routing and the retrieval-utility classifier		
p_{IK}	Self-knowledge probe: the model's estimated probability that it already knows the answer.	Chapter 7
p_{RAG}	Predicted retrieval utility: the classifier's probability that retrieval will help this query.	Chapter 7
τ_{RUC}	Retrieval-utility threshold: route to retrieval only when $p_{\text{RAG}} \geq \tau_{\text{RUC}} = 0.375$.	Chapter 7
Trust score and the four-outcome gate		
u_{stored}	Calibrated composite trust score: the single honest probability the answer is correct.	Chapter 10
τ_{store}	Storage threshold: store when $u_{\text{stored}} \geq 0.60$.	Chapter 10
τ_{defer}	Deferral threshold: defer when $0.45 \leq u_{\text{stored}} < 0.60$.	Chapter 10
τ_{train}	Training trust cut: only episodes with $u_{\text{stored}} \geq 0.70$ train the model.	Chapter 12
τ_{retro}	Retroverification prune floor: drop a stored episode whose re-scored trust falls below 0.50.	Chapter 5
ϕ_{safe}	Safety floor: send a query to retrieval whenever $u_{\text{pre}} < 0.38$.	Chapter 4
Calibration, learning, and theory		
$\rho_{\min}$	Retention floor: abort a cycle if any probe ratio falls below 0.93.	Chapter 12
α	Shrinkage weight (0.6): the blend of a per-benchmark curve toward the pooled curve; also, in theory, verifier balanced accuracy.	Chapter 10
λ	A decay or penalty rate: recency decay in memory, the blend weight in routing, the anchor strength in training.	Chapter 5
M, K	Self-consistency chains ($M = 3$) and dropout passes ($K = 5$).	Chapter 9
π	Pool purity: the fraction of stored episodes that are actually correct.	Chapter 13
p	Base accuracy: the generator's correctness rate before the gate filters it.	Chapter 13
d	Cohen's d: a standardised effect size, used for verifier discrimination.	Chapter 13

Table B.1: Mathematical symbols used in the book.

B.2 Terms

CAEM
Confidence-Aware Episodic Memory with Self-Improvement: the three-tier, verifier-gated, cyclically self-improving architecture this book describes.

Abstain
The honest-refusal outcome of the gate: when an answer is both low-confidence and ungrounded, the system returns "I do not know" rather than guess.

Asymmetric rollback
The safety property that, when a cycle aborts, the model weights revert but the episodic memory is kept. Training can be undone; remembered knowledge is not.

Atomic-fact decomposition
Splitting an answer into its component factual claims so each can be checked for grounding separately; the weakest one drives $p_{\text{ground,atomic}}$.

Calibration
Replacing a raw score with a probability whose value means what it says, so that a stored score of 0.60 corresponds to a sixty-percent chance of being correct.

Calibration fold
A held-out, leakage-disjoint slice of labelled answers on which the composite and the pre-routing temperature are fitted; separate from the evaluation fold.

Cold start
Cycle zero: the full architecture running before any self-improvement fine-tuning.

Composite
The calibrated fusion of the verifier's nine signals into the single trust score u_{stored}, via per-signal isotonic curves, per-benchmark shrinkage, and a log-odds boost.

Composite evaluation score (CES)
A headline scalar: the geometric mean of five axes (accuracy, epistemic quality, retention, calibration, verifier reliability), so a collapse on any one axis drags the whole score down.

Composite hallucination metric (CHM)
The honesty axis: the equal-weighted mean of eight measured failure-mode rates, lower being better.

Confident confabulation
The costliest failure mode: a wrong answer the system was nonetheless confident enough to store.

Consolidation
The cycle-boundary merge of near-duplicate stored episodes onto a single highest-trust representative, guarded by answer agreement and a trust-spread cap.

Defer
The gate outcome for a promising but not-yet-trusted answer: it is parked in a buffer for reconsideration by a later, better model.

Discard
The gate outcome for a low-confidence answer that is dropped silently, distinct from an abstention because some grounding exists.

Early-exit
The confabulation gate inside the verifier: when internal confidence is high but grounding is near zero, the answer is discarded and regenerated.

Episodic memory
The growing store of verified question-answer episodes, embedded for similarity search and served by the cheap memory tier.

Exact match (EM)
The headline accuracy axis: a normalised answer either matches a gold answer or does not.

Four-outcome gate
The decision step that reads u_{stored} and commits to one of store, defer, abstain, or discard.

Isotonic regression
A monotone, piecewise-constant fit that turns a raw signal into a calibrated probability of correctness, learning its direction from the data.

Low-rank adaptation (LoRA)
The fine-tuning method: the backbone is frozen and only a small low-rank adapter trains, which bounds catastrophic forgetting.

Pool reweighting
The temperature-mixed rebalancing of the training pool each cycle so that no benchmark or answer type dominates the fine-tune.

Purity theorem
The result that a verifier only modestly better than chance lifts the stored pool's purity above the generator's base accuracy, so an imperfect verifier still yields a clean memory.

Retention guard
The check that runs three held-out probes after each fine-tune and aborts the cycle if the worst probe falls below $\rho_{\min}$.

Retrieval-utility classifier (RUC)
The learned gate that predicts whether retrieval will help a query and routes accordingly, withdrawing grounding where it amplifies the wrong answer.

Retroverification
The cycle-boundary re-scoring of every stored episode under the updated model, pruning any whose trust has fallen below τ_{retro}.

Router
The dispatcher that sends each query to the cheap memory tier, the zero-shot tier, or the retrieval-augmented tier, using similarity, stored trust, and u_{pre}.

Self-improvement loop
The outer cycle that fine-tunes the model on its own verified answers, re-checks retention, and re-scores memory at each boundary.

Store
The gate outcome for a trusted answer: it is admitted to episodic memory as a verified episode.

Tier 1 / Tier 2 / Tier 3
The three execution paths: a precision-first memory-direct cache, zero-shot generation, and retrieval-augmented generation.

Training versus transfer
The panel split: three benchmarks the loop trains on and two it never trains on, the latter measuring generalisation.

Two clocks
The architecture's two timescales: a fast per-query pipeline that only reads state, and a slow per-cycle loop that is the only thing allowed to change it.

This closes the book. The architecture has been motivated, mapped, built stage by stage, proven where proof was available, measured across a real six-cycle trajectory, priced component by component, and reduced to a registry of constants and a glossary of terms. Every name and number in the preceding chapters can now be traced to its definition here or to the live code and report the book was built from.

References

[1] Stephanie Lin, Jacob Hilton, and Owain Evans. "TruthfulQA: Measuring How Models Mimic Human Falsehoods". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3214–3252. DOI: 10.18653/v1/2022.acl-long.229.

[2] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 12076–12100. DOI: 10.18653/v1/2023.emnlp-main.741.

[3] Hussam Alkaissi and Samy I McFarlane. "Artificial Hallucinations in ChatGPT: Implications in Scientific Writing". In: *Cureus* 15.2 (2023). PMID: 36895261, e35179. DOI: 10.7759/cureus.35179.

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.

[5] Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. "Large Language Models Can Self-Improve". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 1051–1068. DOI: 10.18653/v1/2023.emnlp-main.67.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, June 2017, pp. 1321–1330. URL: http://proceedings.mlr.press/v70/guo17a/guo17a.pdf.

[7] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. *Language Models (Mostly)*

Know What They Know. arXiv preprint arXiv:2207.05221. Nov. 2022. DOI: 10.48550/arXiv.2207.05221.

[8] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. *Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs*. arXiv preprint arXiv:2306.13063. 2023. DOI: 10.48550/arXiv.2306.13063.

[9] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. "Overcoming catastrophic forgetting in neural networks". In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3521–3526. DOI: 10.1073/pnas.1611835114. eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.1611835114.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. 2022, pp. 24824–24837.

[12] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. "Self-Consistency Improves Chain of Thought Reasoning in Language Models". In: *International Conference on Learning Representations (ICLR 2023)*. 2023. URL: https://openreview.net/forum?id=1PL1NIMMrw.

[13] Potsawee Manakul, Adian Liusie, and Mark Gales. "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557.

[14] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. "Detecting Hallucinations in Large Language Models Using Semantic Entropy". In: *Nature* 630.8017 (2024). PMID: 38867046, pp. 625–630. DOI: 10.1038/s41586-024-07421-0.

[15] M. Nandakishor. "Confidence-Aware Routing for Large Language Model Reliability Enhancement: A Multi-Signal Approach to Pre-Generation Hallucination Mitigation". In: *arXiv preprint arXiv:2510.01237* (2025).

[16] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity". In: *Proceedings of the 2024 Annual Conference of the*

North American Chapter of the Association for Computational Linguistics (NAACL 2024). 2024. URL: https://arxiv.org/abs/2403.14403.

[17] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*. Association for Computational Linguistics, 2023, pp. 9802–9822. DOI: 10.18653/v1/2023.acl-long.546.

[18] Yile Wang, Peng Li, Maosong Sun, and Yang Liu. "Self-Knowledge Guided Retrieval Augmentation for Large Language Models". In: *Findings of EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 10303–10315. DOI: 10.18653/v1/2023.findings-emnlp.691.

[19] Jinheon Baek, Akshay Chandrasekaran, Silvio Lin, Sung Ju Hwang, and Jaime Carbonell. "Probing-RAG: Self-Probing to Guide Language Models in Selective Document Retrieval". In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025.

[20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

[21] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. "FEVER: a Large-scale Dataset for Fact Extraction and VERification". In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Ed. by Marilyn Walker, Heng Ji, and Amanda Stent. New Orleans, Louisiana: Association for Computational Linguistics, June 2018, pp. 809–819. DOI: 10.18653/v1/N18-1074.

[22] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. "TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, July 2017, pp. 1601–1611. DOI: 10.18653/v1/P17-1147.

[23] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. "CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge". In: *Proceedings of NAACL-HLT 2019*. Association for Computational Linguistics, 2019, pp. 4149–4158. DOI: 10.18653/v1/N19-1421.

[24] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. "Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies". In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 346–361. DOI: 10.1162/tacl_a_00370.

[25] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. "Active Retrieval Augmented Generation". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 7969–7992. DOI: 10.18653/v1/2023.emnlp-main.495.

[26] Quinn McNemar. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages". In: *Psychometrika* 12.2 (1947), pp. 153–157. DOI: 10.1007/BF02295996.

[27] Sture Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.